



面向 AI 智能体的操作系统内核

AgentOS-uCore: 基于 RISC-V uCore 的系统原生智能体运行机制

项目名称	面向 AI 智能体的操作系统内核
队伍名称	happy-legend
目标平台	RISC-V 64 / uCore / QEMU
文档版本	决赛设计开发文档

姓名	联系方式 (QQ)
王浩沣	2091576055
康俊豪	488718235

2026 年 08 月

摘 要

智能体工作流已经从单轮问答发展为持续的工具调用、文件处理和多角色协作。现有应用层框架能够安排模型和工具，却往往只在应用日志里保存角色、工作流代次、上下文因果关系和资源归属。当工具真正写入文件、发送消息或申请 CPU 服务时，内核看到的仍是普通进程、文件和 IPC，无法据此作出一致判断。

本项目名称为“面向 AI 智能体的操作系统内核”，基于 LearningOS/uCore 实现 AgentOS-uCore。系统沿用 RISC-V uCore 的进程、虚拟内存、VFS、系统调用和调度器，并把 Agent（智能体）登记为内核能够识别的工作负载。Agent 进程创建与地址空间设计把低频的身份、配额和生命周期状态留在 PCB，将高频读取的 Context 镜像和用户缓存映射到固定用户地址；Agent-OS 内核结构化交互接口以 26 项工具目录、schema（参数模式）、执行约定（Execution Contract）和任务通道承接工具请求；上下文路径、来源追溯（provenance）和执行记录保存请求、结果与数据来源；文件查询、事件等待和工作流 EEVDF 则分别处理属性检索、Agent Loop 唤醒和 CPU 服务分配。

在集成层，AgentOS 控制台按固定步骤执行科研恢复工作流；AgentOS Nexus 则在同一次 QEMU 启动中持续运行协调、系统和研究三个业务角色。Nexus 的 child Task 由 Coordinator 经原生 delegated Task Channel 委派，System 或 Research 领取后把结果写为 Guest artifact（任务结果文件），Coordinator 从唯一 terminal CQE 结算。工作区工具先在 Guest 的 Metadata Catalog 中登记 Host 提供的版本化 manifest，再由 Live Query 选择候选，并用 Typed Watch 处理 generation 变化；Host 只在已选候选中匹配正文或返回指定 revision 的字节。真实结果回到 Guest 后进入 Research artifact 与 Context，跨轮历史也由 Guest Context active path 重建，Host 不私建或替换 Provider 历史。

以下性能结果固定对应冻结在源码提交 2b14fb 的可复现测量批次；当前版本后续功能改动另由功能回归验证，未重新计入这组数据。该批次使用 30 次相互独立的全新 QEMU 启动，保留 33 份原始串口输出、19 份 CSV 数据表和 7,498 条结构化记录。在 16 组独立工作流配对中，带索引核心查询的耗时均低于遍历查询，配对加速比中位数为 **3.118 倍**；在同样 16 组配对的端到端窗口中，索引路径仅有 **3/16 组**更快，其耗时减去遍历路径的中位差值为 **+13.452 ms**，说明核心查询之外的准备与收尾开销尚未得到控制。文件查询网格覆盖目录规模与命中数的 12 种组合，任务通道保留 96 条包含 16 个操作的序列；EEVDF 的 504 条唤醒探针均在 0–1 个时钟节拍（tick）内获得服务。

设计主题	内核机制	可观察行为
Agent 进程创建与地址空间设计	PCB 扩展、受信任映像、七页 Agent Context 区	普通进程与 Agent 进程共存；Guest 可直接读取六页内核镜像，并使用一页用户缓存。
Agent-OS 内核结构化交互接口	工具目录、V2/V3 请求和固定大小返回结构	工具可发现，参数按 schema 解码；未知工具、类型错误和权限不足均返回明确状态。
工具调用协议	Execution Contract、Batch、任务通道 SQ/CQ、原生跨 Agent 委派	同步短调用、带约定调用和有界队列共用一套工具语义；child Task 通过 claim/complete、协作式终态确认和唯一 terminal CQE 结算。
上下文路径管理	128 条有界路径、只读镜像、分支和回滚	连续调用形成可直接读取的 active path；Nexus 从该路径重建 USER、TOOL 与成功 FINAL，失败或取消时回滚。
面向 Agent 查询优化的文件系统扩展	Metadata Catalog、选择性索引、Live Query 和 Typed Watch	Agent 按属性选择 Host manifest 候选；Host 只返回候选的版本绑定字节，正文再作为 Guest artifact 和 Context 使用。
Agent Loop 内核运行机制	心跳、事件等待、消息路由、工作流 EEVDF	无事件时线程休眠，消息或 TIMER 到达后唤醒；多个工作流按实体而非线程数获得服务。
综合场景	恢复控制台和 Nexus 三角色协作	同一 RV64 Guest 中完成 Context 驱动的多轮交互、原生 Task Channel 委派、版本化工作区读取与会话关闭。

表 1: AgentOS-uCore 的系统能力与可观察行为

目录

摘要	ii
第一章 项目背景	1
1.1 项目背景与设计目标	1
1.1.1 智能体对操作系统的新需求	1
1.1.2 基于 uCore 的系统扩展	1
1.1.3 机制与策略的分层	1
1.1.4 相关机制与技术选择	3
1.2 设计目标与阅读线索	3
第二章 系统设计	4
2.1 AgentOS-uCore 总体架构	4
2.1.1 以 uCore 为底座的产品分层	4
2.1.2 六项核心机制与请求流向	5
2.1.3 一次智能体工具调用	5
2.1.4 上层 runtime	5
2.2 跨模块状态约束	6
第三章 系统实现	7
3.1 Agent 进程创建与地址空间设计	7
3.1.1 从普通进程到 workflow 中的 Agent	7
3.1.2 身份字段与角色策略	8
3.1.3 Agent Context 区的七页布局	8
3.1.4 生命周期代次与并发控制	9
3.1.5 随 workflow 创建的资源主体	9
3.1.6 验证结果	9
3.2 Agent-OS 内核结构化交互接口	9
3.2.1 工具调用协议与工具目录	9
3.2.2 自适应的 V2 调用	10
3.2.3 四种执行路径	10
3.2.4 工具执行期间的资源使用租约	10
3.2.5 带类型信息的任务通道	11
3.2.6 原生 delegated Task Channel	11
3.2.7 验证结果	12
3.3 上下文路径管理、数据来源与 Workflow Fence	12
3.3.1 从多轮调用到 Context Path	12
3.3.2 分支、回滚与有界淘汰	13
3.3.3 数据来源传播	13
3.3.4 工具清单与副作用检查	13
3.3.5 执行记录环	14
3.3.6 Workflow Fence	14
3.3.7 验证结果	14
3.4 面向 Agent 查询优化的文件系统扩展	14
3.4.1 从路径访问到属性查询	14
3.4.2 元数据与 inode 代次	14
3.4.3 遍历查询与选择性索引	15
3.4.4 带类型信息的实时订阅与缺口重新同步	15
3.4.5 查询性能	15
3.4.6 验证结果	17
3.5 Agent Loop 内核运行机制	17
3.5.1 从轮询循环到内核等待	17
3.5.2 心跳与模型请求关联号	17
3.5.3 工作流资源账户	18
3.5.4 按工作流汇总的 EEVDF	18
3.5.5 唤醒延迟与公平性	18
3.5.6 验证结果	19
3.6 AgentOS 控制台综合工作流	19
3.6.1 科研恢复场景	19
3.6.2 遍历与索引路径的配对实验	19
3.6.3 核心查询与完整流程窗口	20
3.6.4 系统结果	21
3.7 综合场景：AgentOS Nexus 多智能体协作平台	21
3.7.1 长驻会话与三角角色协作	21
3.7.2 一次交互回合	21
3.7.3 原生 delegated Task Channel	22
3.7.4 Guest Catalog 与 Host 工作区边界	23

3.7.5	artifact、来源与 Context	23
3.7.6	跨轮 Context 路径	24
3.7.7	工具发现与内核观测	24
3.7.8	运行验证	24
3.8	关键实现	24
3.8.1	启动、身份与 workflow 代次	24
3.8.2	结构化工具调用与执行约定	28
3.8.3	上下文、回滚边界与数据来源	36
3.8.4	元数据查询与实时查询事件	40
3.8.5	任务通道与等待路径	46
3.8.6	资源使用租约与 workflow EEVDF	58
3.8.7	Guest runtime（客户机运行时）与结果文件发布	66
第四章	项目测试	71
4.1	功能与性能测试	71
4.1.1	测试组织	71
4.1.2	任务一至任务五的 Guest 综合验收	72
4.1.3	结构化工具与任务通道测量	72
4.1.4	结果文件发布与冲突处理	73
4.1.5	文件查询路径的 I/O 观测	73
4.1.6	测量数据与可重复性	74
4.2	测试路径与结果	75
4.2.1	Guest 原生工作负载	75
4.2.2	工具、执行约定与任务通道测试	77
4.2.3	Nexus 会话与 Host Relay	83
4.2.4	失效状态与恢复分支	85
第五章	总结与展望	86
5.1	阶段性结论	86
5.2	当前技术限制	86
5.3	后续工作	86
第六章	参考说明	87
6.1	uCore 与系统基础	87
6.2	协议、工具与 Runtime	87
6.3	测量资料	87
6.4	文献使用	87
第七章	参考文献	88

1. 项目背景

智能体的“计划-调用-等待-修正”循环会同时经过进程、文件、系统调用和调度器。若角色、 workflow 标识和来源关系只保存在应用层，内核看到的便只能是无差别的线程和文件描述符，因而无法在创建、副作用提交、资源占用或唤醒服务等时点作出一致判断。本章依次讨论 AgentOS-uCore 所要解决的问题、uCore 的扩展边界和设计目标。

1.1. 项目背景与设计目标

1.1.1. 智能体对操作系统的新需求

大模型工具调用已从单次问答发展为持续运行的智能体循环。典型工作流会反复执行“读取上下文、选择工具、等待结果、修正计划”，并将任务拆分后委派给多个专职智能体。这类负载具有三个突出特点：生命周期较长，工具副作用丰富，协作过程中会产生大量中间状态。

传统进程抽象记录 PID、内存、文件和 CPU 时间，却无法直接表达智能体的角色、工作流代次、上下文因果关系、工具模式和数据来源。若这些信息只保存在提示词、JSON 格式文本和应用日志中，内核便难以在文件、IPC 与资源操作实际发生前进行统一检查。单一角色还可以通过创建更多线程扩大调度份额，进而影响其他工作流获得服务的时间。

我们将 AgentOS-uCore 的目标概括为：**使操作系统能够识别智能体工作负载，并在文件、消息和资源等副作用发生时提供一致的身份、资源、因果与调度机制。**

1.1.2. 基于 uCore 的系统扩展

项目建立在 LearningOS/uCore 教学内核之上 [1]。uCore 提供 RISC-V 启动流程、进程、虚拟内存、VFS、异常处理、系统调用和基础调度机制。我们保留这些主干，只在现有 PCB、VFS 对象、调度队列和系统调用路径中补充智能体状态。所有新用户态接口均使用定长结构，并明确写入版本号和结构大小，再由内核和用户态共同检查布局。普通 uCore 进程仍沿用原有路径；智能体进程则通过受控的创建和 exec 路径取得内核身份。

文件元数据采用当前启动周期的内存附属记录，并与实际的 dev+inum+incarnation 绑定。这样既不改变原有 uCore 磁盘格式，也能分别维护索引、订阅和增量缺口后的重新同步。

1.1.3. 机制与策略的分层

文件、消息、资源和调度等必须由内核裁定的动作留在内核；目标拆分、历史整理和工具选择留在 Guest 用户态；传输加密、服务接口密钥和模型服务协议则留在 Host（宿主机）侧。这样，无论使用固定策略、严格回放还是实际模型服务，Guest 都经过同一条内核调用路径。

系统对象	uCore 基础	新增机制
进程与线程	PID、内存、文件表、运行状态	智能体身份、角色与能力位、控制标识、工作流生命周期
虚拟内存	页表、用户页与内核页	六页只读的上下文镜像与一页用户缓存
文件系统	inode、目录、读写与 fsync	以本次启动为范围的元数据、选择性索引、带类型信息的实时查询
系统调用	参数拷贝、执行与返回	结构化工具目录、带类型信息的 V2 与 V3、任务通道的 SQ 与 CQ
进程间通信	管道、信号与基本等待	消息路由、订阅与等待、心跳、模型请求关联号
调度与资源	线程运行队列	工作流调度实体、资源账户、EEVDF 的滞后量与虚拟截止时间

表 1.1: uCore 基础与 AgentOS-uCore 的扩展

层次	主要对象	职责
Host 侧	命令行、观察程序、模型服务提供方 (Provider)、工作区 broker、QEMU 串口	负责交互、传输、加密和消息格式转换；在显式 root 边界内提供 manifest 与版本绑定字节，不选择文件或改写 Provider 历史
Guest 用户态	协调智能体、专职智能体、Context active path、Metadata Catalog 窗口与 artifact 存储	负责拆分目标，通过原生 Task Channel 委派任务，选择工作区候选，并把真实结果写回 artifact 与 Context
AgentOS 内核	身份、上下文、工具、Task Channel、VFS、IPC、资源、调度器	负责身份与 TASK route 检查、任务 claim/complete、数据来源记录、等待、资源记账与调度

表 1.2: AgentOS-uCore 的三级职责划分

1.1.4. 相关机制与技术选择

资源记账借鉴 Linux 的 `cpuacct` 与 `rstat` 分层汇总思路 [2, 3]，执行记录借鉴 BPF 环形缓冲区的有序提交方式 [4]，调度采用 EEVDF 的滞后量与虚拟截止时间概念 [5]。任务通道采用 `io_uring` 的 SQ 与 CQ 通信形式 [6]，文件元数据和选择性索引参考 Haiku BFS[7]。智能体工具循环中“模型选择、应用执行、结果回传”的职责划分借鉴现有工具调用实践 [8, 9]。

在 `uCore` 中，身份、上下文、VFS 访问范围、资源账户和 EEVDF 都记录同一个 `id+generation`。因此，文件查询、工具调用和调度器在判断某个智能体归属时会得到同一答案。

1.2. 设计目标与阅读线索

项目没有另起一套脱离 `uCore` 的 Agent runtime（运行时），而是把赛题关注的能力放进现有内核对象。Agent 进程创建与地址空间设计负责回答“谁在运行、哪些数据需要内核保护、哪些数据应由用户态直接读取”；Agent-OS 内核结构化交互接口与工具调用协议负责表达请求及其结果；上下文路径和文件查询保留每轮行动所依据的信息；事件等待与工作流调度负责 Agent Loop 的休眠、唤醒和 CPU 服务。每条路径都有相应的 Guest（客户机）程序和 Host（宿主机）侧验证，性能比较只使用工作负载和结果指纹一致的样本。

智能体运行环节	内核提供的机制	文中可核对的现象
创建与装入	受信任映像、智能体身份、控制标识、七页 Context 区、生命周期代次	Agent 与普通进程共存； <code>create</code> 、 <code>fork</code> 、 <code>exec</code> 、 <code>exit</code> 后的身份和映射保持一致
发起工具调用	工具目录、V2/V3、参数 schema、执行约定、任务通道 SQ/CQ	Guest 可列举并调用工具；错误参数、过期请求、重试、取消和背压得到不同结果
积累并检索信息	上下文路径、来源追溯、文件元数据、摘要、索引与订阅	路径可直接读取并有界淘汰；按属性查询与遍历结果一致，索引查询减少候选扫描
等待下一轮与协作	消息路由、原子等待、心跳、资源账户、工作流 EEVDF	无事件时休眠，事件到达后唤醒；线程数增加不会等比例扩大工作流的服务份额

表 1.3: 从 Agent Loop 的运行环节到内核可观察行为

2. 系统设计

AgentOS-uCore 沿用 uCore 的进程、页表、VFS、异常处理、系统调用和调度主干，不以独立运行时（Runtime）取代内核对象。一个智能体发起工具调用时，身份检查、上下文记录、文件访问、消息投递、资源记账和调度选择都读取进程中保存的同一组 workflow 编号和代次。图示、数据结构和状态机明确规定这些模块何时读取字段、何时更新状态，避免同一请求在不同位置被认作不同工作流。

2.1. AgentOS-uCore 总体架构

2.1.1. 以 uCore 为底座的产品分层

AgentOS-uCore 按部署职责分为五部分：uCore 基础内核、AgentOS 内核模块、Guest Runtime、Host Relay 和 Agent 应用。图2.1自下而上展示这五部分：uCore 提供进程、内存、VFS、系统调用和调度器等基础对象；AgentOS 在这些对象上补充智能体身份、工具、Context、文件查询、Loop 和工作流调度；Guest Runtime 封装统一 UAPI，Host Relay 连接串口、TLS 与模型 Provider，控制台、Nexus 与科研工作流则负责具体任务。

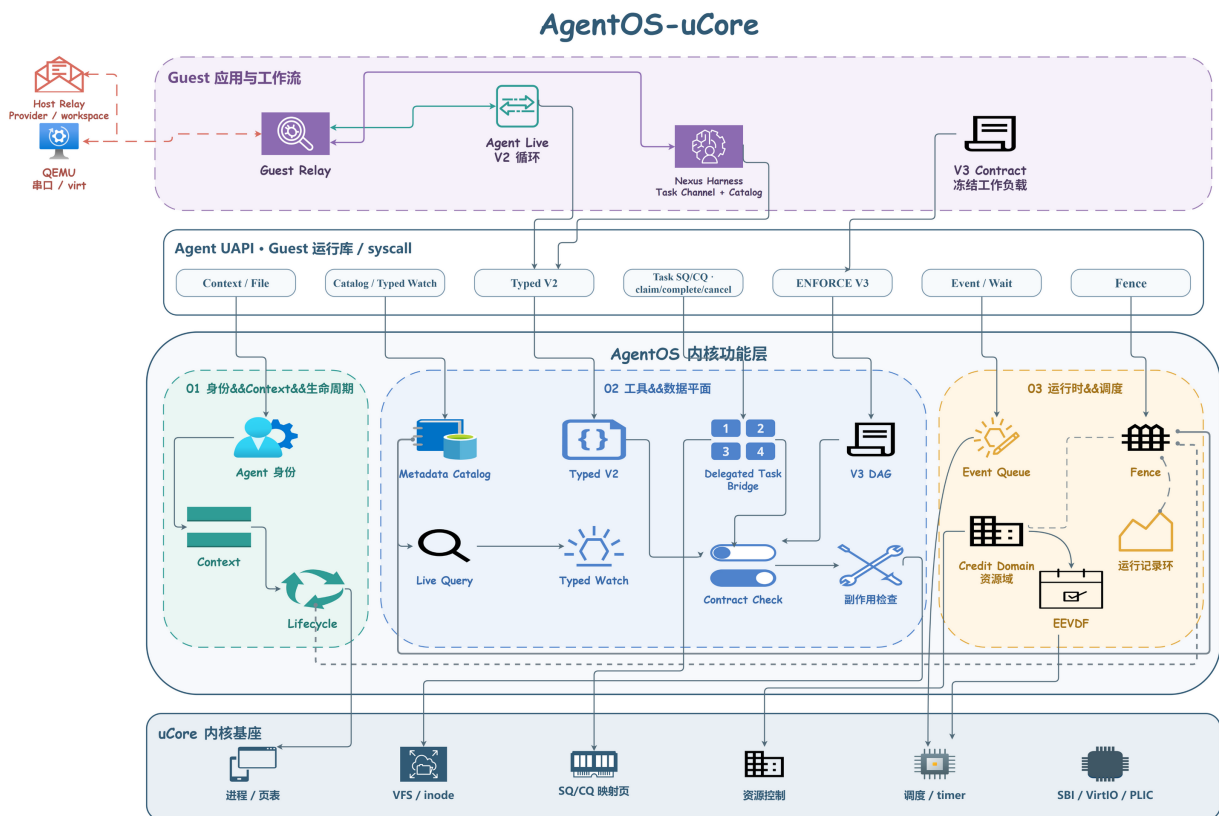


图 2.1: AgentOS-uCore 总体架构

底层 uCore 主干继续承担进程、页表、VFS、异常处理、系统调用和调度器的原有职责。AgentOS 在此基础上把相关机制分为三组：

1. 身份、上下文和资源：身份与生命周期登记智能体；上下文保存每次工具交互；资源

账户和 EEVDF 分别记录资源使用并分配 CPU 服务。

2. **工具、文件和消息**：带类型信息的工具调用检查 schema 和能力位；VFS 元数据与实时查询处理文件对象；消息路由传递跨智能体消息。
3. **来源记录和工作流结束**：上下文路径记录因果关系；来源追溯随文件、工具和 IPC 传播；执行记录环与 Workflow Fence（工作流栅栏）分别保存阶段记录和结束时的快照。

2.1.2. 六项核心机制与请求流向

模块	关键对象	主要功能
Agent 进程与地址空间	身份记录、七页 Context 区、控制标识、生命周期	创建受信任 Agent，绑定角色、能力位和工作流代次，并把高频状态映射给 Guest
结构化交互与工具协议	工具目录、V2/V3、SQ/CQ	发现工具、检查带类型信息的参数，并支持动态调用、冻结执行约定和有界任务提交
上下文路径与来源追溯	路径镜像、分支、执行记录环	连接请求、结果、前驱记录和数据来源，并按配额淘汰旧记录
面向 Agent 的文件查询	元数据、摘要、索引、订阅器	为显式登记的文件提供属性查询、结构化结果、变更通知和缺口重新同步
Agent Loop	事件队列、路由、心跳	提供原子等待与唤醒、消息、模型请求关联、超时和取消
工作流资源与调度	资源账户、EEVDF 实体	按工作流汇总资源记账，并根据滞后量和虚拟截止时间分配 CPU 服务

表 2.1: AgentOS 的核心模块

六项机制使用同一个工作流 id+generation。一次工具调用会同时取得调用者身份、前一条上下文记录、schema、数据来源、资源账户和调度实体。工作流代次变化后，已经回收槽位对应的订阅器和句柄不能再指向新对象。

2.1.3. 一次智能体工具调用

一次完整调用从 Guest 读取工具目录开始。Guest 按角色列出可用工具，再将模型结果编码为带类型信息的请求。请求进入内核后，AgentOS 依次拷入数据，检查版本和大小，检查身份与生命周期，解析工具并解码 schema，再检查执行约定、数据来源、资源和副作用，最后调用 VFS、IPC 或其他 uCore 子系统。返回前，上下文路径写入一条新记录，下一轮可引用其序号。

当智能体等待模型、文件事件或其他角色时，agent_wait 将线程置于等待状态。事件入队与等待者安装按原子顺序执行，从而避免“发现队列为空后恰好错过唤醒”。

2.1.4. 上层 runtime

控制台按固定策略依次执行身份检查、上下文记录、文件查询、IPC 和恢复操作，用于功能演示和配对测量。Nexus 使用同一组系统调用维持会话、分派任务、整理结果文件并连接模型服务。两者访问同一批 AgentOS 内核对象，因此确定性测试和自然语言交互遵循相同的系统规则。

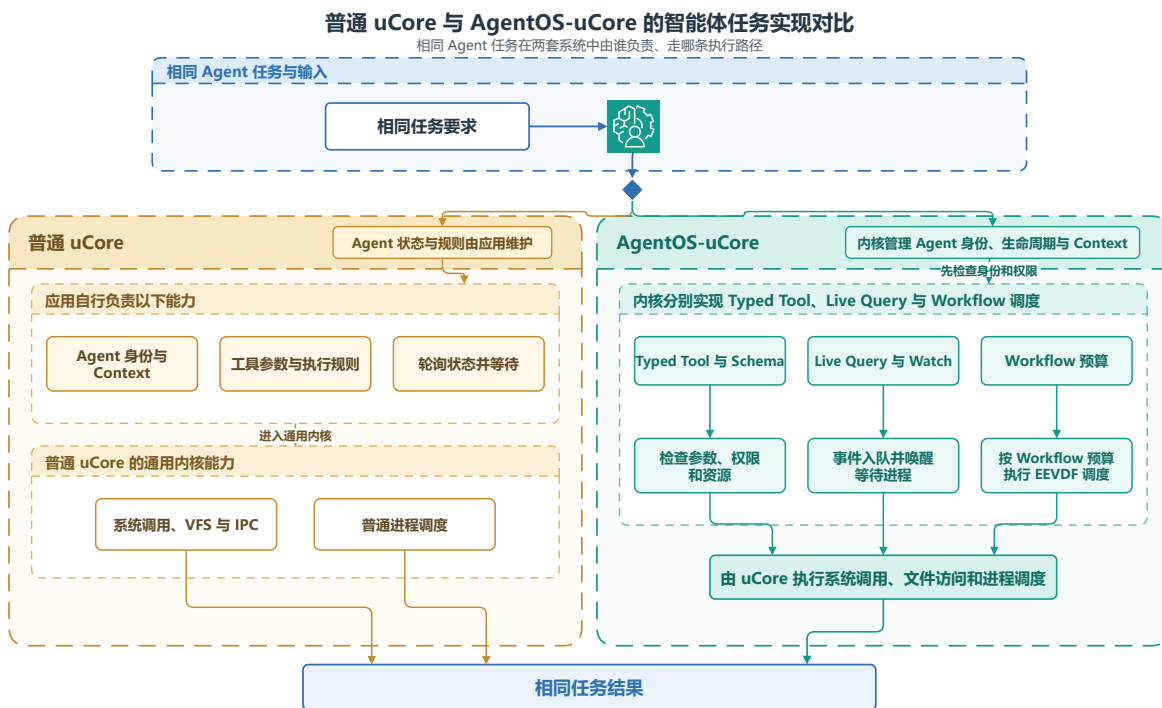


图 2.2: 同一科研恢复任务在普通 uCore 与 AgentOS-uCore 中的执行路径

图2.2的比较目标是说明：业务步骤不变时，AgentOS 把哪些原本散落在 runtime 中的约束交给了内核。普通 uCore 仍可用进程、文件和 IPC 串起科研恢复任务，但角色、上下文和等待关系只能由应用自行约定；AgentOS-uCore 在相同流程中加入可识别的工作流标识、结构化工具、选择性索引、事件等待和工作流调度，使身份检查、数据来源和资源归属贯穿每个阶段。

2.2. 跨模块状态约束

系统将工作流标识写为可复用槽位的 id+generation。模块使用该标识前，先确认对象仍在运行、访问范围相同且代次匹配；资源、元数据和执行记录也按该标识归属。这样，旧槽位关闭、回收并再次使用后，先前的请求不会被新工作流接收。

算法 1：工作流标识的跨模块校验

输入：调用进程、目标对象、请求携带的工作流编号和代次、访问范围。

输出：可继续执行的对象引用，或明确的 STALE、DENIED 结果。

1. 从调用进程读取内核保存的工作流编号、代次和能力位。
2. 检查工作流编号和代次的结构，确认生命周期仍处于活跃状态并且代次匹配。
3. 比对调用者、目标对象和请求的访问范围；如不一致，则拒绝提交副作用或传递事件。
4. 在工具、元数据、任务通道、执行记录或调度路径中，分别执行相应的参数和资源检查。
5. 只有在全部检查通过后，才发布状态、将事件入队或提交资源。

3. 系统实现

本章依据当前仓库中的内核和用户态源代码。内核代码主要位于 `os/agent_*.c`、`os/workflow_*.c` 和 `os/resource_controller.c`；用户态场景位于 `user/src/`，Nexus 还使用 `user/lib/agent_nexus.c` 和 `host_tools/agentos_relayd.py`。各节先说明要解决的问题和内核采取的动作，再给出可直接定位的源代码片段。

实现部分	主要源文件	关键责任
启动与身份	<code>os/main.c</code> 、 <code>os/agent_identity.c</code> 、 <code>os/workflow_lifecycle.c</code>	初始化智能体子系统，分配角色、控制标识和工作流生命周期。
上下文与执行	<code>os/agent_context.c</code> 、 <code>os/agent_core.c</code> 、 <code>os/agent_execution_contract.c</code>	写入上下文，解析 V2 与 V3 请求，并按执行约定检查副作用。
任务与来源	<code>os/agent_task_channel.c</code> 、 <code>os/agent_provenance.c</code> 、 <code>os/agent_evidence_ring.c</code>	管理任务通道的 SQ 与 CQ、来源标签和工作流执行记录的阶段快照。
文件与事件	<code>os/agent_metadata_*.c</code> 、 <code>os/agent_file_state.c</code> 、 <code>os/fs.c</code> 、 <code>os/agent_live_query_events.c</code> 、 <code>os/agent_ipc.c</code>	维护元数据目录、查询与订阅、结果文件发布、消息和等待。
资源与调度	<code>os/resource_controller.c</code> 、 <code>os/workflow_credit_domain.c</code> 、 <code>os/workflow_scheduler.c</code>	处理资源使用租约、额度、工作流级选择和资源记账。
集成场景	<code>user/src/labdemo_ucore.c</code> 、 <code>user/src/agent_nexus_ucore.c</code>	驱动恢复流程和 Coordinator、System、Research 三角色常驻协作。

表 3.1: AgentOS-uCore 的实现源代码分工

3.1. Agent 进程创建与地址空间设计

3.1.1. 从普通进程到工作流中的 Agent

AgentOS 区分普通进程、协调 Agent 和专职 Agent。VFS、IPC、工具执行器和调度器读取同一份身份记录，因此同一个进程在各处拥有相同的角色和能力位。内核将 Agent 身份写入 PCB，再根据受信映像清单和父角色授权完成创建。用户态可以读取身份信息，权限判断始终以内核副本为准。普通进程仍走 uCore 原有的创建与装入路径，两类进程可以在同一系统中并存。

创建受控工作流时，内核先分配生命周期，再写入身份、角色、能力位和控制标识，随

后建立 **Agent Context** 页、VFS 访问范围和资源账户。任一步骤失败，内核都会回收已经分配的对象。创建完成后，各子系统都通过同一组 workflow 编号和代次找到该 **Agent** 所属的工作流。

用户态的 `agent_create()` 和 `agent_create_role()` 进入内核的 **Agent** 创建路径：`agent_info()` 从 **PCB** 返回身份、角色、配额、**Loop** 状态和 **Context** 位置，`agent_workflow_lifecycle_info()` 再给出完整生命周期键。创建接口负责建立对象，查询接口只复制内核已经保存的元信息，两者都不会让用户态自行填写权威身份。

`agent_worker_create()` 不会新建 workflow 根生命周期。它只在已有生命周期中创建一个携带受控待执行映像的子进程；调用者只要在同一 workflow 内具有 **ORCHESTRATE** 即可调用，不要求它必须是控制者。子进程仍须沿通常的 `exec()` 路径装入映像，所以该调用返回时，子进程尚未在新映像中运行。`agent_scope_delegate_fd()` 只接受 **FD_INHERIT_DELEGATE**，当前实现用于管道。该接口不会立即把文件描述符放入某个访问范围，而是在调用线程上登记一次性凭据；下一次创建受控子进程时，内核制作创建快照才会使用该凭据。子进程通过 `filedup()` 获得文件描述符副本，父进程继续保留原描述符，随后内核清除这张凭据。

3.1.2. 身份字段与角色策略

字段	含义	使用模块
<code>agent_id</code>	当前启动周期内的智能体编号	工具、上下文、观测
<code>control_id</code> 角色与能力位	内核分配的控制身份 角色及其具体权限集	控制者绑定、 IPC 路由、句柄校验 工具、文件、消息与 artifact （工作流结果文件）操作
工作流 <code>id+generation</code>	工作流标识和可复用槽位的代次	生命周期、订阅、任务通道、资源、调度
VFS 访问范围	工作流的文件访问范围	元数据、查询、 artifact
执行与存储账户	工作流资源归属	进程、线程、文件、块、页与缓冲区

表 3.2: 智能体身份的核心字段

内核的角色策略将每个角色映射为能力位掩码和调度权重。哨兵角色主要读取进程、元数据和调度状态；调查角色只能读取受限内容；报告角色可以创建报告结果文件；协调角色可以分派任务并创建角色。子智能体继承同一工作流的访问范围，其能力位同时受父角色授权和映像清单限制。

3.1.3. Agent Context 区的七页布局

每个 **Agent** 在用户地址空间的固定 **ABI** 地址拥有七页 **Agent Context** 区。前六页由内核维护，并以只读方式映射到 **Guest**，依次提供身份头、最近返回、上下文记录、路径索引和稳定摘要；第七页可由用户态读写，用于保存查询结果和其他高频策略数据。这样，身份、能力位、访问范围等权威状态仍受内核保护，**Guest** 读取 **Context** 和管理查询缓存时又不必为每个字段反复进入内核。

上下文最多保存 128 条记录。每条记录包含序号、请求、工具、状态、服务起始时钟、原因序号、跨度、分支代次、控制标识和记录摘要。内核按发布序号更新镜像；**Guest** 既可

以调用查询与快照接口，也可以直接读取同一结构。PCB 保存身份、配额、Loop 状态和路径位置等低频元信息，具体路径则通过 Context 区向用户态发布，这一划分兼顾了内核裁定与高频读取。

3.1.4. 生命周期代次与并发控制

workflow槽位可在回收后复用，内核每次重新分配时都会推进代次。订阅器、资源账户、VFS 附属记录、任务句柄和 artifact 都同时保存标识和代次。

生命周期使用普通操作、退出操作和 Workflow Fence（工作流栅栏）三类并发控制。普通系统调用增加普通操作计数；exit 或拆除过程增加退出操作计数；Workflow Fence 在两类操作全部完成后生成阶段快照。当控制者已经关闭、成员数、活跃普通操作数和退出操作数都归零，并且栅栏已经释放时，回收程序会清除该生命周期的上下文、订阅器和执行记录环。普通 VFS 访问范围会被完整回收；对于 preserve_on_retire 持久输出访问范围，只要存储账户进入 DRAINING 或 FREE，内核即可回收生命周期代次，同时保留登记项、文件和存储额度。

3.1.5. 随工作流创建的资源主体

工作流创建时，内核同时分配执行账户和存储账户。工作流中的进程、线程、文件对象、文件系统块、inode、智能体状态页和物理页都记入同一账户。这样，资源使用租约和工作流 EEVDF 都按工作流计算，子进程和多线程也始终记在原工作流名下。

3.1.6. 验证结果

agenttrust_ucore 覆盖受信 exec、角色授权、能力位收缩以及 fork、exec、exit 与普通进程共存；agentscope_ucore 覆盖访问范围与代次复用；agentfinal_ucore 覆盖上下文固定映射、快照、分支、回滚和容量淘汰。所有场景都在 RV64 uCore Guest 中通过真实系统调用运行。

3.2. Agent-OS 内核结构化交互接口

3.2.1. 工具调用协议与工具目录

Agent 既要发现可用能力，也要把工具名称、参数和返回状态交给内核可靠解析。AgentOS 因此没有让 Guest 传入任意 JSON 文本，而是定义了带版本和大小字段的工具调用协议。内核维护 26 项工具目录，每个描述项写明工具编号、名称、schema（参数模式）、标志和副作用类别；第 26 项是原生 delegate_task。启动时检查编号与名称是否唯一，并检查 schema 是否完整。

请求使用工具名称或编号加一组 typed 键值参数，响应返回状态码、Context 序号、结果类别和定长结果字段。版本不符、参数类型错误、工具不存在、能力不足和生命周期过期分别返回可区分的状态；用户态不需要从一段错误文本中猜测失败原因。目录项和请求结构都带 version 与 size，后续扩展可以在明确拒绝旧布局的前提下进行。

sys_tool_list 向 Guest 返回定长描述项。Guest 再按角色和能力位列出模型可见的工具，并补充用途、参数和结果说明。等待、上下文和模型请求等 runtime（运行时）接口

只供 Guest 使用，模型只能请求当前角色允许的产品动作。

3.2.2. 自适应的 V2 调用

V2 请求明确携带版本号、请求大小、工具名称、工具编号、参数数量和带类型信息的参数数组。当前标量类型为 uint64 与字符串，适合 uCore 的紧凑参数解析。内核先把完整参数拷入内核缓冲区，依次检查参数名、类型、数量和长度，再进入共享执行器。

V2 适合根据上一轮结果改变下一步操作的智能体循环。例如，协调智能体可以先查询失败阶段，再根据候选数决定读取摘要还是创建研究子任务。每次调用都会独立检查身份、能力位、访问范围、schema、数据来源和资源状态。

3.2.3. 四种执行路径

路径	输入形式	主要用途	运行语义
紧凑批处理	最多 64 个 agent_op	固定顺序和上下文压力场景	在一次系统调用内顺序执行，逐项返回状态
V2	带类型信息的键值参数	依据上一轮结果选择工具	每次都检查 schema、能力位和访问范围
ENFORCE V3	V2 前缀与 Execution Contract 绑定信息	预先定义的高保障 DAG	冻结节点、前驱、尝试次数、截止时间和副作用清单
任务通道的 SQ 与 CQ	16 槽 SQ 与 16 槽 CQ	有界提交、完成和背压	仅主线程建立通道，绑定提交线程并校验代次；每个已接受任务恰有一条完成 CQE，取消不会额外产生 CQE

表 3.3: 四种工具执行路径

紧凑批处理用于同步执行短操作序列。V3 则在工作流生命周期内冻结一份带版本的执行约定，其中最多包含 24 个按拓扑顺序排列的节点。每个节点关联工具、schema 摘要、所需能力位、输入和输出 artifact 类型、截止时间、尝试次数、来源与副作用清单，以及执行和存储额度。

V3 请求同时绑定执行约定代次、节点、尝试次数、来源节点、前驱上下文序号和 32 字节指纹。无论成功还是失败，完成结果都会写入固定完成槽；合法重试直接读取原结果，避免再次产生副作用。

3.2.4. 工具执行期间的资源使用租约

高风险工具可以在执行前从 workflow 账户预留执行和存储额度。资源使用租约依次经过 ADMITTED->ACTIVE->DEACTIVATED->SETTLED。分配器在发布对象前取得 nonce 声明，发布成功的对象继续占用同一份已用额度，析构时再结算。这样，多个对象在同一轮工具调用中同时达到资源峰值时，账户仍能准确记录用量。

3.2.5. 带类型信息的任务通道

任务通道只能由智能体主线程建立，并映射 16 槽 SQ 和 16 槽 CQ，两者各占一个 Guest 映射页；复制的请求和完成记录、权威资源状态则分别保存在两页内核私有页中。SQE 与 CQE 均为 128 字节。进入通道和资源绑定会记录提交线程标识及身份代次。内核先拷入完整 SQE，再检查进入接口的 ABI、递增请求编号、执行约定、节点、尝试次数、schema、截止时间、取消链接和带类型信息的句柄。提交者、所有者或生命周期不匹配导致 STALE 时，输出只保留 ABI 版本、结构大小和 status，其余字段全部置零；控制标识代次不匹配时，进入接口返回当前通道状态。

内核私有的头尾指针才是权威状态。CQ_FULL 只表示 CQ 暂无可写槽位，是背压，不表示已接受任务已经完成；对应任务仍待发布，直到 CQ 出现可用槽位。SQE 的环或槽位代次不一致，或者发生其他协议错误，都会使通道进入需要重新同步的状态；提交者必须以 AGENT_TASK_CHANNEL_ENTER_F_RESYNC 和空 sq_tail 显式复位，随后才能继续提交。每个已接受任务在成功、拒绝、失败、取消或超时后发布一条完成 CQE。CANCEL 命令使用独立请求编号，只引用目标任务，不会另行生成 CQE；同步策略拒绝仍会消费取消编号，并由进入接口返回 DENIED，目标任务继续保持原运行状态。目标任务已经确认或不存在时，同样会消费取消编号，进入接口返回当前代次和水位线。Task Bridge 既接受空输入，用于验证通道、截止时间和取消和完成语义；也允许 ECHO Provider（服务提供者）读取 UTF-8 资源 snapshot，检查带类型信息的输入能否进入同一个工具执行器。

任务输入也可以由 IMPORT 控制请求从当前进程的只读普通文件描述符导入。请求进入时，内核先固定文件引用和当次 workflow/contract cut；实际读取通过带租约的 readi 从 offset 0 多探测一个字节。只有文件恰好在给出的 1-63 字节处结束、内容不含 NUL 且 UTF-8 合法时，才建立资源。导入提交前，内核在同一条事务 lane 中再次核对 Context 序号、哈希和来源标签，避免把文件内容与另一时刻的 Context 拼成一条来源记录。

导入过程不改变文件描述符的共享 offset，也不把 structfile* 留在通道中；内核只在私有资源槽保存不可变的 inline snapshot、SHA-256、最新 Context 序号和 UNTRUSTED_FILE_DATA 标签。IMPORT 只登记输入，不另行追加 Context 记录。定向 Guest 还让另一线程与导入并发执行 unlink/close，用来检查固定引用和事务边界能够走通。

资源句柄由 slot、type 和 generation 共同标识。BORROWED 输入执行后仍保持 LIVE，可由后续任务继续引用；OWNED 输入在完成后自动消费，显式 RELEASE 也会使旧句柄变为 STALE。ECHO Provider 从内核 snapshot 复制 payload，不再依赖导入文件的后续内容或 offset。若 IMPORT 已成功而最终结果无法复制回用户态，内核会撤销尚未暴露的资源，避免只有内核知道的槽位泄漏。

3.2.6. 原生 delegated Task Channel

跨 Agent child Task 不再封装为用户态消息状态机。Coordinator 把准确的 56 字节 AGENT_ARTIFACT_TASK 描述符导入 Task Channel；描述符绑定目标 PID、Agent 编号、控制标识、任务关联和 Guest capsule handle，大段输入与结果仍由 artifact 承载。delegate_task SQE 进入 Execution Contract 和内核 pending 队列后，只有持有 AGENT_CAP_TASK_ACCEPT 且取得独立 AGENT_IPC_ROUTE_TASK 路由的目标 Agent 才能领取。

目标通过系统调用 agent_task_delegate_claim（编号 567）取得不可变描述符，完成业务与结果 artifact 后，再通过 agent_task_delegate_complete（编号 568）提交终态。

claim 建立的 delegated effect lease 精确绑定执行者 PID、Agent、control_id、线程 identity generation 以及 channel/request/slot 代次；每次直接作用必须是 Execution Contract 副作用清单的子集。正常完成、协作式终态确认或目标进程 quiescence 后，内核才在活动引用归零处撤销该租约。

同一活动生命周期内的 controller 也可复用 complete 接口，以 AGENT_TASK_DELEGATE_COMPLETE_F_REQUEST_CANCEL、CANCELLED、零 ACK 字段和准确的 owner/channel/request/slot/task/correlation tuple 请求取消。内核除核验该完整绑定外，还要求调用者同时持有 AGENT_CAP_ORCHESTRATE、AGENT_CAP_WAIT_CANCEL，并具有指向 owner 的活动 AGENT_IPC_ROUTE_TASK 路由。返回 OK 只确认 control request 已接收，不表示 delegated Task 已经结算，也不替代 terminal CQE。

若该取消、截止时间、owner 退出或生命周期关闭在 claim 后先取得终态优先级，complete 返回带 generation 的终态 offer。处于 CLAIMED 的 provider 仍须先清理预绑定输出，再以 ACK_TERMINAL 和准确 offer 确认；内核随后只向 owner 发布一条 terminal CQE。该首版协议依赖已领取任务的执行者协作，不承诺强制终止永久无响应的 provider。一般 MESSAGE 路由仍可用于进程通信，但不承载 child Task 的正文、状态或结果。

3.2.7. 验证结果

agenttoolabi_ucore 覆盖 26 项目录发现、名称与编号一致性、未知参数、类型错误和返回缓冲区；agentcontract_ucore 覆盖 DAG、截止时间、重试、取消、来源 Context 与来源检查；agenttask_ucore 除了 SQ/CQ 已满、重新同步、取消和完成记录保留，还检查 UTF-8 与 TASK snapshot、BORROWED/OWNED 生命周期、generation 防 ABA、TASK route、claim/complete、controller 取消请求、终态 offer 确认和唯一 CQE。最终 copyout 失败后的回滚分支由结构检查、mutation 测试和资源表模型核对。任务通道的测量数据另保存了 96 条每条 16 个操作的序列，第 4 章集中分析其分布。

3.3. 上下文路径管理、数据来源与 Workflow Fence

3.3.1. 从多轮调用到 Context Path

Agent 的下一步决策往往取决于前几轮工具结果。上下文路径管理把这些请求和结果按前驱关系组织为 Context Path：每次工具调用即将执行可能产生副作用的操作时，AgentOS 建立一条上下文记录，并由附属记录保存归一化的 agent_op、结果和 provenance（来源追溯）信息。附属记录不照搬完整的 V2 参数、V3 绑定或任务通道 SQE，而是保留后续决策真正需要的请求、结果、因果关系和数据来源。

记录序号只在同一次清空周期内递增。记录包含请求、工具、状态、服务起始时钟、原因序号、跨度、分支代次、控制标识、短载荷或结果以及记录哈希；执行清空后，序号从 1 重新开始。path_parent_sequence 指向当前活动头记录，后继索引用于查询分支。

上下文追加依次经过准备、填充、发布和提交；detail 会在记录标为奇数前写入。镜像头发布记录数、头记录、最早记录、最新记录、可见头记录、丢弃数和回滚数。Guest 读取记录后重新核验序号和哈希，并再次确认摘要未变化，便可得到一致快照。

用户态通过 context_push() 追加记录，也可以用 context_query() 复制活动路径；高频读取时则直接检查 Context 镜像。context_rollback() 把可见头移到仍在窗口内的

历史记录，`context_clear()` 清空当前路径。系统调用与直接映射读取使用相同的序号和摘要，因此两种读取方式可以互相核对。

3.3.2. 分支、回滚与有界淘汰

上下文路径最多保留 128 条记录。容量用尽后，内核按序号淘汰最早记录，并更新最早记录、最新记录和丢弃计数。活动路径只引用仍然保留的前驱记录。

回滚将 `visible_head` 移到仍然存在的上下文序号，并为后续调用分配新的分支代次。已经产生的文件、IPC 和模型服务副作用保持不变。需要幂等的副作用由 V3 尝试缓存和具体工具协议处理。因此，上下文分支只改变后续决策看到的路径；手工备注、回滚和清空都不会预留或提交执行记录。

3.3.3. 数据来源传播

AgentOS 使用六类可组合的数据来源标签，标签随上下文、文件、工具返回、IPC 和 artifact 传播。

标签	含义	典型来源
KERNEL_FACT	内核生成的结构化事实	PID、资源、调度器、能力位
TRUSTED_USER_CONTROL	经产品界面提交的用户控制信息	新交互轮次、参数绑定审批
AGENT_DERIVED	智能体根据已有输入推导的数据	摘要、计划、报告
UNTRUSTED_FILE_DATA	来自文件的内容或元数据	查询、读取、摘要哈希
UNTRUSTED_TOOL_OUTPUT	工具执行返回	带类型信息的 V2 或 V3 结果
CROSS_AGENT_DATA	经 IPC 或 artifact 在角色间传递	任务结果、协调者整理的 artifact

表 3.4: 上下文的数据来源标签

文件、工具、邮箱和任务资源按保守的按位或规则传播来源标签。生成摘要、重新计算哈希和在智能体之间整理 artifact 时会增加新来源，同时保留既有标签。因此，文件数据可以供模型查看，但不会自动变成控制授权。

3.3.4. 工具清单与副作用检查

每个内核工具都有安全清单，其中列出可接受的输入来源标签、输出新增标签、所需能力位，以及文件、元数据、IPC、进程、权限、artifact 和订阅的副作用掩码。启用 ENFORCE V3 后，生命周期代次、执行约定中冻结的边、schema、能力位、工具清单和数据来源必须同时匹配。失败请求会在工具产生副作用前返回结构化状态。直接调用或 V3 调用被拒绝时通常记录来源上下文、工具和原因；但是关键记录的预留或提交失败时，调用会返回 NO_SPACE 或 RETRY，因此不能认为每次拒绝都一定有执行记录票号。

3.3.5. 执行记录环

每个活动 workflow 按需分配四个智能体状态页：三页普通记录环，共 48 个 256 字节槽位；一页关键记录环，共 16 个槽位。直接调用或 V3 调用的标准完成或拒绝路径会预留递增的执行记录票号，填充后再提交；手工备注、回滚和清空不使用这套预留和提交规则。当较早编号仍在准备时，读取者会停在当前位置，从而保持全局发布顺序。

普通成功的上下文记录写入普通记录环，获准副作用和成功建立记录的拒绝写入关键记录环。拒绝记录无法预留或提交时，直接调用或 V3 调用返回 NO_SPACE 或 RETRY，而不是伪造关键记录编号。复用环形空间前，已提交分段会汇入内部根记录。该结构为 Workflow Fence 提供按执行记录票号排序的执行记录。

3.3.6. Workflow Fence

控制者通过 `agent_run(count=0, AGENT_RUN_F_FENCE)` 发起 Workflow Fence。内核检查协调者与控制标识的关系，使生命周期进入屏障状态，排空元数据、实时查询延迟工作和 VFS 回收，检查执行与存储账户中的待结算额度归零，最后密封执行记录环。

非空的屏障请求只携带版本号、`struct_size`、必须为零的 `flags`、必须为零的 `reserved`、挑战值和请求编号，**不携带**前序根记录。请求和回执可以同时为 NULL，此时内核执行匿名且不等待结果的屏障。前序根记录只由内核在阶段快照完成后写入返回回执。提供回执时，生成的 320 字节回执绑定前序根记录、挑战值、屏障序号、执行记录票号范围、事件与缺口计数、元数据代次、额度周期、精确已用额度摘要和有序分段摘要。请求编号非零且提供回执时，内核在内部状态提交后复制结果；同一个请求编号再次复制时返回缓存结果。该回执表示当前启动周期内的工作流阶段快照，其标志同时记录元数据是否易失以及覆盖状态。

3.3.7. 验证结果

`agentfinal_ucore` 连续执行 64 次工具调用，并覆盖快照、分支、回滚和窗口淘汰。数据来源场景把文件来源传入受保护的副作用，验证副作用前的拒绝与关键记录环记录。屏障场景覆盖挑战值、重试缓存、待结算额度、元数据代次和结果复制重试。

3.4. 面向 Agent 查询优化的文件系统扩展

3.4.1. 从路径访问到属性查询

科研和恢复 workflow 经常需要回答两个问题：当前哪些对象满足条件，哪些对象刚刚发生变化。传统路径访问适合已知文件名的场景，却不能直接表达“查找某个阶段、状态或类别的结果文件”。AgentOS 在 uCore VFS 之上增加按启动周期隔离的元数据目录，为显式登记的文件维护项目、运行、阶段、状态、类别、依赖关系和内容摘要，使 Agent 可以按属性与摘要查询对象，也可以订阅后续变化。

3.4.2. 元数据与 inode 代次

拥有 META_WRITE 的智能体调用 `agent_file_meta_set()` 写入元数据，字段与真实 `dev+inum+incarnation` 绑定。更新通过 `update_mask` 完成，删除使用显式 DELETE 动

作。文件创建和重命名仍沿用 uCore VFS，只有工作流明确关心的对象才会进入元数据目录。

对象代次用于区分 inode 编号复用前后的对象。VFS 以增量方式维护它，并随查询和订阅结果返回。查询或订阅命中记录后，内核会复核元数据目录代次、生命周期、访问范围和完整条件。unlink、内容大小变化和对象失效都会更新已登记的附属记录。

3.4.3. 遍历查询与选择性索引

元数据目录为 status、stage 和 kind 维护选择性索引。带类型信息的查询只携带条件字段、查询标志和最大命中数；访问范围由调用者身份限定，查询计划由内核选择。结果返回命中数、候选记录数、扫描记录数、全部命中数、实际返回数、truncated、查询计划、计划原因和文件系统代次。调用者可以用 SCAN 强制遍历，也可以用 USE_INDEX 允许内核使用索引。启用索引时，内核按固定的 status->stage->kind 顺序选择第一个已给定字段，不比较字段基数；索引路径仍会对候选记录检查完整条件，因此遍历与索引查询返回相同结果。最大返回数为 8，total_hits 仍报告全部命中数；当前 ABI 没有分页游标。

查询结果是带属性、摘要、digest 和对象代次的定长结构。Guest 可以把输出地址直接放在 Agent Context 区的第七页用户缓存中，由自己的策略决定保留多久；内核只管理这页的映射和配额，不在背后维护另一份不可见的查询缓存。

3.4.4. 带类型信息的实时订阅与缺口重新同步

agent_live_watch() 安装一份完整的 agent_file_query。订阅器只保存查询条件、访问范围和代次，不保存持久结果集。每次元数据变化时，内核重新计算变更前是否满足条件：对象从未命中变为命中时产生 ENTER，对象持续命中且记录改变时产生 UPDATE，对象从命中变为未命中或被删除时产生 LEAVE。事件以 AGENT_EVENT_FILE_QUERY 写入目标智能体的上下文和有界队列。

队列或待处理表无法保存全部增量时，内核会推进 resync_generation 并发送 RESYNC_REQUIRED。正常连续的文件系统代次变化仍只发布 ENTER、UPDATE、LEAVE，不会触发重新同步。恢复时，智能体保留旧订阅，先以相同条件安装不带 ACK_RESYNC 的替换订阅，再执行有界查询。只有 truncated=0 的快照才能作为完整基线；随后才在旧订阅上以对应代次执行 ACK_RESYNC 并取消旧订阅，替换订阅覆盖整个切换时段。单次查询最多返回 8 个命中项，当前没有分页游标；快照一旦截断就不能重新建立基线或确认，调用者必须收紧查询条件，分页留待未来 ABI。重新同步不能把 Workflow Fence 当作通用屏障。关闭工作流只会使生命周期进入关闭状态，不会排空待处理项；屏障会排空删除标记、内容增量并尝试投递标记事件。只要该工作流或访问范围中的重新同步尚未由 ACK_RESYNC 确认，即使标记事件已投递，屏障仍返回 RETRY，最终回收才会清理残留对象。

3.4.5. 查询性能

实验使用目录规模 24、64、96 与精确命中数 1、2、4、8，组成 12 个实测网格。每个网格保留 15 个 AB/BA 内部配对，并记录遍历和 indexed 查询的延迟、候选数和检查记录数。本节及后续性能图表固定对应冻结在源码提交 2b14fb1f74b9bd093e6de939a16554620835699e 的测量批次；当前版本后续功能改动另由功能回归验证，未重新计入这组数据。

图3.1给出的 12 个格点中，indexed 路径的中位加速比为 1.164–2.808 倍；目录规模从

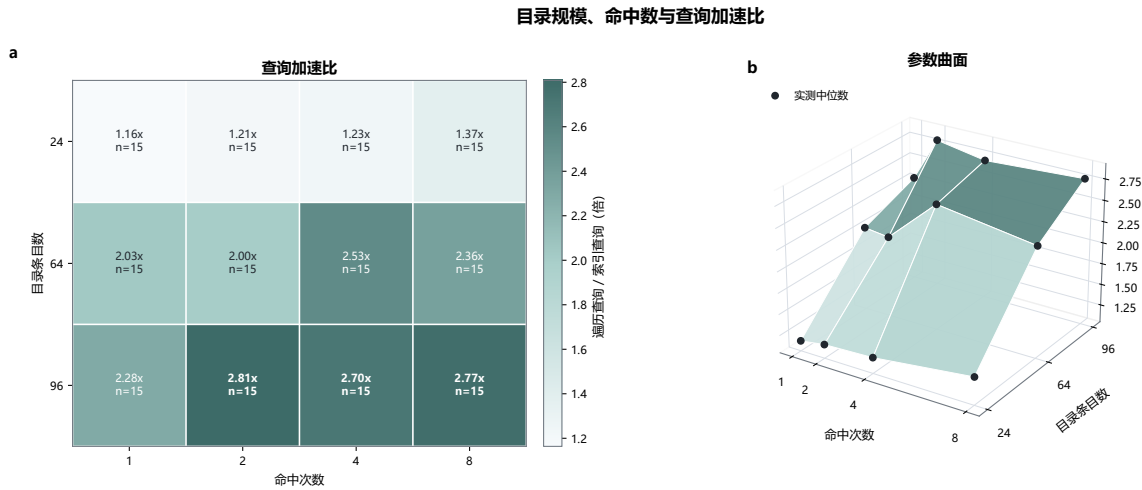


图 3.1: 目录规模、命中数与 indexed 查询加速比

24 增至 64、96 时，indexed 更快的内部配对数分别为 43/60、60/60、58/60，整体收益随规模增大而上升。其原因是 status→stage→kind 的固定选择顺序先缩小候选集，再逐项复核完整条件，而不是改变查询语义。命中数与加速比并不单调，24 项目录中仍有 17/60 个配对不占优；同时这些样本均使用 ready index，未计入建索引和维护索引的成本。因此，当前设计适合目录较大且筛选条件明确的已登记对象，不能把曲面上的最大倍数当作所有目录的固定收益；后续应把索引维护开销和低选择性查询纳入同一条路径评估。

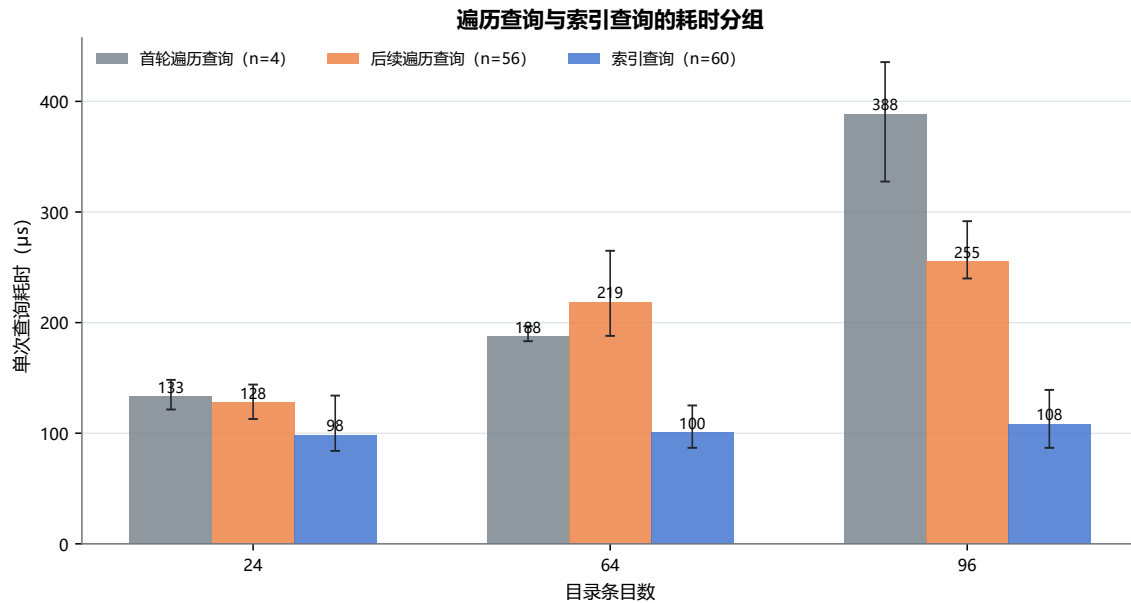


图 3.2: SCAN 与 indexed 查询的耗时分组

图3.2显示，indexed 查询的中位耗时随目录规模由 98.3 微秒升至 108.3 微秒；首次 SCAN 为 388.5 微秒，后续 SCAN 为 255.1 微秒。首个 SCAN 不是冷启动，28 条诊断记录均标为 cache=ready，它反映的是该查询阶段的首次完整遍历而非 Host 页面缓存重新装载。固定索引选择减少了中位延迟及其对目录规模的敏感性，但 64 项目录的后续 SCAN 仍出现 25.7505 ms/query 的尖峰，说明尾延迟尚未稳定。该结果支持把 indexed 查询作为有明确筛选字段时的默认路径，同时保留 SCAN 作为语义等价的回退，并继续定位完整遍历和目录检查造成的长尾。

3.4.6. 验证结果

`agentfs_ucore` 覆盖元数据设置、更新、删除、摘要、摘要哈希、对象代次和访问范围；`agentbench_ucore` 使用同一数据集比较遍历与索引查询的结果集、候选数和延迟。实时查询场景覆盖 ENTER、UPDATE、LEAVE、队列缺口、RESYNC_REQUIRED 以及替换订阅、快照、确认的安全切换。

3.5. Agent Loop 内核运行机制

3.5.1. 从轮询循环到内核等待

Agent Loop 会等待用户输入、模型回复、文件变化和其他 Agent 的消息。持续轮询不仅频繁进入内核，还会让大量等待线程占用调度时间。AgentOS 使用有界事件队列、订阅和等待接口，让没有事件的线程真正睡眠；消息、文件事件或 TIMER 到达后，内核再把相应线程唤醒。

每个智能体拥有容量为 16 的事件队列，其中保留一部分内核槽位，同一事件来源最多排队 4 条事件。路由表同样具有固定容量。发送 MESSAGE 时，内核检查 MESSAGE_SEND 能力位、活动工作流、发送者、目标路由和订阅状态。公开的 `agent_event` 只返回发送者、目标 PID、关联号等事件字段；`control_id` 和数据来源标签保存在内核附属记录中。

Agent 使用 `agent_watch()` 登记关心的事件，以 `agent_wait()` 休眠等待，再用 `agent_unwatch()` 取消订阅。心跳由 `agent_heartbeat_set()` 和 `agent_heartbeat_stop()` 调整。Loop 的下一步决策仍在 Guest 中完成，内核只负责保存订阅、管理等待状态，并在事件或 TIMER 到达时提供可靠唤醒。

非 TIMEOUT 的 `agent_wait` 会先检查已排队事件。选中事件时，内核建立保留项；没有事件时，内核以线程身份代次作为等待键安装等待者并让线程睡眠。事件入队和等待者状态使用相同锁序，确保到达事件与等待代次对应。TIMEOUT 路径在本地直接返回，不建立保留项，也不写入上下文。`agent_wait_cancel()` 只向等待路径注入 AGENT_EVENT_CANCELLED，它不是工具或任务通道的 CANCEL 操作；消费事件后才更新循环状态和上下文。

线程进入工具执行、等待事件和结束本轮处理时，内核分别维护 RUNNING、WAITING 和 IDLE 状态，并把同一进程中各线程的状态汇总到 PCB。`agent_info()` 可读取这一状态，观察程序也会把它与唤醒和调度记录对应起来。任务结束后，Guest 沿普通 `exit` 路径退出；生命周期在成员、在途操作和资源引用排空后统一回收 Context、订阅与队列对象。

3.5.2. 心跳与模型请求关联号

Agent 可以设置心跳间隔。定时器到期时，内核生成 TIMER 事件并唤醒 Agent；Guest 再根据当前状态决定检查文件、发送消息或继续等待。停止心跳会阻止后续 TIMER 产生，但已经进入事件队列的 TIMER 仍按普通事件消费，避免悄然改写队列历史。

LLM_REQUEST 与 LLM_RESPONSE 复用消息与等待机制。请求会登记生命周期、请求者和中继程序的 PID、控制标识、关联号和截止时间。同一请求者的关联号严格递增，并且只有成功入队后才更新。中继程序返回匹配响应时，内核原子消费待处理请求并生成 LLM_DONE。只有同一关联号仍处于活动待处理状态时才返回 DUPLICATE；复用已经完成或过期的关联号则返回 CONFLICT。待处理请求使用 120 秒内核时钟 TTL，完成历史区分

已消费请求和超时请求。

3.5.3. workflow 资源账户

资源账户维护已用额度 U 、待结算额度 P 、空余额度 F 和总额 $U + P + F$ 。当前账户的可用额度先在本地状态中转移：

reserve:F→P commit:P→U cancel:P→F release:U→F

账户补充额度时会检查全局限制、资源类别和账户硬限制。系统压力、上下文切换、账户关闭和 Workflow Fence 都会收缩 F 。Workflow Fence 在同一把锁内执行 trim+snapshot，要求 $P = 0$ ，并导出精确 U 、账户代次、额度周期和向量摘要。

3.5.4. 按 workflow 汇总的 EEVDF

传统线程级公平会让多线程 workflow 得到更多调度候选位。AgentOS 将同一资源账户中的线程合并为一个 workflow 实体。调度器使用 workflow 虚拟运行时间 (vruntime) 计算滞后量，满足 $vruntime \leq global_vtime$ 的实体进入可服务集合，再从中选择虚拟截止时间最早的实体。延迟类别、事件和截止时间可以缩短服务请求。一次分派完成后，内核根据实际服务周期更新 workflow 虚拟运行时间；睡眠实体的滞后量会向全局虚拟时间衰减。

只有一个 workflow 实体时，调度器使用快速路径。实体表容量为 4，其中包含一个 BOOT_SEALED 启动参与者，因此最多可同时纳入三个新建 workflow。实体信息不完整或容量超出时，调度器回到原 uCore 路径，并增加回退计数。

3.5.5. 唤醒延迟与公平性

唤醒与公平性测量使用 6 次全新 QEMU 启动，覆盖并发度为 1、2、3、4 的服务窗口和精确唤醒探针。504 条探针直接读取调度器跟踪中的 ready_age，其中 425 条为 0 个时钟节拍 (tick)，79 条为 1 个 tick。uCore 的 tick 为 10 ms；这些记录表明等待者能够在当前采样粒度内及时得到服务，也说明事件等待没有把 workflow 滞留在睡眠状态。0 tick 只代表未跨过下一个时钟节拍，不能解读为唤醒没有实际开销。

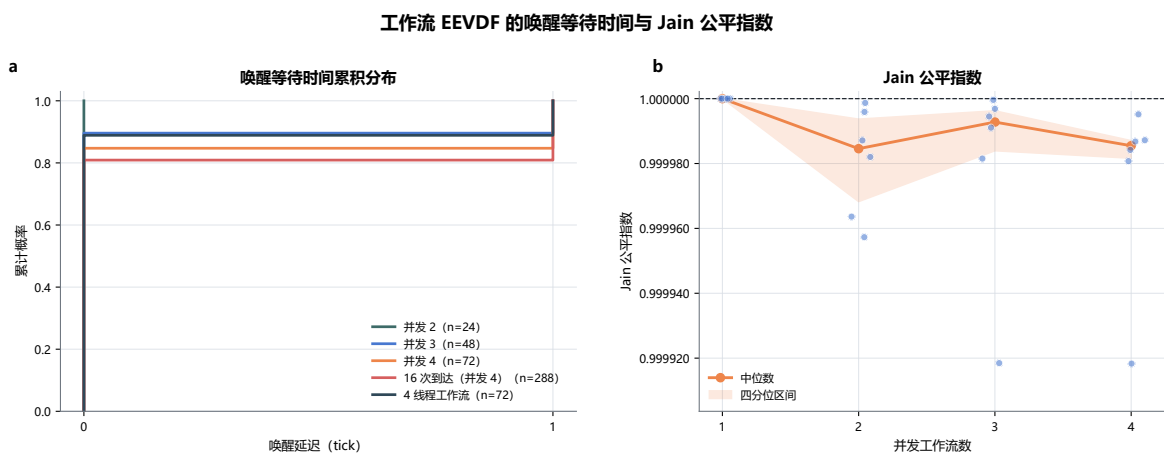


图 3.3: workflow EEVDF 的唤醒 tick 与调度公平性

图 3.3 中，425/504 个探针为 0 tick、79/504 个为 1 tick；并发度为 1、2、3、4 时，Jain 指数中位数分别为 1.000000、0.999985、0.999993 和 0.999985。workflow EEVDF 将同一 workflow

内的线程折叠为一个调度实体，因而能抑制线程数放大带来的服务倾斜。该结论受单核、10 ms 的 tick 粒度和最多四个实体分批纳入的条件限制：在独立的四线程放大场景中，Jain 中位数为 0.9980725，该 workflow 取得 26.895% 的服务份额，高于理想均分的 25%，并非绝对无偏。数据清单还标记了 boot 05 的 3 条 sample marker：串口拼接使它们缺少 wake-bucket 尾字段，但 service 值完整，其中并发度 3 和 4 的两条仍进入 Jain 计算；保守剔除相应测试批次后，两组中位数分别为 0.9999911 和 0.9999842，定性结论不变。精确唤醒分布来自独立的逐探针表，不受这 3 条记录影响。设计上应把当前实现视为对小规模并发 workflows 的公平保护；补采该组记录、扩大实体表、缩小观测粒度并在多核负载下复核，才是将该策略推广到更一般运行条件的前提。

3.5.6. 验证结果

agentloop_ucore 覆盖原子等待、唤醒、超时、取消、心跳和消息路由；agentsched_ucore 与 agent_eevdf_ucore 覆盖快速路径、实体容量、线程放大、睡眠衰减、回退、唤醒直方图和 Jain 公平性。

3.6. AgentOS 控制台综合 workflows

3.6.1. 科研恢复场景

AgentOS 控制台用六组内核机制执行一个确定性的科研恢复 workflow。labdemo_ucore 创建协调、哨兵、调查和恢复四个角色，建立 workflow 身份、MESSAGE 路由、上下文、元数据和资源账户。场景登记 prepare、align、analyze、report、archive 五个阶段及其依赖关系，再查询失败节点、读取摘要、向其他智能体发送恢复请求，完成 align 恢复和 report 更新。

固定策略为每个阶段指定相同的业务工作量，工具结果仍由 AgentOS 内核执行和检查。因此，该场景同时覆盖身份、上下文、实时查询、IPC 等待和恢复写入，并能在不同实现路径之间进行等量比较。

3.6.2. 遍历与索引路径的配对实验

综合场景保留遍历实现和索引实现，两者功能等价。每次独立 QEMU 启动按 A/B 或 B/A 顺序运行，两侧产生相同的工作负载和结果指纹。核心窗口覆盖查询、恢复写入、fsync 和复核，查询区段单独记录纯查询时间。

本章性能数字固定对应冻结在源码提交 2b14fb 的测量批次；当前版本后续功能改动另由功能回归验证，未重新计入这组数据。

测量数据来自 16 次独立 workflow 启动，AB 与 BA 各占一半。16 个核心查询区间中，索引路径的延迟均更低；遍历和索引路径的核心中位数分别为 34.713 ms 与 13.294 ms，配对加速比中位数为 3.118 倍。

图 3.4 显示 16/16 个核心查询配对均由 indexed 路径更快完成，中位加速比为 3.118 倍。选择性索引先减少候选记录，再执行与遍历路径相同的完整条件复核，因此收益来自减少无关扫描，而不是省略检查。AB 与 BA 的中位加速比分别为 1.454 倍和 5.481 倍，indexed 路径的四分位区间为 6.5185–28.9128 ms，表明收益幅度仍受启动顺序和运行状态影响，不能外推为固定吞吐提升。该结果支持在恢复流程的核心查询段优先选择 indexed 路径，同

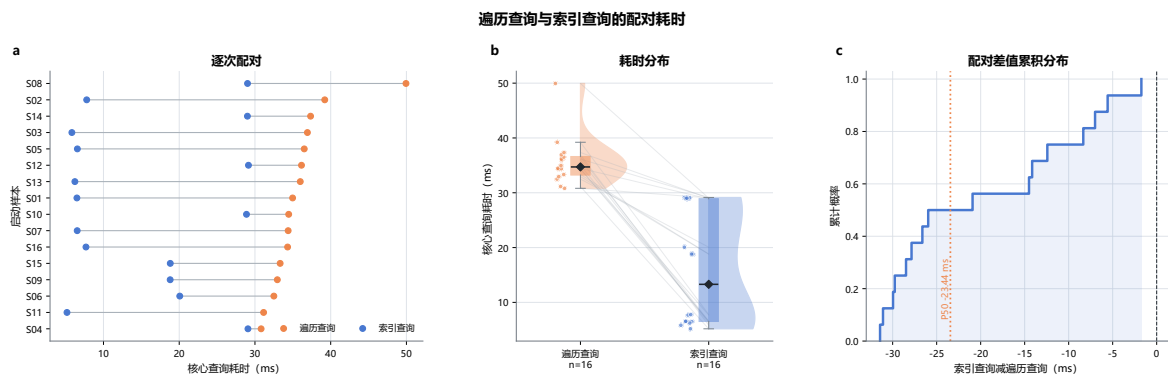


图 3.4: 遍历查询与索引查询的配对耗时

时要求完整流程继续分别观察查询以外的开销。

3.6.3. 核心查询与完整流程窗口

核心窗口用于观察 AgentOS 查询路径，完整流程窗口在此基础上还包含场景启动、文件与元数据准备、工作流收尾和统计输出。在完整流程窗口中，索引路径只在 16 个配对中的 3 个取得更低延迟，indexed-traversal 的中位数为 +13.452 ms。这说明核心窗口以外的总开销已经抵消索引收益；当前计时没有把这些外围步骤逐段拆开，尚不能确定其中哪一步占主导。

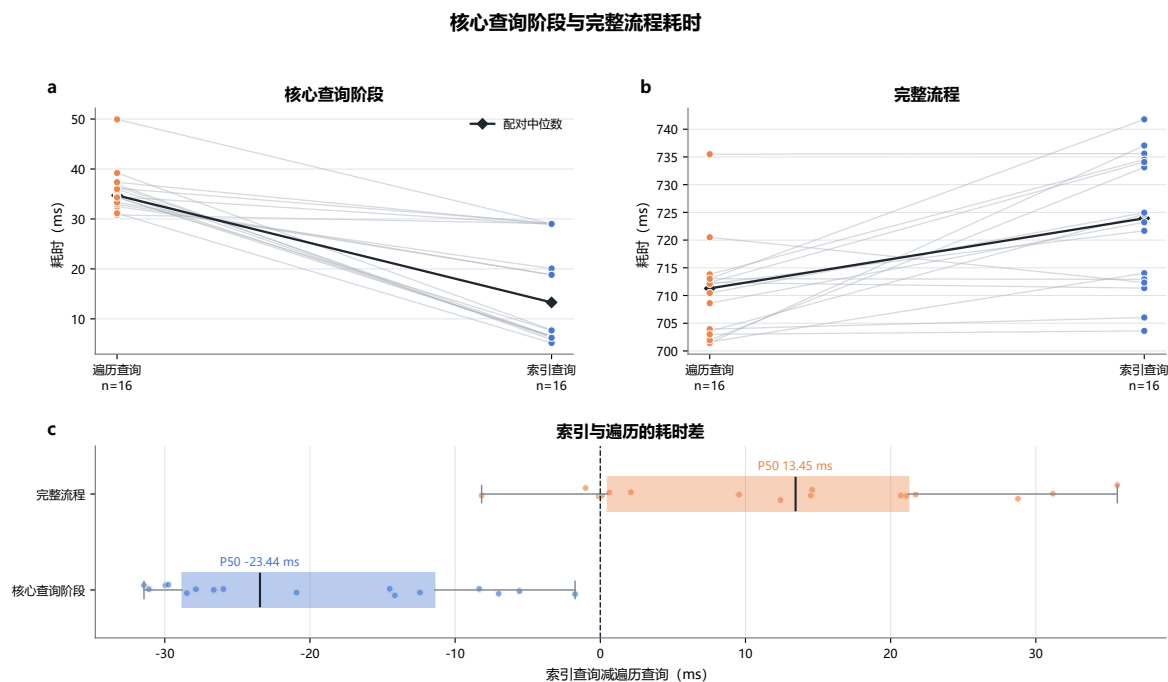


图 3.5: 核心查询阶段与完整流程耗时

图3.5把核心查询段与完整流程分开后，核心段的 indexed-traversal 中位差为 -23.4415 ms，16/16 个配对更快；端到端中位差却为 +13.452 ms，只有 3/16 个配对更快，整体约慢 1.90%。indexed 路径在核心查询以外的余量在 16/16 个配对中更长，中位多 33.477 ms，说明当前系统虽然已经缩短了索引查询本身，但完整流程的准备、统计和清理没有同步受益，最终抵消了核心路径的改进。这是现阶段系统实现的明确不足。下一步需要把外围步骤分别计时，找出占主导的环节后再优化；长期会话能否摊薄这部分成本尚未测量，

不能作为现有结论。

3.6.4. 系统结果

综合 workflows 依次执行智能体创建、上下文记录、带类型信息的工具调用、实时查询、消息等待、恢复写入和执行记录。输出包含业务指纹、各阶段状态、检查记录数、核心与完整流程时间以及工作量计数。遍历查询和索引查询都在同一 AgentOS-uCore 内核与同一 Guest 场景中运行，并以相同的用户态约定比较两种查询实现。

3.7. 综合场景：AgentOS Nexus 多智能体协作平台

3.7.1. 长驻会话与三角色协作

AgentOS Nexus 是构建在 AgentOS-uCore 之上的通用交互 Harness。它接受任意非空用户任务，模型可以直接回答，也可以自主选择 `search_files`、`read_file` 或 `inspect_system`。一轮会话在同一 workflow 中常驻 Coordinator、System 和 Research 三个业务 Agent；每个角色都有独立的 PID、Agent 编号、控制标识、能力位和 Context，并共享 workflow 生命周期、VFS 访问范围、资源账户和 EEVDF 实体。

产品角色/内核角色	职责	主要权限
Coordinator / 协调者	维护会话、调用模型、创建 root/child Task、核验返回	建立 Task Channel、授予 TASK route、读取结果 artifact
System / 哨兵	读取当前 Guest 的状态、进程或 Context	领取系统观察任务、执行只读工具、发布结果 artifact
Research / 调查者	使用 Guest Catalog 选择的工作区对象研究源码与文档	领取研究任务、读取版本绑定字节、发布结果 artifact

表 3.5: Nexus 的三个业务角色

Guest Relay 负责 QEMU 串口帧和模型循环。Host Relay 持有本地套接字、传输加密配置、服务接口密钥和模型格式适配器；Host workspace broker 则只处理显式会话 root 内的 manifest 与字节传输。任务所有权、工作区候选、结果 artifact、Context 和任务终态都保留在 Guest 与 AgentOS 内核中。

3.7.2. 一次交互回合

图3.6展示了从用户目标到最终结果的一次完整运行过程。

1. Guest 把用户输入建立为本轮 root Task，并将 USER 短节点写入 Relay Agent 的 Context active path。
2. Relay 直接查询 active path，并用第七页用户缓存补充仍然可见的有界正文，形成下一次 MODEL_REQUEST。
3. 模型可以直接返回 FINAL，也可以选择一个公开工具。`inspect_system` 交给 System；`search_files` 和 `read_file` 交给 Research。
4. Coordinator 把目标身份、任务关联和 capsule handle 写入 56 字节描述符，经 Execution

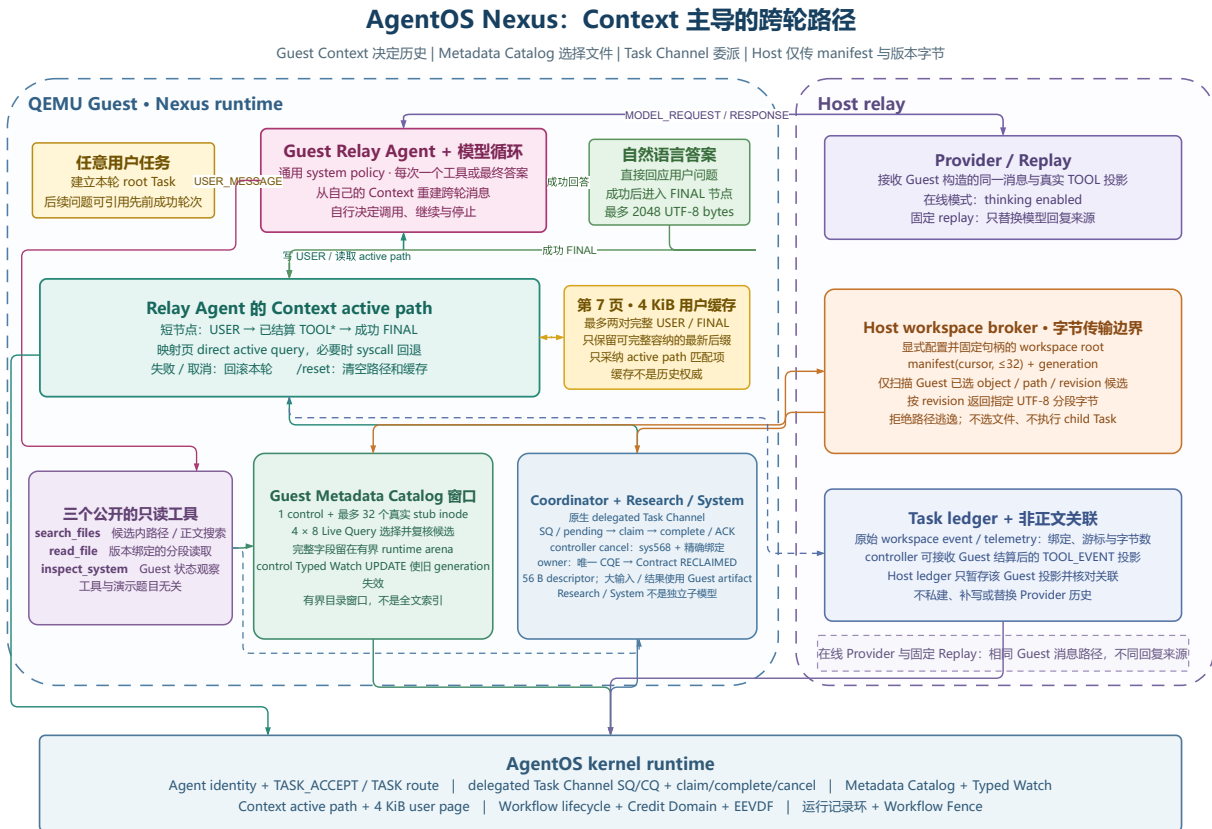


图 3.6: Nexus 中由 Context、Catalog 与 Task Channel 共同驱动的协作流程

Contract 和原生 Task Channel 提交 child Task。

- 目标 Agent claim 任务, 读取 Guest capsule artifact, 执行受能力位与副作用清单约束的操作, 再发布结果 artifact 并 complete。
- Coordinator 从唯一 terminal CQE 结算, 核验结果 artifact 后把真实 TOOL 结果写入 Context, 并交给下一轮模型决策。
- 成功 FINAL 采用准备与提交两阶段写入 Context; 失败或取消则回滚本轮路径, 避免未结算内容进入下一轮历史。

3.7.3. 原生 delegated Task Channel

Nexus 不再用用户态消息状态机模拟 child Task。内核工具目录中的第 26 项 `delegate_task` 接收准确的 56 字节 `AGENT_ARTIFACT_TASK` 描述符; 描述符只绑定目标 PID、Agent/control 身份、任务和父任务关联、capsule handle 与保留字段, 大段任务正文和结果留在 Guest artifact 中。

Coordinator 是 Task Channel owner, 也是单一 issuer。目标必须持有任务接收能力位, 并获得与普通消息分离的 TASK route。提交成功后任务进入 Execution Contract 的 RUNNING 状态和内核 pending 队列; System 或 Research 使用 claim/complete 系统调用领取并提交终态, owner 只收到一条 terminal CQE。相关 ABI 名称和系统调用号如下:

```
1 capability AGENT_CAP_TASK_ACCEPT
2 route AGENT_IPC_ROUTE_TASK
3 syscall 567 agent_task_delegate_claim
4 syscall 568 agent_task_delegate_complete
```

claim 同时建立 delegated effect lease，精确绑定 worker 身份、线程 generation 与 channel/request/slot 代次。worker 或已登记 helper 的直接副作用必须是 Contract manifest 的子集，complete 以及终态确认都要等待活动 lease 引用归零。

同一活动生命周期内的 Nexus controller 通过上述 complete 接口设置 AGENT_TASK_DELEGATE_COMPLETE_F_REQUEST_CANCEL 发起取消：它提交 CANCELLED、零 ACK 字段以及准确的 owner/channel/request/slot/task/correlation tuple；内核还要求 AGENT_CAP_ORCHESTRATE、AGENT_CAP_WAIT_CANCEL 和指向 owner 的活动 AGENT_IPC_ROUTE_TASK 路由。返回 OK 只是 control request 的接收凭据，不是 delegated Task 的 terminal CQE。若任务已经处于 CLAIMED，provider 仍须清理预绑定输出，再以 ACK_TERMINAL 确认最新 generation 的 offer；首版依赖执行者协作，不强制终止永久无响应的已领取任务。一般进程通信机制与 Nexus child Task 的 payload、状态和结果保持分离。

3.7.4. Guest Catalog 与 Host 工作区边界

Host workspace broker 在会话开始时固定一个显式 root，并以 generation、object id、相对 path、revision、大小和类型构造分页 manifest。revision 来自与后续搜索或读取相同的已验证文件句柄和字节；broker 拒绝绝对路径、目录跳转、符号链接逃逸和超出边界的对象。Host 不决定 Nexus 应当读取哪个文件。

Guest 为当前 manifest 页面建立一个 control inode 和最多 32 个 data-stub inode，把对象登记到 AgentOS Metadata Catalog。每页按四个 stage 执行最多 8 项的 Live Query，并在有界 runtime arena 中复核完整 path、object id 与 revision。control inode 上的 Typed Watch 接收 generation UPDATE；如果出现 RESYNC_REQUIRED 或 stale 结果，Guest 使旧窗口失效，从 cursor 0 重建 Catalog 与订阅。

search_files 先由 Guest Catalog 选出候选，Host 只在这些 object/path/revision 绑定的候选中执行正文匹配。read_file 先用完整路径在 manifest 中定位对象，再由 Catalog 精确复核，Host 只返回该 revision 的指定行段。Metadata Catalog 是有界对象目录和候选选择器，不是全文索引；正文扫描仍由 Host 完成，但 Host 无权扩大候选集合。路径、manifest 游标、字节预算与输出大小均有明确上限。

3.7.5. artifact、来源与 Context

Host 返回的实际搜索或读取正文首先进入 Guest，成为 Research 的输入 artifact。Research 核对任务、对象绑定、摘要与来源后，以自身身份发布结果 artifact；manifest 同时记录生命周期、句柄 generation、任务/父任务、producer、owner、权限、载荷大小、provenance label 和 producer Context sequence。Coordinator 只有在 Task Channel terminal CQE 与 artifact 彼此匹配后才读取结果。

工作区结果不会在 Host 内部直接替换模型请求。Coordinator 把已结算的结果写为 TOOL Context，Relay 再从 Guest Context active path 重建下一轮消息。公开给模型的投影可截断或结构化，但其来源仍绑定同一个真实 Guest artifact；inspect_system 的运行观察还会标记为不可信 Guest runtime 数据，不能自动成为控制授权。

3.7.6. 跨轮 Context 路径

Nexus 的跨轮历史由 Guest Relay Agent 的 AgentOS Context active path 决定。每轮 USER、已经结算的 TOOL 和成功 FINAL 都形成短 Context 节点；第七页 4 KiB 用户缓存只为仍在 active path 上的 USER/FINAL 对补充有界正文。Relay 优先直接读取 Context 映射页，必要时回退到系统调用查询。

失败或取消会把 active head 回滚到本轮 USER 之前，/reset 同时清空 Context、用户缓存和工作区 Catalog/watch 状态。Host Relay 不私建、补写或替换 Provider 请求中的对话正文与 Guest 工具结果；在线 Provider 与固定 Replay 都直接接收 Guest 重建的消息以及真实 TOOL artifact 投影。因此，Nexus 复用的是 AgentOS 已有的 Context 路径，而不是一套并列的 Host 记忆机制。

3.7.7. 工具发现与内核观测

Nexus 启动时读取 AgentOS 内核的 26 项 Tool Registry，检查名称、编号和 schema，再按业务角色、内核角色、能力位和副作用类别筛选。模型公开表面保持三个通用只读工具：search_files、read_file 和 inspect_system；delegate_task 是 Harness 在 Guest 内部使用的内核工具，不要求模型构造 Task descriptor。

观察路径通过 timeline、audit、identity、Context、Task Channel 和 Catalog 状态生成结构化事件。它可以显示 TASK route、claim/complete、terminal CQE、artifact 接收、Context sequence、Catalog generation、Typed Watch 失效以及 Agent 的等待和调度状态。controller 只接收 Guest 已结算的投影与非正文关联信息，不拥有任务状态机。

3.7.8. 运行验证

固定 ReplayProvider 的每条响应都与 Guest 当次生成的规范 MODEL_REQUEST SHA-256 绑定，并在当前 RV64 镜像的真实 QEMU 启动中执行。回放与在线模式使用同一个 Guest 模型循环、Context active path、Task Channel、Metadata Catalog、Live Query、Typed Watch 和 artifact 路径；固定数据只替换 Provider 回复，不替换工具执行结果。回归覆盖直接回答、模型自主选工具、版本化工作区读取、跨 Agent claim/complete、controller 取消请求、provider 终态确认、Context 回滚/清空与正常会话关闭。

3.8. 关键实现

下列代码把设计章节中的机制逐一对应到当前代码树。代码框保留相对路径和物理行号，便于直接核对内核实现。所选片段覆盖启动、身份、调用、执行约定、上下文、文件事件、任务通道、结果文件发布、资源和调度，说明一个智能体工作流如何创建、执行并结束。

3.8.1. 启动、身份与 workflow 代次

内核入口先初始化进程、控制台和页表等基本设施，再调用 agentinit()。文件系统就绪后，系统启用智能体存储和文件代次机制。随后加载的初始用户程序直接使用已建立的生命周期、元数据和资源机制，不需要在用户态临时拼接旁路状态。

代码 1: uCore 启动序列中的 AgentOS 初始化

c

```

19 void main()
20 {
21     clean_bss();
22     printf("hello world!\n");
23     proc_init();
24     console_init();
25     agentinit();
26     kinit();
27     kvm_init();
28     if (kalloc_physical_policy_init(PHYSICAL_PAGE_SYSTEM_RESERVE,
29                                     PHYSICAL_PAGE_ORDINARY_LIMIT) < 0)
30         panic("physical page policy");
31     trap_init();
32     plicinit();
33     virtio_disk_init();
34     binit();
35     fsinit();
36     timer_init();
37     agent_storage_init();
38     fs_epoch_runtime_enable();
39     load_init_app();
40     infof("start scheduler!");
41     /* 仅启动期使用的映像加载完成后, 才开放运行时输入输出准入。 */
42     bio_policy_start();
43     virtio_disk_runtime_start();
44     scheduler();

```

生命周期表用 id+generation 共同定位 workflow；槽位复用时，代次单调增加。创建工作流时，内核先准备记录，再一次写入创建者、访问范围、成员数和代次。因此，旧代次的订阅、任务或资源句柄会在下次检查时失效，不会误指向新工作流。

代码 2: 工作流生命周期的创建与代次绑定

c

```

56
57 static struct workflow_lifecycle_record *
58 workflow_lifecycle_find_locked(struct workflow_lifecycle_key key)
59 {
60     uint slot;
61     struct workflow_lifecycle_record *record;
62
63     if (!workflow_lifecycle_key_valid(key) ||
64         key.id > WORKFLOW_LIFECYCLE_CAP)
65         return 0;
66     slot = key.id - 1;
67     record = &workflow_lifecycles[slot];
68     if (!record->used || record->generation != key.generation)
69         return 0;
70     return record;
71 }
72
73 static struct workflow_lifecycle_key
74 workflow_lifecycle_record_key(uint slot,
75                               const struct workflow_lifecycle_record *record)

```

```

76 {
77     struct workflow_lifecycle_key key = {
78         .id = slot + 1,
79         .generation = record->generation,
80     };
81
82     return key;
83 }
84
85 int workflow_lifecycle_create(uint scope_id,
86                             struct workflow_lifecycle_key *key)
87 {
88     int enabled;
89     int result = -1;
90     uint allocated_slot = WORKFLOW_LIFECYCLE_CAP;
91     uint64 next_generation = 0;
92
93     if (key == 0 || scope_id < VFS_SCOPE_FIRST_DYNAMIC ||
94         scope_id >= FS_OWNER_SCOPE_FLAG)
95         return -1;
96     *key = workflow_lifecycle_none();
97     enabled = intr_save();
98     for (uint i = 0; i < WORKFLOW_LIFECYCLE_CAP; i++)
99         if (workflow_lifecycles[i].used &&
100             workflow_lifecycles[i].scope_id == scope_id)
101             goto out;
102     for (uint i = 0; i < WORKFLOW_LIFECYCLE_CAP; i++) {
103         struct workflow_lifecycle_record *record =
104             &workflow_lifecycles[i];
105         uint64 generation;
106
107         if (record->used ||
108             workflow_lifecycle_generations[i] == ~0ULL)
109             continue;
110         generation = workflow_lifecycle_generations[i] + 1;
111         if (generation == 0 ||
112             !agent_identity_lease_lifecycle_contains(i, generation))
113             continue;
114         workflow_lifecycle_generations[i] = generation;
115         record->used = 1;
116         record->scope_id = scope_id;
117         record->generation = generation;
118         record->controller_control_id = 0;
119         record->next_context_branch = 0;
120         record->fence_sequence = 0;
121         record->members = 1;
122         record->active_operations = 0;
123         record->departing_operations = 0;
124         record->fence_gate = 0;
125         record->closing = 0;
126         *key = workflow_lifecycle_record_key(i, record);
127         allocated_slot = i;
128         next_generation = generation == ~0ULL ? 0 : generation + 1;
129         result = 0;
130         break;
131     }
132 out:

```



```

133     intr_restore(enabled);
134     if (allocated_slot < WORKFLOW_LIFECYCLE_CAP)
135         agent_identity_lease_lifecycle_note_next(
136             allocated_slot, next_generation);

```

角色规则不是用户态的字符串约定。内核按角色取得能力位掩码和调度权重，再在授权入口检查父角色、目标角色和能力位的合法关系。因此，工具、VFS、IPC 和调度器都依据同一份身份信息作出判断。

代码 3：角色策略表查询与调度权重取得

c

```

128     AGENT_IDENTITY_ALLOCATOR_AGENT);
129     return 0;
130 }
131
132 const struct agent_role_policy *
133 agent_identity_role_policy(int role)
134 {
135     for (uint i = 0; i < NELEM(role_policies); i++)
136         if (role_policies[i].role == role)
137             return &role_policies[i];
138     return 0;
139 }
140
141 int
142 agent_identity_role_valid(int role)
143 {
144     return agent_identity_role_policy(role) != 0;
145 }
146
147 int
148 agent_identity_role_sched_weight(int role)
149 {
150     const struct agent_role_policy *policy =

```

代码 4：父角色授权与进程访问范围校验

c

```

190     p->agent_capability_mask = 0;
191     p->agent_role_grant_mask = 0;
192 }
193
194 int
195 agent_identity_authority_check(struct proc *p, int role)
196 {
197     if (!agent_identity_role_valid(role))
198         return AGENT_STATUS_BAD_PARAM;
199     if (p == 0 ||
200         (p->is_agent ?
201          (!exec_policy_process_allows_role(p, p->agent_role) ||
202           !exec_policy_process_allows_role(p, role)) :
203          (!exec_policy_process_bootstrap(p) ||
204           !exec_policy_process_allows_role(p, role))) ||
205         (p->agent_role_grant_mask & AGENT_ROLE_GRANT_BIT(role)) == 0)
206         return AGENT_STATUS_DENIED;

```

```

207     return AGENT_STATUS_OK;
208 }
209
210 uint
211 agent_identity_proc_scope(struct proc *p)
212 {
213     if (p == 0 || !p->is_agent ||
214         p->agent_control_state != AGENT_CONTROL_OPEN ||
215         p->vfs_scope_id < VFS_SCOPE_FIRST_DYNAMIC ||
216         p->vfs_scope_id >= FS_OWNER_SCOPE_FLAG ||

```

算法 2：带代次的工作流进入检查

1. 内核从当前进程控制块读取工作流编号和代次，并验证 id 与 generation 均非零。
2. 内核在生命周期表中同时查找这两个字段；仅槽位相同不能通过校验。
3. 内核进入操作保护区后才执行系统副作用；屏障或回收已关闭该路径时返回重试或过期错误。
4. 异步对象在完成、取消或回收时再次比较代次，再释放生命周期引用。

3.8.2. 结构化工具调用与执行约定

V3 调用先检查用户内存范围、ABI 版本和大小、保留字段、参数个数和生命周期绑定。请求中的执行约定、节点、尝试号、schema 摘要和输入指纹会复制到内核记录。因此，一次工具调用不再是孤立的函数调用，而是执行约定 DAG 中可核验的节点执行。

代码 5：V3 请求的 ABI、生命周期和执行约定绑定检查

c

```

2590     }
2591     if (copyinstr(p->pagetable, path, pathaddr, sizeof(path)) < 0) {
2592         proc_discard_fd_delegations();
2593         return -1;
2594     }
2595     path[sizeof(path) - 1] = 0;
2596     {
2597         int result = agent_worker_create_proc(path, requested_caps);
2598
2599         if (result < 0)
2600             proc_discard_fd_delegations();
2601         return result;
2602     }
2603 }
2604
2605 int sys_agent_info(uint64 addr, int launch_identity)
2606 {
2607     struct proc *p = curr_proc();
2608     struct agent_info info;
2609     int result;
2610
2611     if (launch_identity) {
2612         memset(&info, 0, sizeof(info));
2613         info.is_agent = p->is_agent;
2614         info.agent_id = p->agent_id;

```

```

2615     info.agent_role = p->agent_role;
2616     info.capability_mask = agent_identity_proc_scope(p) != VFS_SCOPE_NONE ?
2617         p->agent_capability_mask : 0;
2618     info.filesystem_domain = p->vfs_scope_id;
2619     info.filesystem_capability_mask =
2620         vfs_scope_active(p->vfs_scope_id) ? p->vfs_effective_caps : 0;
2621 } else {
2622     agent_info_fill(p, &info);
2623 }
2624 result = copyout(p->pagetable, addr, (char *)&info, sizeof(info));
2625 return result < 0 || !launch_identity ? result : p->pid;
2626 }
2627
2628 int sys_agent_run(uint64 opsaddr, uint64 resultsaddr, int count, uint64 flags)
2629 {
2630     struct proc *p = curr_proc();
2631     struct thread *t = curr_thread();
2632     struct workflow_lifecycle_key lifecycle;
2633     struct agent_op op;
2634     struct agent_result res;
2635     int status = 0;
2636
2637     if (!p->is_agent)
2638         return -1;
2639     if (count < 0 || count > AGENT_BATCH_MAX)
2640         return -1;
2641     if (count == 0 && flags == AGENT_RUN_F_FENCE) {
2642         struct agent_workflow_fence_request request;
2643         struct agent_workflow_fence_receipt receipt;
2644         const struct agent_workflow_fence_request *request_ptr = 0;
2645         struct agent_workflow_fence_receipt *receipt_ptr = 0;
2646
2647         /* Anonymous fences are fire-and-forget; receipts require a retry id. */
2648         if (opsaddr == 0 && resultsaddr != 0)
2649             return -1;
2650         if (opsaddr != 0) {
2651             if (user_range_check(p->pagetable, opsaddr,
2652                 sizeof(request), PTE_R) < 0 ||
2653                 copyin(p->pagetable, (char *)&request, opsaddr,
2654                     sizeof(request)) < 0)
2655                 return -1;
2656             request_ptr = &request;
2657         }
2658         if (resultsaddr != 0) {
2659             if (user_range_check(p->pagetable, resultsaddr,
2660                 sizeof(receipt), PTE_W) < 0)
2661                 return -1;
2662             receipt_ptr = &receipt;
2663         }
2664         status = agent_workflow_fence_execute(
2665             p, request_ptr, receipt_ptr);
2666         if (receipt_ptr != 0 &&
2667             copyout(p->pagetable, resultsaddr, (char *)&receipt,

```

解析阶段先按工具名称或编号查找定义，再逐项从用户内存读取带类型信息的参数。每个参数都必须符合目录规则规定的名称、类型、长度和唯一性，最后还要确认必填参数

全部出现。用户态即使构造出形式正确的结构，也不能绕过内核目录中的 schema。

代码 6: V2 参数 schema 解码器

c

```

548 }
549
550 int agent_tool_protocol_decode_v2(pagetable_t pagetable,
551     struct agent_request_v2 *request,
552     struct agent_tool_match *match,
553     struct agent_op *op, char *error,
554     int error_size)
555 {
556     uint seen = 0, count = rule_count(match->tool_id);
557     const struct param_rule *tool_rules = rules_for_tool(match->tool_id);
558
559     memset(op, 0, sizeof(*op));
560     op->version = AGENT_CALL_VERSION_V1;
561     op->tool_id = match->tool_id;
562     op->request_id = request->request_id;
563     for (uint i = 0; i < request->param_count; i++) {
564         struct agent_param_v2 param;
565         uint text_length = 0;
566         int index = -1;
567
568         if (copyin(pagetable, (char *)&param,
569             request->params + (uint64)i * sizeof(param),
570             sizeof(param)) < 0)
571             return -1;
572         if (param.version != AGENT_PARAM_VERSION)
573             REJECT(AGENT_STATUS_BAD_VERSION, "bad_param_version");
574         if (param.size != sizeof(param) ||
575             string_length(param.key, AGENT_PARAM_KEY_SIZE, 0) < 0)
576             REJECT(AGENT_STATUS_BAD_SIZE, "bad_param_size");
577         for (uint j = 0; j < count; j++)
578             if (!strncmp(param.key, rule_key(&tool_rules[j]),
579                 AGENT_PARAM_KEY_SIZE)) {
580                 index = j;
581                 break;
582             }
583         if (index < 0)
584             REJECT(AGENT_STATUS_UNKNOWN_PARAM, "unknown_param");
585         if (seen & (1U << index))
586             REJECT(AGENT_STATUS_DUPLICATE, "duplicate_param");
587         seen |= 1U << index;
588         if (param.type != rule_type(&tool_rules[index]))
589             REJECT(AGENT_STATUS_BAD_TYPE, "bad_param_type");
590         if (param.type == AGENT_PARAM_UINT64) {
591             if (param.value_size != sizeof(param.value.uint64_value))
592                 REJECT(AGENT_STATUS_BAD_SIZE, "bad_value_size");
593         } else if (param.value_size == 0 ||
594             param.value_size > AGENT_PARAM_STRING_SIZE ||
595             string_length(param.value.string_value, param.value_size,
596                 &text_length) < 0 ||
597             text_length + 1 != param.value_size)
598             REJECT(AGENT_STATUS_BAD_SIZE, "bad_value_size");
599         switch (rule_target(&tool_rules[index])) {

```

```

600     case PARAM_ARG0: op->arg0 = param.value.uint64_value; break;
601     case PARAM_ARG1: op->arg1 = param.value.uint64_value; break;
602     default: safestrcpy(op->payload, param.value.string_value,
603         sizeof(op->payload)); break;
604 }
605 }
606 for (uint i = 0; i < count; i++)
607     if (rule_required(&tool_rules[i]) && !(seen & (1U << i)))

```

参数校验通过后，调用先进入生命周期操作保护区，再调用 `agent_execute_one()`。该入口汇总调用结果、输出 **artifact** 类型、拒绝原因、来源标签和执行记录票号；受规则约束的拒绝会尝试写入安全记录。关键记录无法预留或提交时，直接调用和 V3 调用的拒绝会返回 `NO_SPACE` 或 `RETRY`，因此不是每次拒绝都会得到执行记录票号。

代码 7：V3 调用的解码、检查与执行进入

c

```

2678     if (count == 0)
2679         return 0;
2680     if (user_range_check(p->pagetable, opsaddr,
2681         (uint64)count * sizeof(struct agent_op), PTE_R) < 0)
2682         return -1;
2683     if (user_range_check(p->pagetable, resultsaddr,
2684         (uint64)count * sizeof(struct agent_result),
2685         PTE_W) < 0)
2686         return -1;
2687     lifecycle = vfs_proc_lifecycle(p);
2688     if (workflow_lifecycle_key_valid(lifecycle) &&
2689         workflow_lifecycle_operation_enter(lifecycle) < 0)
2690         return -1;
2691     agent_identity_thread_loop_set(t, AGENT_LOOP_RUNNING);
2692     for (int i = 0; i < count; i++) {
2693         struct bio_checkpoint_result checkpoint;
2694
2695         if (copyin(p->pagetable, (char *)&op,
2696             opsaddr + i * sizeof(struct agent_op),
2697             sizeof(op)) < 0) {
2698             status = -1;
2699             break;
2700         }
2701         if (agent_execute_one(p, &op, &res, agent_ticks(), 0, 0, 0, 0) < 0 ||
2702             copyout(p->pagetable,
2703                 resultsaddr + i * sizeof(struct agent_result),
2704                 (char *)&res, sizeof(res)) < 0) {
2705             status = -1;
2706             break;
2707         }
2708         status = i + 1;
2709         checkpoint = bio_request_checkpoint();
2710         if (bio_checkpoint_should_stop(checkpoint))
2711             break;
2712         if (kernel_work_checkpoint(KERNEL_WORK_OPERATION_UNITS) < 0)
2713             break;
2714     }
2715     agent_identity_thread_loop_set(t, AGENT_LOOP_IDLE);
2716     if (workflow_lifecycle_key_valid(lifecycle))

```



```

2717     workflow_lifecycle_operation_leave(lifecycle);
2718     return status;
2719 }
2720
2721 int sys_agent_tool_list(uint64 addr, int max)
2722 {
2723     return agent_tool_protocol_list_v1(curr_proc()->pagetable, addr, max);
2724 }
2725
2726 int sys_tool_list(uint64 addr, int max, uint desc_size, uint version)
2727 {
2728     return agent_tool_protocol_list_v2(curr_proc()->pagetable, addr, max,

```

代码 8：受控拒绝的安全记录与返回封装

c

```

2730 }
2731
2732 int sys_agent_call(uint64 reqaddr, uint64 respaddr)
2733 {
2734     struct proc *p = curr_proc();
2735     struct workflow_lifecycle_key lifecycle;
2736     struct agent_request req;
2737     struct agent_response resp;
2738     struct agent_op op;
2739     struct agent_result res;
2740     struct agent_tool_match tool;
2741     char validate_error[AGENT_RESULT_SIZE];
2742     int status;
2743
2744     if (copyin(p->pagetable, (char *)&req, reqaddr, sizeof(req)) < 0)
2745         return -1;
2746     if (user_range_check(p->pagetable, respaddr, sizeof(resp), PTE_W) < 0)
2747         return -1;
2748     memset(&resp, 0, sizeof(resp));
2749     resp.version = AGENT_CALL_VERSION_V1;
2750     resp.request_id = req.request_id;
2751     if (req.version != AGENT_CALL_VERSION_V1) {
2752         resp.status = AGENT_STATUS_BAD_VERSION;
2753         safestrncpy(resp.result, "bad_request_version", sizeof(resp.result));
2754         return copyout(p->pagetable, respaddr, (char *)&resp,
2755             sizeof(resp));
2756     }
2757     if (!p->is_agent) {
2758         resp.status = AGENT_STATUS_NOT_AGENT;
2759         safestrncpy(resp.result, "not_agent", sizeof(resp.result));
2760         return copyout(p->pagetable, respaddr, (char *)&resp,
2761             sizeof(resp));
2762     }
2763     memset(validate_error, 0, sizeof(validate_error));
2764     status = agent_tool_protocol_resolve(req.tool_id, req.tool_name, &tool,

```

执行约定的接纳检查会逐项比对绑定的生命周期、执行约定代次、节点编号、截止时间、工具编号和资源额度。任务通道的内部请求还限制为“不产生输出”的预检形式；被拒绝的任务不会因此发布伪造的带类型信息输出。

代码 9：执行约定节点的代次、资源与工具一致性检查

c

```

1157         p, curr_thread(),
1158         tool_manifest.provenance.side_effect_mask,
1159         &claim->delegated_slot,
1160         &claim->delegated_generation)) {
1161     claim->legacy = 1;
1162     claim->delegated_active = 1;
1163     claim->active = 1;
1164     intr_restore(enabled);
1165     return AGENT_EXECUTION_ADMISSION_EXECUTE;
1166 }
1167 record->denial_count++;
1168 agent_execution_result_error(
1169     result, AGENT_STATUS_DENIED,
1170     "execution_contract_required",
1171     AGENT_EXECUTION_REASON_CONTRACT_REQUIRED, claim);
1172 goto denied;
1173 }
1174 if (record->state == AGENT_EXECUTION_STATE_BUILDING) {
1175     record->denial_count++;
1176     agent_execution_result_error(
1177         result, AGENT_STATUS_DENIED,
1178         "execution_contract_required",
1179         AGENT_EXECUTION_REASON_CONTRACT_REQUIRED, claim);
1180     goto denied;
1181 }
1182 if (record->bare_inflight == (uint)-1) {
1183     agent_execution_result_error(
1184         result, AGENT_STATUS_RETRY, "legacy_gate_full",
1185         AGENT_EXECUTION_REASON_NODE_BUSY, claim);
1186     goto denied;
1187 }
1188 record->bare_inflight++;
1189 claim->legacy = 1;
1190 claim->active = 1;
1191 /* Legacy callers retain their compatibility provenance policy. */
1192 claim->manifest.accepted_input_labels = AGENT_PROVENANCE_ALL;
1193 intr_restore(enabled);
1194 return AGENT_EXECUTION_ADMISSION_EXECUTE;
1195 }
1196
1197 memmove(claim->request_digest, request_digest,
1198         sizeof(claim->request_digest));
1199
1200 if (!workflow_lifecycle_key_equal(binding->lifecycle, current)) {
1201     agent_execution_result_error(
1202         result, AGENT_STATUS_STALE, "stale_lifecycle",
1203         AGENT_EXECUTION_REASON_STALE_LIFECYCLE, claim);
1204     goto denied_counted;
1205 }
1206 if (!agent_execution_record_enforced(record, current)) {
1207     agent_execution_result_error(
1208         result, AGENT_STATUS_DENIED, "execution_contract_missing",
1209         AGENT_EXECUTION_REASON_CONTRACT_REQUIRED, claim);
1210     goto denied_counted;

```

```

1211     }
1212     if (record->state == AGENT_EXECUTION_CONTRACT_RETIRING) {
1213         agent_execution_result_error(
1214             result,
1215             (binding->internal_flags &
1216              AGENT_EXECUTION_BINDING_INTERNAL_F_TASK_CHANNEL) != 0 ?
1217              AGENT_STATUS_DENIED : AGENT_STATUS_CANCELLED,
1218              "contract_retiring",
1219              AGENT_EXECUTION_REASON_CONTRACT_RETIRING, claim);
1220         goto denied_counted;
1221     }
1222     if (binding->contract_generation == 0 ||
1223         binding->contract_generation != record->generation) {
1224         agent_execution_result_error(
1225             result, AGENT_STATUS_STALE, "stale_contract",
1226             AGENT_EXECUTION_REASON_STALE_CONTRACT, claim);
1227         goto denied_counted;
1228     }
1229     if (binding->node_id >= record->node_count) {
1230         agent_execution_result_error(
1231             result, AGENT_STATUS_DENIED, "unknown_contract_node",
1232             AGENT_EXECUTION_REASON_UNKNOWN_NODE, claim);
1233         goto denied_counted;
1234     }
1235     node = &record->nodes[binding->node_id];
1236     runtime = &record->runtime[binding->node_id];
1237     claim->output_artifact_type = node->output_artifact_type;
1238     claim->charge_class = node->charge_class;
1239     contract_deadline = node->deadline_tick != 0 &&
1240         (record->deadline_tick == 0 ||
1241          node->deadline_tick < record->deadline_tick) ?
1242         node->deadline_tick : record->deadline_tick;
1243     claim->deadline_tick = request_deadline_tick != 0 ?
1244         request_deadline_tick : contract_deadline;
1245     claim->manifest.accepted_input_labels = node->accepted_input_labels;
1246     claim->manifest.output_add_labels = node->output_add_labels;
1247     claim->manifest.required_capabilities = node->required_capabilities;
1248     claim->manifest.side_effect_mask = node->side_effect_mask;
1249     for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++) {
1250         claim->exec_amounts[kind] = node->exec_envelope[kind];

```

真正触发副作用之前，内核再次确认节点仍在运行，请求号、尝试号、摘要与当前声明一致，并处理超时或取消的胜出分支。这个检查缩短了“执行约定通过”和“副作用实际发生”之间的时间差，避免状态过期后继续产生内核可见副作用。

代码 10：副作用开始前的执行约定检查

c

```

1781     }
1782     runtime->flags |= AGENT_EXECUTION_RUNTIME_F_TIMEOUT_REQUESTED;
1783     claim->deadline_tick = request_deadline_tick;
1784     claim->decision_reason = AGENT_EXECUTION_REASON_DEADLINE_EXPIRED;
1785     claim->retry_forbidden = 1;
1786     claim->input_provenance_labels = agent_provenance_current_labels(p);
1787     memmove(claim->request_digest, runtime->request_digest,
1788             sizeof(claim->request_digest));

```

```

1789     claim->active = 1;
1790     intr_restore(enabled);
1791     return AGENT_EXECUTION_ADMISSION_EXECUTE;
1792
1793 denied_counted:
1794     record->denial_count++;
1795     intr_restore(enabled);
1796     return AGENT_EXECUTION_ADMISSION_DENIED;
1797 }
1798
1799 int
1800 agent_execution_contract_claim_cached(
1801     struct agent_execution_claim *claim, struct agent_result *result,
1802     struct agent_execution_outcome *outcome)
1803 {
1804     struct agent_execution_contract_record *record;
1805     int copied = 0;
1806     int enabled;
1807
1808     if (claim == 0 || result == 0 || !claim->active || claim->legacy ||
1809         claim->slot < 0 || claim->slot >= WORKFLOW_LIFECYCLE_CAP)
1810         return 0;
1811     enabled = intr_save();
1812     record = &agent_execution_contracts[claim->slot];
1813     if (agent_execution_record_matches(record, claim->lifecycle) &&
1814         record->generation == claim->contract_generation)
1815         copied = agent_execution_cache_copy(
1816             record, claim->node_id, claim->attempt_id,
1817             claim->request_digest, result, outcome);
1818     if (copied)
1819         claim->active = 0;
1820     intr_restore(enabled);
1821     return copied;
1822 }
1823
1824 enum agent_execution_effect_admission
1825 agent_execution_contract_effect_begin(struct agent_execution_claim *claim)
1826 {
1827     struct agent_execution_contract_record *record;
1828     struct agent_execution_node_runtime *runtime;
1829     int enabled;
1830
1831     if (claim == 0 || !claim->active)

```

算法 3：一次 V3 工具调用的内核决策顺序

1. 内核验证固定大小 ABI、用户地址、智能体身份和工作流代次。
2. 内核查找工具描述符并按 schema 解码；失败时构造结构化拒绝结果，关键记录无法建立时返回 NO_SPACE 或 RETRY。
3. 接纳检查核对执行约定 DAG、输入来源、能力位、截止时间和资源额度。
4. 开始副作用前，内核再次确认当前声明；执行后写入上下文、执行记录、来源信息和完成结果。

3.8.3. 上下文、回滚边界与数据来源

上下文附属记录只保存归一化的 agent_op、结果和归因，不完整保留 V2 参数、V3 绑定或任务 SQE。镜像页按 prepare--begin--end 顺序发布；detail 在记录标为奇数前写入，序号只在一次清空后的时期内单调增加，清空后从 1 重新开始。因此，Guest 看到的只读页是稳定快照，而不是写到一半的记录。

代码 11：上下文附属记录的写入与发布窗口

c

```

1152 static int
1153 agent_context_append_flags(struct proc *p, struct agent_op *op,
1154                          struct agent_result *latest, uint64 tick,
1155                          uint64 flags, int authority_effect,
1156                          int causal_audit,
1157                          struct agent_evidence_context_reservation *reservation,
1158                          struct agent_evidence_security_reservation *
1159                          security_reservation,
1160                          uint64 *evidence_ticket_out,
1161                          uint64 *provenance_labels_out)
1162 {
1163     struct agent_context_append_receipt receipt;
1164     struct agent_context_detail detail;
1165     struct agent_context_record record;
1166     struct thread *t = curr_thread();
1167     uint64 slot;
1168
1169     if (p == 0 || op == 0 || latest == 0 || p->agent_ctx_base == 0 ||
1170         p->context_path_capacity == 0 || !agent_context_layout_ok() ||
1171         p->agent_current_span_id == 0 ||
1172         p->agent_current_span_owner == 0 || t == 0 || t->process != p ||
1173         p->agent_context_lane_owner_tid != t->tid ||
1174         p->agent_context_lane_depth == 0 ||
1175         latest->sequence != p->agent_call_count)
1176         return -1;
1177     if ((reservation != 0 && security_reservation != 0) ||
1178         ((reservation != 0 || security_reservation != 0) &&
1179          (evidence_ticket_out == 0 ||
1180           agent_observe_recording_suppressed(p))) ||
1181         (reservation != 0 && !reservation->active) ||
1182         (security_reservation != 0 && !security_reservation->active))
1183         return -1;
1184     slot = p->context_path_head % p->context_path_capacity;
1185
1186     memset(&record, 0, sizeof(record));
1187     record.sequence = latest->sequence;
1188     record.request_id = latest->request_id;
1189     record.cause_sequence = p->agent_current_cause_sequence;
1190     record.span_id = p->agent_current_span_id;
1191     record.branch_generation = p->context_branch_generation;
1192     record.path_parent_sequence = p->context_path_visible_head;
1193     record.arg0 = op->arg0;
1194     record.value0 = latest->value0;
1195     record.value1 = latest->value1;
1196     record.value2 = latest->value2;
1197     record.tick = tick;

```



```

1198     record.flags = agent_provenance_context_flags(
1199         p, latest->request_id, latest->tool_id, flags);
1200     if (provenance_labels_out != 0)
1201         *provenance_labels_out =
1202             AGENT_CONTEXT_PROVENANCE_DECODE(record.flags);
1203     if (strlen(op->payload) >= sizeof(record.payload) ||
1204         strlen(latest->result) >= sizeof(record.result))
1205         record.flags |= AGENT_CONTEXT_RECORD_F_TRUNCATED;
1206     record.tool_id = latest->tool_id;
1207     record.status = latest->status;
1208     safestrcpy(record.payload, op->payload, sizeof(record.payload));
1209     safestrcpy(record.result, latest->result, sizeof(record.result));
1210     record.prev_hash = p->agent_context_chain_hash;
1211     record.record_hash = agent_context_record_hash(&record);
1212     if (agent_context_append_receipt_prepare(p, &record, slot, &receipt) < 0)
1213         return -1;
1214
1215     memset(&detail, 0, sizeof(detail));
1216     detail.sequence = latest->sequence;
1217     detail.flags = record.flags;
1218     memmove(&detail.op, op, sizeof(*op));
1219     memmove(&detail.result, latest, sizeof(*latest));
1220     if (agent_context_store(p, slot, &detail,
1221         p->agent_current_span_owner,
1222         p->agent_current_cause_pid,
1223         p->agent_current_cause_control) < 0)
1224         return -1;
1225     agent_context_publish_begin(p);
1226     agent_context_write_record(p, slot, &record);
1227     if (p->context_path_count < p->context_path_capacity) {
1228         if (p->context_path_count == 0)
1229             p->context_path_oldest = latest->sequence;
1230         p->context_path_count++;
1231     } else {
1232         p->context_path_oldest =
1233             latest->sequence - p->context_path_capacity + 1;
1234     }
1235     p->context_path_latest = latest->sequence;
1236     p->context_path_head = (slot + 1) % p->context_path_capacity;
1237     if (record.cause_sequence != 0)
1238         p->agent_provenance_edges++;
1239     if (agent_context_append_receipt_commit(p, &record, &receipt) < 0)
1240         panic("Agent context append receipt");
1241     p->agent_context_chain_hash = record.record_hash;
1242     p->agent_current_cause_sequence = latest->sequence;
1243     p->context_cause_branch_generation = record.branch_generation;
1244     p->agent_current_cause_pid = p->pid;
1245     p->agent_current_cause_control = p->agent_control_id;
1246     p->agent_current_span_id = record.span_id;
1247     agent_provenance_context_committed(
1248         p, record.request_id, record.tool_id, record.flags);
1249     if (agent_context_write_latest(p, latest) < 0 ||
1250         agent_context_write_header_locked(p) < 0)
1251         panic("Agent context direct publish");
1252     /* Reserved terminal and denial records stay behind an odd sequence until
1253      * their canonical Evidence ticket is committed. The release below is the
1254      * only point at which direct readers may accept the new Context snapshot. */

```

```

1255     if (reservation != 0)
1256         *evidence_ticket_out =
1257             agent_observe_commit_context_reserved_ticket(
1258                 p, &record, authority_effect, causal_audit,
1259                 reservation);
1260     else if (security_reservation != 0)
1261         *evidence_ticket_out =
1262             agent_observe_commit_security_reserved_ticket(
1263                 p, &record, security_reservation);
1264     agent_context_publish_end(p);

```

执行入口在执行约定接纳之前为上下文追加记录做准备；输入来源标签随执行约定声明合并到当前进程。面对截止时间、依赖失败和策略拒绝，代码会尝试生成完成记录或安全记录，让调试和测试区分“未执行”“拒绝”和“执行后失败”。执行记录无法建立时，直接调用和 V3 调用的拒绝仍以 NO_SPACE 或 RETRY 表达失败。手工注记、回滚和清空不使用执行记录的预留与提交票号规则。

代码 12：执行入口中的上下文准备与来源合并

c

```

1577     }
1578     agent_execution_contract_complete(claim, res, outcome);
1579     return 0;
1580 }
1581
1582 static int
1583 agent_execute_one(struct proc *p, struct agent_op *op,
1584                 struct agent_result *res, uint64 tick,
1585                 const struct agent_execution_binding *binding,
1586                 uint64 request_deadline_tick, uint admission_policy_flags,
1587                 struct agent_execution_outcome *outcome)
1588 {
1589     struct agent_execution_claim claim;
1590     struct agent_provenance_request provenance_request;
1591     struct agent_provenance_decision provenance_decision;
1592     struct agent_evidence_context_reservation evidence_reservation = { 0 };
1593     struct resource_phase_lease phase_lease = { 0 };
1594     struct resource_account_handle storage = resource_account_none();
1595     enum agent_execution_admission admission;
1596     enum agent_execution_effect_admission effect_admission;
1597     int bound = binding != 0;
1598     int phase_started = 0;
1599     uint64 phase_generation = 0;
1600     uint64 terminal_tick = tick;
1601     int result = 0;
1602     int status;
1603
1604     if (outcome != 0)
1605         memset(outcome, 0, sizeof(*outcome));
1606     op->payload[AGENT_OP_PAYLOAD_SIZE - 1] = 0;
1607     agent_result_init(res, op);
1608     if (agent_lifecycle_context_lane_enter(p) < 0) {
1609         res->status = AGENT_STATUS_NO_SPACE;
1610         agent_result_text(res, "context_interrupted");
1611         return -1;

```

```

1612     }
1613     if (agent_context_append_prepare(p, p->agent_call_count + 1) < 0) {
1614         res->status = AGENT_STATUS_NO_SPACE;
1615         agent_result_text(res, "context_sync_unavailable");
1616         agent_lifecycle_context_lane_leave(p);
1617         return -1;
1618     }
1619     admission = agent_execution_contract_admit(
1620         p, binding, op, request_deadline_tick, admission_policy_flags,
1621         tick, &claim, res, outcome);
1622     if (bound && admission == AGENT_EXECUTION_ADMISSION_EXECUTE &&
1623         claim.input_provenance_labels != 0 &&
1624         agent_provenance_merge_current(
1625             p, claim.input_provenance_labels) != AGENT_STATUS_OK)
1626         panic("contract input provenance merge");
1627     if (outcome != 0 && admission != AGENT_EXECUTION_ADMISSION_CACHED) {
1628         outcome->output_artifact_type = claim.output_artifact_type;
1629         outcome->terminal_tick = terminal_tick;
1630     }
1631     if (admission == AGENT_EXECUTION_ADMISSION_DENIED) {
1632         if (res->status == AGENT_STATUS_DENIED ||
1633             res->status == AGENT_STATUS_STALE) {
1634             result = agent_execution_security_denial(
1635                 p, &claim, op, bound, res, outcome);
1636             agent_execution_publish_policy_denial(
1637                 p, op, res, &claim);
1638             goto out;
1639         }
1640         p->agent_call_count++;
1641         res->sequence = p->agent_call_count;
1642         result = agent_context_append(p, op, res, tick, 0);
1643         if (result < 0)
1644             p->agent_call_count--;
1645         goto out;
1646     }
1647     if (admission == AGENT_EXECUTION_ADMISSION_CACHED) {
1648         if (outcome != 0)
1649             outcome->completion_flags |= AGENT_RESPONSE_V3_F_CACHED;
1650         goto out;
1651     }
1652     if (claim.deadline_expired || claim.dependency_failed) {
1653         if (agent_evidence_context_reserve(
1654             p, &evidence_reservation) < 0) {
1655             res->status = AGENT_STATUS_RETRY;
1656             agent_result_text(res, "evidence_reservation_unavailable");
1657             agent_execution_contract_abort(&claim);
1658             goto out;
1659         }
1660         p->agent_call_count++;
1661         res->sequence = p->agent_call_count;
1662         res->status = claim.deadline_expired ?
1663             AGENT_STATUS_TIMEOUT : AGENT_STATUS_CANCELLED;
1664         agent_result_text(
1665             res, claim.deadline_expired ?
1666                 "contract_deadline" : "dependency_failed");
1667         if (outcome != 0) {
1668             outcome->decision_reason = claim.decision_reason;

```

回滚接口只恢复上下文路径的可见记录和分支状态，不会撤销已经发生的文件写入、消息发送或外部服务调用。因此，上下文回滚用于修正后续推理和可见因果关系，不承担通用事务回滚的职责。

3.8.4. 元数据查询与实时查询事件

文件查询先确定执行方式：调用者可以明确要求遍历、关闭索引，或者在启用索引时让内核按固定的 `status->stage->kind` 顺序选择第一个给定字段。选择时不比较基数。随后，内核在目录读取快照中扫描候选记录，检查访问范围和筛选条件，并定期让出内核工作预算。结果同时返回命中总数、实际返回数、truncated、是否使用索引、原因、查询时钟数和文件系统代次，供用户态解释本次查询。

代码 13：元数据查询的索引计划与快照执行

c

```

143 static void
144 agent_query_plan_build(const struct agent_file_query *q,
145                       struct agent_file_query_result *r,
146                       struct agent_query_plan *plan)
147 {
148     memset(plan, 0, sizeof(*plan));
149     plan->limit = q->max_hits;
150     if (plan->limit <= 0 || plan->limit > AGENT_FILE_QUERY_MAX_HITS)
151         plan->limit = AGENT_FILE_QUERY_MAX_HITS;
152     r->plan = AGENT_FILE_QUERY_PLAN_SCAN;
153     r->index_bucket = -1;
154     if (q->flags & AGENT_FILE_QUERY_SCAN) {
155         plan->force_scan = 1;
156         plan->reason |= AGENT_FILE_QUERY_REASON_FORCED_SCAN;
157         return;
158     }
159     if ((q->flags & AGENT_FILE_QUERY_USE_INDEX) == 0) {
160         plan->reason |= AGENT_FILE_QUERY_REASON_INDEX_OFF;
161         return;
162     }
163     for (int i = AGENT_QUERY_INDEX_FIRST; i < AGENT_QUERY_FIELDS; i++) {
164         const struct agent_query_alias *alias = &agent_query_aliases[i];
165         const char *value = (const char *)q + alias->offset;
166
167         if (!value[0])
168             continue;
169         plan->index = AGENT_CATALOG_INDEX_STATUS +
170             i - AGENT_QUERY_INDEX_FIRST;
171         plan->use_index = 1;
172         plan->index_key = value;
173         r->plan = AGENT_FILE_QUERY_PLAN_STATUS_INDEX +
174             i - AGENT_QUERY_INDEX_FIRST;
175         plan->reason |= AGENT_FILE_QUERY_REASON_STATUS_INDEX <<
176             (i - AGENT_QUERY_INDEX_FIRST);
177         return;
178     }
179     plan->reason |= AGENT_FILE_QUERY_REASON_NO_INDEX_KEY;
180 }
181
182 static void agent_query_result_reset(struct agent_file_query_result *r)

```

```

183 {
184     memset(r, 0, sizeof(*r));
185 }
186
187 int
188 agent_metadata_query_execute_snapshot(
189     uint scope, const struct agent_file_query *q,
190     struct agent_file_query_result *r)
191 {
192     struct agent_catalog_read_snapshot snapshot;
193     struct agent_file_meta meta;
194     struct agent_query_plan plan;
195     uint owner;
196     uint work = 0;
197     int bucket = -1;
198     int copied;
199     uint64 start;
200
201     agent_query_result_reset(r);
202     agent_query_plan_build(q, r, &plan);
203     start = agent_file_state_now();
204     copied = agent_metadata_catalog_read_begin(
205         scope, plan.index, plan.index_key, plan.force_scan,
206         &snapshot, &bucket);
207     if (copied < 0)
208         return copied;
209     if (plan.use_index)
210         r->index_bucket = bucket;
211     for (int slot = agent_metadata_catalog_read_next(&snapshot, -1);
212          slot >= 0;
213          slot = agent_metadata_catalog_read_next(&snapshot, slot)) {
214         r->scanned_records++;
215         copied = agent_metadata_catalog_read_copy(
216             &snapshot, slot, &meta, &owner);
217         if (copied < 0)
218             return copied;
219         if (copied > 0 && agent_object_scope_visible(scope, owner)) {
220             r->candidate_records++;
221             if (agent_metadata_query_matches(scope, owner, q, &meta)) {
222                 r->total_hits++;
223                 if (r->returned < plan.limit) {
224                     agent_file_state_project_hit(
225                         &r->hits[r->returned++],
226                         &meta, owner);
227                 } else {
228                     r->truncated = 1;
229                 }
230             }
231         }
232         if (++work == AGENT_QUERY_READ_GRANULE) {
233             work = 0;
234             (void)kernel_work_checkpoint_cleanup(
235                 KERNEL_WORK_OPERATION_UNITS);
236         }
237     }
238     if (!agent_metadata_catalog_read_end(&snapshot))
239         return AGENT_CATALOG_STALE;

```

```

240     r->used_index = plan.use_index;
241     r->index_rebuild_records = 0;
242     r->query_ticks = agent_file_state_now() - start;
243     r->plan_reason = plan.reason;
244     r->fs_generation = snapshot.fs_generation;
245     return r->returned;

```

实时查询根据元数据变化前后的状态生成 ENTER、UPDATE、LEAVE 三类事件。订阅只保存筛选条件和代次，不保存结果集；每个对象都会先检查工作流和访问范围是否可见。只有投递失败或队列出现缺口才需要重新同步，正常的文件系统代次变化仍持续投递三类增量。安全恢复时，智能体保留旧订阅，先按相同条件安装不带 ACK_RESYNC 的替代订阅，取得 truncated=0 的有界快照后，才在旧订阅上确认缺口并取消订阅；替代订阅覆盖切换窗口。Workflow Fence 会排空延迟工作；只要工作流或访问范围内的黏性重新同步尚未确认，即使标记已投递也返回 RETRY，最后的回收才会清理残留。

代码 14：实时查询的 ENTER、UPDATE、LEAVE 事件投递

c

```

1250     struct workflow_lifecycle_key source_key =
1251         agent_live_query_proc_key(source);
1252     uint source_scope = agent_identity_proc_scope(source);
1253     uint64 fid;
1254     int delivered = 0;
1255
1256     if (!agent_live_query_domain_valid(key, owner_scope) ||
1257         identity == 0 || !identity->used || identity->fid <= 0)
1258         return 0;
1259     if (generation == 0)
1260         generation = agent_file_state_scope_generation(owner_scope);
1261     fid = (uint64)(uint)identity->fid;
1262     for (struct proc *target = pool; target < &pool[NPROC]; target++) {
1263         struct workflow_lifecycle_key target_key;
1264         struct proc *attributed = 0;
1265         uint target_scope;
1266         uint legacy_changes;
1267         uint typed_changes;
1268         int enabled = intr_save();
1269
1270         if (!agent_live_query_target_visible(target, key, owner_scope) ||
1271             !agent_live_query_target_valid(
1272                 target, &target_key, &target_scope)) {
1273             intr_restore(enabled);
1274             continue;
1275         }
1276         legacy_changes = agent_live_query_target_changes(
1277             target, owner_scope, before, after);
1278         typed_changes = agent_live_query_typed_target_changes(
1279             target, owner_scope, before, after);
1280         if (legacy_changes == 0 && typed_changes == 0) {
1281             intr_restore(enabled);
1282             continue;
1283         }
1284         if (source != 0 && source_scope == target_scope &&
1285             agent_live_query_key_equal(source_key, target_key))
1286             attributed = source;

```



```

1287     intr_restore(enabled);
1288     for (uint stream = 0; stream < 2; stream++) {
1289         uint changes = stream == 0 ? legacy_changes : typed_changes;
1290         int event_type = stream == 0 ? AGENT_EVENT_FILE_STATUS :
1291             AGENT_EVENT_FILE_QUERY;
1292
1293         for (int change = AGENT_LIVE_QUERY_ENTER;
1294             change <= AGENT_LIVE_QUERY_LEAVE; change++) {
1295             char payload[AGENT_EVENT_PAYLOAD_SIZE];
1296             int result;
1297
1298             if ((changes & (1U << change)) == 0)
1299                 continue;
1300             agent_live_query_payload(payload, change, key);
1301             result = agent_ipc_deliver_live_event(
1302                 target, attributed, event_type, fid,
1303                 generation, payload, 0);
1304             if (result > 0) {
1305                 delivered += result;
1306                 continue;
1307             }
1308             if (result < 0) {
1309                 enabled = intr_save();
1310                 agent_live_query_proc_resync_mark_locked(
1311                     target, generation,
1312                     1U << event_type);
1313                 intr_restore(enabled);
1314                 (void)agent_live_query_proc_resync_flush(
1315                     target);
1316             }
1317         }
1318     }
1319 }
1320 return delivered;
1321 }

```

代码 15: 实时查询的重新同步确认与 Workflow Fence 入口

c

```

1323 void
1324 agent_live_query_watch_installed(struct proc *p)
1325 {
1326     struct workflow_lifecycle_key key;
1327     uint64 generation;
1328     uint scope_id;
1329     int enabled = intr_save();
1330
1331     if (!agent_live_query_target_valid(p, &key, &scope_id) ||
1332         !agent_live_query_has_file_watch(p))
1333         goto out;
1334     generation = agent_file_state_scope_generation(scope_id);
1335     generation = MAX(generation,
1336         agent_live_query_domain_generation_locked(key, scope_id));
1337     agent_live_query_proc_resync_mark_locked(
1338         p, generation, 1U << AGENT_EVENT_FILE_STATUS);
1339     (void)agent_live_query_proc_resync_flush(p);
1340 out:

```

```

1341     intr_restore(enabled);
1342 }
1343
1344 void
1345 agent_live_query_watch_removed(struct proc *p)
1346 {
1347     int enabled;
1348     uint slot;
1349
1350     if (p == 0 || p < pool || p >= &pool[NPROC])
1351         return;
1352     enabled = intr_save();
1353     if (!agent_live_query_has_file_watch(p)) {
1354         slot = p - pool;
1355         agent_live_query_proc_resync_clear(
1356             &agent_live_query_proc_resync[slot]);
1357     }
1358     intr_restore(enabled);
1359 }
1360
1361 void
1362 agent_live_query_proc_reset(struct proc *p)
1363 {
1364     int enabled;
1365
1366     if (p == 0 || p < pool || p >= &pool[NPROC])
1367         return;
1368     enabled = intr_save();
1369     for (uint slot = 0; slot < AGENT_LIVE_QUERY_SUBSCRIPTION_CAP; slot++)
1370         if (agent_live_query_subscriptions[slot].used &&
1371             agent_live_query_subscriptions[slot].target == p)
1372             agent_live_query_subscription_remove_locked(
1373                 &agent_live_query_subscriptions[slot], 0);
1374     agent_live_query_proc_resync_clear(
1375         &agent_live_query_proc_resync[p - pool]);
1376     intr_restore(enabled);
1377 }
1378
1379 int
1380 agent_live_query_resync_ack(struct workflow_lifecycle_key key,
1381     uint scope_id, uint64 generation)
1382 {
1383     int acknowledged = 0;
1384     int enabled;
1385
1386     if (!agent_live_query_domain_valid(key, scope_id) || generation == 0)
1387         return 0;
1388     enabled = intr_save();
1389     for (uint slot = 0; slot < AGENT_LIVE_QUERY_DOMAIN_CAP; slot++) {
1390         struct agent_live_query_domain_resync *state =
1391             &agent_live_query_domain_resync[slot];
1392
1393         if (!state->used || state->generation > generation ||
1394             !agent_live_query_domain_equal(
1395                 state->key, state->scope_id, key, scope_id))
1396             continue;
1397         memset(state, 0, sizeof(*state));

```

```

1398     acknowledged = 1;
1399 }
1400 if (scope_id == VFS_SCOPE_SYSTEM &&
1401     agent_live_query_global_resync_generation != 0 &&
1402     agent_live_query_global_resync_generation <= generation) {
1403     agent_live_query_global_resync_generation = 0;
1404     acknowledged = 1;
1405 }
1406 intr_restore(enabled);
1407 return acknowledged;
1408 }
1409
1410 static int
1411 agent_live_query_proc_resync_pending_domain(
1412     struct workflow_lifecycle_key key, uint scope_id)
1413 {
1414     for (struct proc *target = pool; target < &pool[NPROC]; target++) {
1415         struct agent_live_query_proc_resync *pending =
1416             &agent_live_query_proc_resync[target - pool];
1417
1418         if (!agent_live_query_proc_resync_valid(pending, target))
1419             continue;
1420         if (scope_id == VFS_SCOPE_SYSTEM ||
1421             agent_live_query_domain_equal(
1422                 pending->key, pending->scope_id, key, scope_id))
1423             return 1;
1424     }
1425     return 0;
1426 }
1427
1428 int
1429 agent_live_query_fence_drain(struct workflow_lifecycle_key key, uint scope_id)
1430 {
1431     int retry = 0;
1432
1433     if (!agent_live_query_domain_valid(key, scope_id) ||
1434         !agent_metadata_txn_owned(0))
1435         return AGENT_STATUS_RETRY;
1436     for (uint slot = 0; slot < AGENT_LIVE_QUERY_TOMBSTONE_CAP; slot++) {
1437         int pending;
1438         int enabled = intr_save();
1439
1440         pending = agent_live_query_tombstones[slot].used &&
1441             agent_live_query_domain_equal(
1442                 agent_live_query_tombstones[slot].key,
1443                 agent_live_query_tombstones[slot].scope_id,
1444                 key, scope_id);
1445         intr_restore(enabled);
1446         if (!pending)
1447             continue;
1448         if (agent_live_query_tombstone_process(slot) < 0)
1449             retry = 1;
1450     }
1451     for (uint slot = 0; slot < AGENT_FILE_META_MAX; slot++) {
1452         int pending;
1453         int enabled = intr_save();
1454     }

```

```

1455     pending = agent_live_query_content_pending[slot].used &&
1456         agent_live_query_domain_equal(
1457             agent_live_query_content_pending[slot]
1458                 .receipt.lifecycle,
1459             agent_live_query_content_pending[slot]
1460                 .receipt.scope_id,
1461             key, scope_id);
1462     intr_restore(enabled);
1463     if (!pending)
1464         continue;
1465     if (agent_live_query_content_process(slot) < 0)
1466         retry = 1;
1467 }
1468 for (struct proc *target = pool; target < &pool[NPROC]; target++) {

```

3.8.5. 任务通道与等待路径

任务通道设置只允许智能体主线程使用；SQ 和 CQ 各占一个 Guest 映射页，复制后的请求和完成记录，以及权威资源状态分别放在两页内核私有页。进入操作和资源绑定会记录发起线程编号和身份代次。进入内核后，代码仍检查进入 ABI、CQ 确认、重新同步标志和 SQ 边界。发起者、创建者或生命周期不匹配导致的 STALE 只保留 ABI 版本、大小和 status，其他输出为零；控制代次不匹配时，结果填充函数返回当前通道状态。CQ_FULL 只是完成队列背压，不会把已接纳的请求直接变为完成记录。目标已确认或不存在时，取消操作仍会消费取消编号，结果填充函数返回当前代次和水位线，也不会额外生成取消 CQE。环形队列或槽位协议错误进入黏性重新同步后，发起者必须以空 SQ 尾指针的显式重新同步进入操作复位。这里的队列是受执行约定和执行记录约束的内核对象，不是普通用户态环形数组。

代码 16：任务通道进入操作的代次、重新同步与 SQ 边界检查

c

```

2012     return agent_task_consumed_after_expire(status);
2013 }
2014 if (hook_status == AGENT_TASK_HOOK_COMPLETE ||
2015     hook_status == AGENT_TASK_HOOK_DENIED) {
2016     status = agent_task_channel_complete(
2017         p, sqe.ring_generation, sqe.request_id,
2018         sqe.slot_generation, &completion, ops);
2019     if (status == AGENT_TASK_CHANNEL_OK)
2020         return 1;
2021     return status;
2022 }
2023 if (hook_status != AGENT_TASK_HOOK_PENDING)
2024     return AGENT_TASK_CHANNEL_EVIDENCE;
2025 return 1;
2026 }
2027
2028 int
2029 agent_task_channel_enter(struct proc *p, struct thread *issuer,
2030     const struct agent_task_channel_enter *enter,
2031     struct agent_task_channel_enter_result *result,
2032     const struct agent_task_channel_ops *ops)
2033 {

```

```

2034     struct agent_task_channel_state *state;
2035     struct agent_task_private_header *header;
2036     uint submitted = 0;
2037     uint completed = 0;
2038     uint limit;
2039     int status = AGENT_TASK_CHANNEL_OK;
2040     int enabled;
2041
2042     if (result == 0)
2043         return AGENT_TASK_CHANNEL_BAD_REQUEST;
2044     memset(result, 0, sizeof(*result));
2045     result->version = AGENT_TASK_CHANNEL_VERSION;
2046     result->size = sizeof(*result);
2047     result->status = AGENT_TASK_CHANNEL_BAD_REQUEST;
2048     if (p == 0 || issuer == 0 || enter == 0 ||
2049         enter->version != AGENT_TASK_CHANNEL_VERSION ||
2050         enter->size != sizeof(*enter) ||
2051         (enter->flags & ~AGENT_TASK_CHANNEL_ENTER_F_ALL) != 0 ||
2052         ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_DRAIN) != 0 &&
2053          enter->max_submit != 0) ||
2054         enter->max_submit > AGENT_TASK_CHANNEL_CAPACITY ||
2055         enter->min_complete > AGENT_TASK_CHANNEL_CAPACITY ||
2056         enter->reserved != 0 || enter->reserved_tail[0] != 0 ||
2057         enter->reserved_tail[1] != 0)
2058         return result->status;
2059     limit = (enter->flags & AGENT_TASK_CHANNEL_ENTER_F_DRAIN) != 0 ? 0 :
2060         (enter->max_submit == 0 ? AGENT_TASK_CHANNEL_CAPACITY :
2061          enter->max_submit);
2062     enabled = intr_save();
2063     state = agent_task_channel_find_locked(p);
2064     if (!agent_task_channel_issuer_valid_locked(state, p, issuer) ||
2065         state->ops != ops) {
2066         intr_restore(enabled);
2067         result->status = AGENT_TASK_CHANNEL_STALE;
2068         return result->status;
2069     }
2070     header = &state->private->header;
2071     if (enter->generation != header->generation) {
2072         agent_task_channel_enter_result_fill(
2073             state, result, AGENT_TASK_CHANNEL_STALE, 0, 0);
2074         intr_restore(enabled);
2075         return result->status;
2076     }
2077     status = agent_task_channel_ack_locked(state, enter->cq_head);
2078     if (status < 0) {
2079         agent_task_channel_enter_result_fill(state, result, status, 0, 0);
2080         intr_restore(enabled);
2081         return result->status;
2082     }
2083     header = &state->private->header;
2084     if ((header->flags & AGENT_TASK_CHANNEL_RING_F_RESYNC) != 0) {
2085         if ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_RESYNC) == 0 ||
2086             enter->sq_tail != 0) {
2087             agent_task_channel_enter_result_fill(
2088                 state, result,
2089                 AGENT_TASK_CHANNEL_RESYNC_REQUIRED, 0, 0);
2090             intr_restore(enabled);

```

```

2091     return result->status;
2092 }
2093 header->flags &= ~AGENT_TASK_CHANNEL_RING_F_RESYNC;
2094 } else if ((enter->flags & AGENT_TASK_CHANNEL_ENTER_F_RESYNC) != 0) {
2095     agent_task_channel_enter_result_fill(
2096         state, result, AGENT_TASK_CHANNEL_BAD_REQUEST, 0, 0);
2097     intr_restore(enabled);
2098     return result->status;
2099 }
2100 if (enter->sq_tail < header->sq_tail ||
2101     enter->sq_tail < header->sq_head ||
2102     enter->sq_tail - header->sq_head > AGENT_TASK_CHANNEL_CAPACITY) {
2103     status = agent_task_channel_protocol_fault_locked(state);
2104     agent_task_channel_enter_result_fill(state, result, status, 0, 0);
2105     intr_restore(enabled);
2106     return result->status;
2107 }
2108 header->sq_tail = enter->sq_tail;
2109 completed += agent_task_channel_flush_locked(state);
2110 agent_task_channel_publish_locked(state);
2111 intr_restore(enabled);
2112 if (ops != 0 && ops->expire != 0) {
2113     status = agent_task_channel_expire(p, agent_task_now(), ops);
2114     if (status < 0)
2115         goto finish;
2116     status = AGENT_TASK_CHANNEL_OK;
2117 }
2118 while (submitted < limit) {
2119     uint consumed = 0;
2120
2121     status = agent_task_channel_consume_one(
2122         p, issuer, enter->generation, ops, &consumed);
2123     submitted += consumed;
2124     if (status <= 0)
2125         break;
2126     if (consumed != 1)
2127         panic("Task Channel consumed result");
2128 }
2129 finish:
2130     enabled = intr_save();
2131     state = agent_task_channel_find_locked(p);
2132     if (!agent_task_channel_issuer_valid_locked(state, p, issuer) ||
2133         state->ops != ops) {

```

资源导入先根据文件描述符取得可读普通文件，再从 offset 0 读取请求长度并额外探测一个字节。长度、精确 EOF、NUL 与 UTF-8 校验全部通过后，内核把内容、SHA-256、最新非零 Context 序号和 provenance 写入通道私有槽位。IMPORT 本身不追加 Context，不会推进共享文件 offset，也不会把文件对象指针保存到任务句柄中。

SQE 引用资源时仍要核对 slot、type 和 generation。BORROWED 任务只在执行期间取得引用，完成后资源继续可用；OWNED 任务则在终态 CQE（terminal CQE）发布后消费输入。显式释放、自动消费和槽位复用都会推进 generation，旧句柄因此返回 STALE。如果 IMPORT 的最终 copyout 失败，刚建立且尚未对 Guest 可见的资源会被撤销。

代码 17: Task UTF-8 文件 snapshot 的校验与来源绑定

c

```

2085 static int
2086 agent_task_bridge_resource_import (
2087     struct proc *p, struct file *file,
2088     const struct agent_task_channel_resource *control,
2089     struct agent_task_resource_import *imported)
2090 {
2091     struct agent_context_record context;
2092     struct open_file_io_token lease = OPEN_FILE_IO_TOKEN_INIT;
2093     struct vfs_cred cred;
2094     uint64 context_hash;
2095     uint64 context_sequence;
2096     uint64 provenance_labels;
2097     uint length;
2098     int got;
2099     int context_lane_held = 0;
2100     int lease_held = 0;
2101     int status = AGENT_TASK_CHANNEL_BAD_REQUEST;
2102
2103     if (imported == 0)
2104         return status;
2105     memset(imported, 0, sizeof(*imported));
2106     if (p == 0 || control == 0 || !p->is_agent || p->pid <= 0 ||
2107         p->agent_control_id == 0 ||
2108         (control->resource_type != AGENT_ARTIFACT_UTF8 &&
2109          control->resource_type != AGENT_ARTIFACT_TASK) ||
2110         control->resource_flags != AGENT_TASK_HANDLE_F_OWNED ||
2111         control->source_handle >= FD_BUFFER_SIZE || control->length == 0 ||
2112         control->length > AGENT_TASK_RESOURCE_UTF8_MAX ||
2113         (control->resource_type == AGENT_ARTIFACT_TASK &&
2114          control->length != sizeof(struct agent_task_delegate_descriptor)) ||
2115         file == 0)
2116         return status;
2117     length = (uint)control->length;
2118     if (!file->readable || file->type != FD_INODE || file->ip == 0) {
2119         status = AGENT_TASK_CHANNEL_BAD_REQUEST;
2120         goto out;
2121     }
2122     got = ivalid(file->ip);
2123     if (got < 0) {
2124         status = got == FS_LOOKUP_BUSY ? AGENT_TASK_CHANNEL_RETRY :
2125             AGENT_TASK_CHANNEL_EVIDENCE;
2126         goto out;
2127     }
2128     if (file->ip->type != T_FILE) {
2129         status = AGENT_TASK_CHANNEL_BAD_REQUEST;
2130         goto out;
2131     }
2132     vfs_cred_from_proc(p, &cred);
2133     if (!vfs_inode_authorize(file->ip, &cred, VFS_OP_READ)) {
2134         status = AGENT_TASK_CHANNEL_DENIED;
2135         goto out;
2136     }
2137     if (agent_lifecycle_context_lane_enter(p) < 0) {
2138         status = AGENT_TASK_CHANNEL_RETRY;

```

```

2139     goto out;
2140 }
2141 context_lane_held = 1;
2142 context_sequence = p->context_path_latest;
2143 if (context_sequence == 0 || p->context_path_count == 0 ||
2144     p->context_path_capacity == 0 ||
2145     context_sequence < p->context_path_oldest ||
2146     agent_context_read_record(
2147         p, (context_sequence - 1U) % p->context_path_capacity,
2148         &context) < 0 ||
2149     context.sequence != context_sequence || context.record_hash == 0 ||
2150     context.record_hash != agent_context_record_hash(&context)) {
2151     status = AGENT_TASK_CHANNEL_EVIDENCE;
2152     goto out;
2153 }
2154 context_hash = context.record_hash;
2155 provenance_labels = agent_provenance_current_labels(p);
2156 if (open_file_io_lease_acquire(
2157     file, VFS_OP_READ, &lease, &cred) < 0) {
2158     status = AGENT_TASK_CHANNEL_RETRY;
2159     goto out;
2160 }
2161 lease_held = 1;
2162 got = readi_lease(file->ip, &cred, &lease, 0,
2163     (uint64)imported->snapshot, 0, length + 1U);
2164 if (got < 0) {
2165     status = got == FS_LOOKUP_BUSY ? AGENT_TASK_CHANNEL_RETRY :
2166         AGENT_TASK_CHANNEL_EVIDENCE;
2167     goto out;
2168 }
2169 if ((uint)got != length) {
2170     status = AGENT_TASK_CHANNEL_BAD_REQUEST;
2171     goto out;
2172 }
2173 if (p->context_path_latest != context_sequence ||
2174     p->context_path_count == 0 || p->context_path_capacity == 0 ||
2175     context_sequence < p->context_path_oldest ||
2176     agent_context_read_record(
2177         p, (context_sequence - 1U) % p->context_path_capacity,
2178         &context) < 0 ||
2179     context.sequence != context_sequence ||
2180     context.record_hash != context_hash ||
2181     context.record_hash != agent_context_record_hash(&context) ||
2182     agent_provenance_current_labels(p) != provenance_labels) {
2183     status = AGENT_TASK_CHANNEL_RETRY;
2184     goto out;
2185 }
2186 if (control->resource_type == AGENT_ARTIFACT_UTF8) {
2187     for (uint i = 0; i < length; i++) {
2188         if (imported->snapshot[i] == 0) {
2189             status = AGENT_TASK_CHANNEL_BAD_REQUEST;
2190             goto out;
2191         }
2192     }
2193     if (!agent_task_bridge_utf8_valid(imported->snapshot, length)) {
2194         status = AGENT_TASK_CHANNEL_BAD_REQUEST;
2195         goto out;

```

```

2196     }
2197     imported->snapshot[length] = 0;
2198 } else {
2199     struct agent_task_delegate_descriptor descriptor;
2200
2201     memmove(&descriptor, imported->snapshot, sizeof(descriptor));
2202     if (!agent_task_bridge_delegate_descriptor_valid(&descriptor)) {
2203         status = AGENT_TASK_CHANNEL_BAD_REQUEST;
2204         goto out;
2205     }
2206 }
2207 status = AGENT_TASK_CHANNEL_OK;
2208
2209 out:
2210 if (lease_held)
2211     open_file_io_token_end(&lease);
2212 if (context_lane_held)
2213     agent_lifecycle_context_lane_leave(p);
2214 if (status != AGENT_TASK_CHANNEL_OK) {
2215     memset(imported, 0, sizeof(*imported));
2216     return status;
2217 }
2218 imported->resource_type = control->resource_type;
2219 imported->resource_flags = AGENT_TASK_HANDLE_F_OWNED;
2220 imported->producer_node_id = AGENT_EXECUTION_NODE_NONE;
2221 imported->producer_pid = p->pid;
2222 imported->producer_control_id = p->agent_control_id;
2223 imported->source_handle = control->source_handle;
2224 imported->length = length;
2225 imported->provenance_labels =
2226     provenance_labels | AGENT_PROVENANCE_UNTRUSTED_FILE_DATA;
2227 imported->producer_context_sequence = context_sequence;
2228 agent_sha256(imported->snapshot, length, imported->content_digest);
2229 return AGENT_TASK_CHANNEL_OK;
2230 }

```

第 26 项 `delegate_task` 还允许资源槽保存准确的 56 字节委派描述符。目标 Agent 必须持有 TASK 接收能力和独立路由；系统调用 567 在 `claim` 时复核 `owner`、目标身份、生命周期、`channel/request/slot generation` 并建立 `delegated effect lease`。系统调用 568 除了处理正常 `complete` 和带 `ACK_TERMINAL` 的协作式终态确认，还允许同一活动生命周期内的 `controller` 以 `AGENT_TASK_DELEGATE_COMPLETE_F_REQUEST_CANCEL` 请求取消。该 `control request` 必须使用 `CANCELLED`、零 `ACK` 字段和准确的 `owner/channel/request/slot/task/correlation tuple`；内核同时要求 `AGENT_CAP_ORCHESTRATE`、`AGENT_CAP_WAIT_CANCEL` 以及指向 `owner` 的活动 TASK route。返回 OK 只确认 `control request`；已处于 `CLAIMED` 的 `provider` 仍要清理输出并确认最新终态 `offer`。只有最新终态被确认且 `lease` 引用归零，`owner` 才会取得唯一 `terminal CQE`。

代码 18: delegated Task 的 claim 身份、路由与代次核验

c

```

611 agent_task_delegate_claim_prepare(
612     struct proc *worker, struct thread *thread,
613     const struct agent_task_delegate_claim *claim,
614     struct agent_task_delegate_claim_result *result)

```

```

615 {
616     struct workflow_lifecycle_key lifecycle;
617     int enabled;
618
619     agent_task_delegate_claim_result_fill(
620         0, result, AGENT_TASK_CHANNEL_BAD_REQUEST,
621         AGENT_TASK_DELEGATE_STATE_NONE);
622     if (worker == 0 || thread == 0 || claim == 0 ||
623         claim->version != AGENT_TASK_DELEGATE_VERSION ||
624         claim->size != sizeof(*claim) ||
625         (claim->flags & ~AGENT_TASK_DELEGATE_CLAIM_F_ALL) != 0 ||
626         claim->reserved != 0 || claim->reserved_tail[0] != 0 ||
627         claim->reserved_tail[1] != 0 || claim->reserved_tail[2] != 0 ||
628         claim->reserved_tail[3] != 0 || thread != curr_thread() ||
629         thread->process != worker || thread != &worker->threads[0] ||
630         thread->identity_generation == 0)
631         return result->status;
632     lifecycle.id = claim->lifecycle.id;
633     lifecycle.generation = claim->lifecycle.generation;
634     for (;;) {
635         struct agent_task_delegate_slot *selected = 0;
636         uint64 first_sequence = ~0ULL;
637         int wait_status;
638
639         enabled = intr_save();
640         if (!proc_teardown_live(worker) || !worker->is_agent ||
641             worker->agent_control_id == 0 ||
642             !agent_task_delegate_lifecycle_matches(worker, lifecycle) ||
643             !workflow_lifecycle_active(lifecycle) ||
644             !agent_identity_has_cap(worker, AGENT_CAP_TASK_ACCEPT)) {
645             intr_restore(enabled);
646             result->status = AGENT_TASK_CHANNEL_STALE;
647             return result->status;
648         }
649         /* A committed claim survives a failed result copyout. */
650         for (uint i = 0; i < AGENT_TASK_DELEGATE_CAPACITY; i++) {
651             struct agent_task_delegate_slot *slot =
652                 &agent_task_delegate_slots[i];
653
654             if (slot->state != AGENT_TASK_DELEGATE_SLOT_CLAIMED ||
655                 slot->worker_pid != worker->pid ||
656                 slot->worker_agent_id != worker->agent_id ||
657                 slot->worker_control_id != worker->agent_control_id ||
658                 slot->worker_thread_generation !=
659                     thread->identity_generation ||
660                 !workflow_lifecycle_key_equal(
661                     slot->lifecycle, lifecycle))
662                 continue;
663             agent_task_delegate_claim_result_fill(
664                 slot, result, AGENT_TASK_CHANNEL_OK,
665                 AGENT_TASK_DELEGATE_STATE_CLAIMED);
666             intr_restore(enabled);
667             return result->status;
668         }
669         for (uint i = 0; i < AGENT_TASK_DELEGATE_CAPACITY; i++) {
670             struct agent_task_delegate_slot *slot =
671                 &agent_task_delegate_slots[i];

```

```

672     struct proc *owner;
673     struct proc *target;
674
675     if (slot->state != AGENT_TASK_DELEGATE_SLOT_QUEUED ||
676         slot->descriptor.target_pid != worker->pid ||
677         slot->descriptor.target_agent_id !=
678             (uint)worker->agent_id ||
679         slot->descriptor.target_control_id !=
680             worker->agent_control_id ||
681         !workflow_lifecycle_key_equal(
682             slot->lifecycle, lifecycle) ||
683         slot->enqueue_sequence >= first_sequence)
684         continue;
685     owner = agent_task_delegate_find_proc_locked(
686         slot->owner_pid, slot->owner_agent_id,
687         slot->owner_control_id);
688     target = agent_task_delegate_target_locked(
689         owner, &slot->descriptor, slot->lifecycle, slot);
690     if (target != worker) {
691         /*
692          * Route and capability decisions remain revocable until
693          * the provider starts the effect. Do not expose a stale
694          * descriptor; hand terminalization back to the owner lane.
695          */
696         slot->terminal_status = owner == 0 ?
697             AGENT_STATUS_INDETERMINATE :
698             AGENT_STATUS_DENIED;
699         slot->terminal_tick = agent_task_bridge_now();
700         slot->state = AGENT_TASK_DELEGATE_SLOT_READY;
701         agent_task_delegate_wake_owner_locked(slot);
702         continue;
703     }
704     selected = slot;
705     first_sequence = slot->enqueue_sequence;
706 }
707 if (selected != 0) {
708     selected->state = AGENT_TASK_DELEGATE_SLOT_CLAIMING;
709     selected->worker_pid = worker->pid;
710     selected->worker_agent_id = worker->agent_id;
711     selected->worker_control_id = worker->agent_control_id;
712     selected->worker_thread_generation =
713         thread->identity_generation;
714     agent_task_delegate_claim_result_fill(
715         selected, result, AGENT_TASK_CHANNEL_OK,
716         AGENT_TASK_DELEGATE_STATE_CLAIMED);
717     intr_restore(enabled);
718     return result->status;
719 }
720 if ((claim->flags & AGENT_TASK_DELEGATE_CLAIM_F_WAIT) == 0) {
721     intr_restore(enabled);
722     result->status = AGENT_TASK_CHANNEL_RETRY;
723     return result->status;
724 }
725 wait_status = wait_queue_sleep_irq(
726     &agent_task_delegate_waiters[worker - pool]);
727 intr_restore(enabled);
728 /* Enqueue/cancel interrupts are notifications, not claim failure. */

```

```

729     if (wait_status == WAIT_QUEUE_INTERRUPTED)
730         continue;
731     if (wait_status != WAIT_QUEUE_OK) {
732         result->status = AGENT_TASK_CHANNEL_STALE;
733         return result->status;
734     }
735 }
736 }

```

代码 19: delegated Task controller 取消请求的完整绑定核验

c

```

903 static int
904 agent_task_delegate_cancel_request_ready(
905     struct proc *requester, struct thread *thread,
906     const struct agent_task_delegate_complete *complete,
907     struct agent_task_delegate_complete_result *result)
908 {
909     struct workflow_lifecycle_key lifecycle;
910     struct agent_task_delegate_slot *slot = 0;
911     struct proc *owner;
912     enum agent_execution_delegate_cancel_admission admission;
913     uint state;
914     uint64 now;
915     int channel_status;
916     int enabled;
917
918     agent_task_delegate_complete_result_fill(
919         0, result, AGENT_TASK_CHANNEL_BAD_REQUEST,
920         AGENT_TASK_DELEGATE_STATE_NONE);
921     if (requester == 0 || thread == 0 || complete == 0 ||
922         complete->version != AGENT_TASK_DELEGATE_VERSION ||
923         complete->size != sizeof(*complete) ||
924         complete->flags != AGENT_TASK_DELEGATE_COMPLETE_F_REQUEST_CANCEL ||
925         complete->reserved != 0 ||
926         complete->terminal_status != AGENT_STATUS_CANCELLED ||
927         complete->ack_terminal_status != 0 ||
928         complete->terminal_generation != 0 || complete->owner_pid <= 0 ||
929         complete->owner_control_id == 0 ||
930         complete->channel_generation == 0 || complete->request_id == 0 ||
931         complete->slot_generation == 0 || complete->task_id == 0 ||
932         complete->correlation_id == 0 || thread != curr_thread() ||
933         thread->process != requester || thread->identity_generation == 0)
934         return result->status;
935     lifecycle.id = complete->lifecycle.id;
936     lifecycle.generation = complete->lifecycle.generation;
937     enabled = intr_save();
938     if (!proc_teardown_live(requester) || !requester->is_agent ||
939         requester->agent_control_id == 0 ||
940         !agent_task_delegate_lifecycle_matches(requester, lifecycle) ||
941         !workflow_lifecycle_active(lifecycle) ||
942         !agent_identity_has_cap(requester, AGENT_CAP_ORCHESTRATE) ||
943         !agent_identity_has_cap(requester, AGENT_CAP_WAIT_CANCEL)) {
944         result->status = AGENT_TASK_CHANNEL_DENIED;
945         goto out;
946     }
947     /* The exact binding is also the idempotency key for a lost result copyout. */

```



```

948     for (uint i = 0; i < AGENT_TASK_DELEGATE_CAPACITY; i++) {
949         const struct agent_task_delegate_cancel_receipt *receipt =
950             &agent_task_delegate_cancel_receipts[i];
951
952         if (!agent_task_delegate_cancel_receipt_binding_matches(
953             receipt, requester, complete, lifecycle))
954             continue;
955         result->status = AGENT_TASK_CHANNEL_OK;
956         result->state = receipt->state;
957         result->channel_generation = receipt->channel_generation;
958         result->request_id = receipt->request_id;
959         result->slot_generation = receipt->slot_generation;
960         result->task_id = receipt->task_id;
961         result->correlation_id = receipt->correlation_id;
962         result->terminal_status = receipt->terminal_status;
963         result->terminal_generation = receipt->terminal_generation;
964         goto out;
965     }
966     for (uint i = 0; i < AGENT_TASK_DELEGATE_CAPACITY; i++) {
967         struct agent_task_delegate_slot *candidate =
968             &agent_task_delegate_slots[i];
969
970         if (candidate->state != AGENT_TASK_DELEGATE_SLOT_FREE &&
971             candidate->owner_pid == complete->owner_pid &&
972             candidate->owner_control_id == complete->owner_control_id &&
973             candidate->sqe.ring_generation ==
974                 complete->channel_generation &&
975             candidate->sqe.request_id == complete->request_id &&
976             candidate->sqe.slot_generation ==
977                 complete->slot_generation &&
978             candidate->descriptor.task_id == complete->task_id &&
979             candidate->descriptor.correlation_id ==
980                 complete->correlation_id &&
981             workflow_lifecycle_key_equal(
982                 candidate->lifecycle, lifecycle)) {
983             slot = candidate;
984             break;
985         }
986     }
987     if (slot == 0) {
988         result->status = AGENT_TASK_CHANNEL_STALE;
989         goto out;
990     }
991     owner = agent_task_delegate_find_proc_locked(
992         slot->owner_pid, slot->owner_agent_id, slot->owner_control_id);
993     if (owner == 0 ||
994         !agent_task_delegate_lifecycle_matches(owner, lifecycle) ||
995         !agent_identity_has_cap(owner, AGENT_CAP_ORCHESTRATE) ||
996         !agent_ipc_task_route_allows_locked(requester, owner)) {
997         result->status = AGENT_TASK_CHANNEL_DENIED;
998         goto fill;
999     }
1000     if (slot->state == AGENT_TASK_DELEGATE_SLOT_PREPARING ||
1001         slot->state == AGENT_TASK_DELEGATE_SLOT_CLAIMING) {
1002         result->status = AGENT_TASK_CHANNEL_RETRY;
1003         goto fill;
1004     }

```

```

1005     if (slot->state != AGENT_TASK_DELEGATE_SLOT_QUEUED &&
1006         slot->state != AGENT_TASK_DELEGATE_SLOT_CLAIMED) {
1007         result->status = AGENT_TASK_CHANNEL_STALE;
1008         goto fill;
1009     }
1010     /*
1011      * Validate the owner lane before mutating the Contract. Interrupts stay
1012      * disabled through both commits, so a prior denied SQ CANCEL cannot turn
1013      * this exact binding into a user-triggerable invariant failure.
1014      */
1015     channel_status = agent_task_channel_delegate_cancel_preflight_locked(
1016         owner, complete->channel_generation, complete->request_id,
1017         complete->slot_generation);
1018     if (channel_status != AGENT_TASK_CHANNEL_OK) {
1019         result->status = channel_status;
1020         goto fill;
1021     }
1022     now = agent_task_bridge_now();
1023     admission = agent_execution_contract_delegate_cancel_preflight_locked(
1024         &slot->claim, now);
1025     if (admission == AGENT_EXECUTION_DELEGATE_CANCEL_TIMEOUT) {
1026         agent_task_delegate_terminal_intent_locked(
1027             slot, AGENT_STATUS_TIMEOUT);
1028         /* No provider effect exists in QUEUED, so timeout is immediately final. */
1029         if (slot->state == AGENT_TASK_DELEGATE_SLOT_QUEUED) {
1030             slot->terminal_pending = 0;
1031             slot->state = AGENT_TASK_DELEGATE_SLOT_READY;
1032         }
1033         agent_task_delegate_wake_target_locked(
1034             slot->descriptor.target_pid,
1035             (int)slot->descriptor.target_agent_id,
1036             slot->descriptor.target_control_id);
1037         agent_task_delegate_wake_owner_locked(slot);
1038         result->status = AGENT_TASK_CHANNEL_STALE;
1039         goto fill;
1040     }
1041     if (admission == AGENT_EXECUTION_DELEGATE_CANCEL_DENIED) {
1042         result->status = AGENT_TASK_CHANNEL_DENIED;
1043         goto fill;
1044     }
1045     if (admission != AGENT_EXECUTION_DELEGATE_CANCEL_ALLOWED) {
1046         result->status = AGENT_TASK_CHANNEL_STALE;
1047         goto fill;
1048     }
1049     channel_status = agent_task_channel_delegate_cancel_locked(
1050         owner, complete->channel_generation, complete->request_id,
1051         complete->slot_generation);
1052     if (channel_status != AGENT_TASK_CHANNEL_OK) {
1053         result->status = channel_status;
1054         goto fill;
1055     }
1056     agent_execution_contract_delegate_cancel_commit_locked(&slot->claim);
1057     agent_task_delegate_terminal_intent_locked(slot, AGENT_STATUS_CANCELLED);
1058     state = slot->state == AGENT_TASK_DELEGATE_SLOT_CLAIMED ?
1059         AGENT_TASK_DELEGATE_STATE_CLAIMED :
1060         AGENT_TASK_DELEGATE_STATE_READY;
1061     if (slot->state == AGENT_TASK_DELEGATE_SLOT_QUEUED) {

```

```

1062     slot->terminal_pending = 0;
1063     slot->state = AGENT_TASK_DELEGATE_SLOT_READY;
1064 }
1065 agent_task_delegate_cancel_receipt_record_locked(slot, requester, state);
1066 agent_task_delegate_wake_target_locked(
1067     slot->descriptor.target_pid,
1068     (int)slot->descriptor.target_agent_id,
1069     slot->descriptor.target_control_id);
1070 agent_task_delegate_wake_owner_locked(slot);
1071 result->status = AGENT_TASK_CHANNEL_OK;
1072
1073 fill:
1074     state = agent_task_delegate_public_state(slot);
1075     agent_task_delegate_complete_result_fill(
1076         slot, result, result->status, state);
1077 out:
1078     intr_restore(enabled);
1079     return result->status;
1080 }

```

代码 20: delegated Task provider 对终态 offer 的清理确认

c

```

1214     agent_task_delegate_complete_result_fill(
1215         slot, result, status, AGENT_TASK_DELEGATE_STATE_READY);
1216     intr_restore(enabled);
1217     return result->status;
1218 }
1219 if (slot->completion_ack_pending) {
1220     now = agent_task_bridge_now();
1221     if ((slot->sqe.flags & AGENT_TASK_SQE_F_HARD_DEADLINE) != 0 &&
1222         now >= slot->sqe.deadline_tick)
1223         agent_task_delegate_terminal_intent_locked(
1224             slot, AGENT_STATUS_TIMEOUT);
1225     if (!slot->worker_completion_recorded ||
1226         slot->submitted_status != complete->terminal_status) {
1227         agent_task_delegate_complete_result_fill(
1228             slot, result, AGENT_TASK_CHANNEL_STALE,
1229             AGENT_TASK_DELEGATE_STATE_CLAIMED);
1230         intr_restore(enabled);
1231         return result->status;
1232     }
1233     if ((complete->flags &
1234         AGENT_TASK_DELEGATE_COMPLETE_F_ACK_TERMINAL) == 0) {
1235         agent_task_delegate_complete_result_fill(
1236             slot, result, AGENT_TASK_CHANNEL_RETRY,
1237             AGENT_TASK_DELEGATE_STATE_CLAIMED);
1238         intr_restore(enabled);
1239         return result->status;
1240     }
1241     if (complete->ack_terminal_status != slot->terminal_status ||
1242         complete->terminal_generation != slot->terminal_generation) {
1243         agent_task_delegate_complete_result_fill(
1244             slot, result, AGENT_TASK_CHANNEL_RETRY,
1245             AGENT_TASK_DELEGATE_STATE_CLAIMED);
1246         intr_restore(enabled);
1247         return result->status;

```

```

1248     }
1249     slot->submitted_flags = complete->flags;
1250     slot->completion_ack_pending = 0;
1251     slot->terminal_pending = 0;
1252     slot->provenance.output_labels |=
1253         agent_provenance_current_labels(worker) |
1254         AGENT_PROVENANCE_CROSS_AGENT_DATA;
1255     slot->claim.executor_context_sequence =
1256         worker->context_path_latest;
1257     /* Cleanup ACK is the first boundary after which effects are quiescent. */
1258     slot->terminal_status = agent_task_delegate_kernel_terminal_locked(
1259         slot, slot->terminal_status, now);
1260     slot->terminal_tick = now;

```

3.8.6. 资源使用租约与 workflow EEVDF

资源控制器通过资源使用租约锁定并激活一组资源账户额度。租约记录创建者、生命周期、代次和账户对；激活时，内核将租约令牌写入线程冷状态，停用、结算和放弃时都必须匹配同一代次。这样，一段工具执行只能使用受限且可结算的资源额度。

代码 21：资源使用租约的创建与账户对提交

c

```

1770 int resource_phase_lease_begin(
1771     struct resource_phase_lease *lease,
1772     struct workflow_lifecycle_key lifecycle,
1773     struct resource_account_handle exec_handle,
1774     struct resource_account_handle storage_handle,
1775     enum resource_charge_class charge_class,
1776     const uint64 exec_amounts[RESOURCE_KIND_COUNT],
1777     const uint64 storage_amounts[RESOURCE_KIND_COUNT], uint node_id,
1778     uint64 request_id)
1779 {
1780     struct resource_account *accounts[RESOURCE_PHASE_ACCOUNT_COUNT];
1781     struct resource_phase_account_lock
1782         *locks[RESOURCE_PHASE_ACCOUNT_COUNT];
1783     struct resource_account_handle handles[RESOURCE_PHASE_ACCOUNT_COUNT] = {
1784         [RESOURCE_PHASE_EXEC] = exec_handle,
1785         [RESOURCE_PHASE_STORAGE] = storage_handle,
1786     };
1787     uint64 amounts[RESOURCE_PHASE_ACCOUNT_COUNT][RESOURCE_KIND_COUNT];
1788     uint masks[RESOURCE_PHASE_ACCOUNT_COUNT], union_mask;
1789     struct resource_phase_registry_entry *entry;
1790     struct thread *owner;
1791     int registry_slot;
1792     int enabled;
1793
1794     if (lease == 0 || lease->state != RESOURCE_PHASE_LEASE_EMPTY ||
1795         exec_amounts == 0 || storage_amounts == 0 || request_id == 0 ||
1796         !workflow_lifecycle_key_valid(lifecycle) ||
1797         !resource_charge_class_valid(charge_class) ||
1798         resource_account_handle_equal(exec_handle, storage_handle))
1799         return -1;
1800     memmove(amounts[RESOURCE_PHASE_EXEC], exec_amounts,
1801             sizeof(amounts[RESOURCE_PHASE_EXEC]));

```

```

1802     memmove(amounts[RESOURCE_PHASE_STORAGE], storage_amounts,
1803             sizeof(amounts[RESOURCE_PHASE_STORAGE]));
1804     for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1805         for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++)
1806             if (amounts[role][kind] > (uint)-1)
1807                 return -1;
1808         masks[role] = resource_phase_amount_mask(amounts[role]);
1809     }
1810     union_mask = masks[RESOURCE_PHASE_EXEC] |
1811                masks[RESOURCE_PHASE_STORAGE];
1812     if (union_mask == 0)
1813         return -1;
1814
1815     enabled = intr_save();
1816     owner = curr_thread();
1817     registry_slot = resource_phase_registry_free_slot();
1818     accounts[RESOURCE_PHASE_EXEC] = resource_account_lookup(exec_handle);
1819     accounts[RESOURCE_PHASE_STORAGE] =
1820         resource_account_lookup(storage_handle);
1821     if (accounts[RESOURCE_PHASE_EXEC] == 0 ||
1822         accounts[RESOURCE_PHASE_STORAGE] == 0 ||
1823         accounts[RESOURCE_PHASE_EXEC]->kind != RESOURCE_ACCOUNT_EXEC ||
1824         accounts[RESOURCE_PHASE_STORAGE]->kind !=
1825             RESOURCE_ACCOUNT_STORAGE ||
1826         registry_slot < 0 || owner == 0 || owner->trapframe == 0 ||
1827         owner->identity_generation == 0 || owner->process == 0 ||
1828         !proc_teartdown_live(owner->process) ||
1829         resource_phase_registry_owner_busy(owner) ||
1830         charge_class !=
1831             (owner->process->resource_slot_reserved ?
1832              RESOURCE_CHARGE_RESERVED :
1833              RESOURCE_CHARGE_ORDINARY) ||
1834         resource_phase_generation == RESOURCE_LIMIT_UNBOUNDED ||
1835         !workflow_lifecycle_active(lifecycle) ||
1836         !resource_phase_owner_binding_matches(
1837             lifecycle, exec_handle, owner) ||
1838         !resource_phase_lifecycle_pair_admissible(
1839             lifecycle, handles) ||
1840         !resource_phase_lock_records_locked(accounts, handles, locks) ||
1841         !resource_phase_pair_admissible_locked(
1842             accounts, charge_class, amounts, masks, union_mask)) {
1843         intr_restore(enabled);
1844         return -1;
1845     }
1846     resource_phase_pair_reclaim_locked(
1847         accounts, charge_class, amounts, union_mask);
1848     resource_phase_pair_commit_locked(
1849         accounts, handles, locks, charge_class, amounts, masks);
1850     resource_phase_generation++;
1851     entry = &resource_phase_registry[registry_slot];
1852     memset(entry, 0, sizeof(*entry));
1853     entry->owner = owner;
1854     entry->lease.lifecycle = lifecycle;
1855     entry->lease.generation = resource_phase_generation;
1856     entry->lease.request_id = request_id;
1857     entry->lease.owner_thread_generation = owner->identity_generation;
1858     entry->lease.node_id = node_id;

```

```

1859     entry->lease.charge_class = charge_class;
1860     entry->lease.state = RESOURCE_PHASE_LEASE_ADMITTED;
1861     for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1862         entry->lease.account[role].handle = handles[role];
1863         entry->lease.account[role].kind_mask = masks[role];
1864         for (uint kind = 0; kind < RESOURCE_KIND_COUNT; kind++)
1865             entry->lease.account[role].remaining[kind] =
1866                 (uint) amounts[role][kind];
1867     }
1868     entry->used = 1;
1869     lease->generation = resource_phase_generation;
1870     lease->request_id = request_id;
1871     lease->node_id = node_id;
1872     lease->registry_slot = (uint) registry_slot;
1873     lease->state = RESOURCE_PHASE_LEASE_ADMITTED;
1874     intr_restore(enabled);
1875     return 0;
1876 }

```

代码 22：资源使用租约的线程激活与去激活

c

```

1878 int resource_phase_lease_activate(struct resource_phase_lease *lease,
1879     uint64 expected_generation)
1880 {
1881     struct thread *thread;
1882     struct resource_account *account;
1883     struct resource_phase_registry_entry *entry;
1884     struct resource_phase_lease_record *record;
1885     int enabled;
1886
1887     if (lease == 0 || expected_generation == 0)
1888         return -1;
1889     enabled = intr_save();
1890     thread = curr_thread();
1891     entry = resource_phase_registry_lookup(
1892         lease, expected_generation);
1893     record = entry == 0 ? 0 : &entry->lease;
1894     if (entry == 0 || record->state != RESOURCE_PHASE_LEASE_ADMITTED ||
1895         thread == 0 || entry->owner != thread ||
1896         record->owner_thread_generation != thread->identity_generation ||
1897         thread->trapframe == 0 ||
1898         thread->identity_generation == 0 ||
1899         thread_trap_cold(thread)->resource_phase_lease_generation != 0 ||
1900         thread_trap_cold(thread)->resource_phase_claim_depth != 0 ||
1901         !proc_teardown_live(thread->process) ||
1902         !workflow_lifecycle_active(record->lifecycle) ||
1903         !resource_phase_owner_matches(record, thread))
1904         goto fail;
1905     for (uint role = 0; role < RESOURCE_PHASE_ACCOUNT_COUNT; role++) {
1906         account = resource_account_lookup(record->account[role].handle);
1907         if (account == 0 || account->state != RESOURCE_ACCOUNT_ACTIVE ||
1908             resource_phase_account_lock_lookup(
1909                 record->account[role].handle) == 0)
1910             goto fail;
1911     }

```



```

1912     record->owner_thread_generation = thread->identity_generation;
1913     record->state = RESOURCE_PHASE_LEASE_ACTIVE;
1914     lease->state = RESOURCE_PHASE_LEASE_ACTIVE;
1915     thread_trap_cold(thread)->resource_phase_lease_slot =
1916         lease->registry_slot;
1917     thread_trap_cold(thread)->resource_phase_lease_generation =
1918         expected_generation;
1919     intr_restore(enabled);
1920     return 0;
1921 fail:
1922     intr_restore(enabled);
1923     return -1;
1924 }
1925
1926 int resource_phase_lease_deactivate(struct resource_phase_lease *lease,
1927                                     uint64 expected_generation)
1928 {
1929     struct thread *thread;
1930     struct resource_phase_registry_entry *entry;
1931     struct resource_phase_lease_record *record;
1932     int enabled;
1933
1934     if (lease == 0 || expected_generation == 0)
1935         return -1;
1936     enabled = intr_save();
1937     thread = curr_thread();
1938     entry = resource_phase_registry_lookup(
1939         lease, expected_generation);
1940     record = entry == 0 ? 0 : &entry->lease;
1941     if (entry == 0 || record->state != RESOURCE_PHASE_LEASE_ACTIVE ||
1942         thread == 0 || entry->owner != thread ||
1943         thread->trapframe == 0 ||
1944         thread_trap_cold(thread)->resource_phase_lease_slot !=
1945             lease->registry_slot ||
1946         thread_trap_cold(thread)->resource_phase_lease_generation !=
1947             expected_generation ||
1948         record->owner_thread_generation != thread->identity_generation ||
1949         record->outstanding_claims != 0 ||
1950         thread_trap_cold(thread)->resource_phase_claim_depth != 0) {
1951         intr_restore(enabled);
1952         return -1;
1953     }
1954     thread_trap_cold(thread)->resource_phase_lease_slot =
1955         RESOURCE_PHASE_REGISTRY_SLOT_NONE;
1956     thread_trap_cold(thread)->resource_phase_lease_generation = 0;
1957     record->state = RESOURCE_PHASE_LEASE_DEACTIVATED;
1958     lease->state = RESOURCE_PHASE_LEASE_DEACTIVATED;
1959     intr_restore(enabled);
1960     return 0;
1961 }

```

调度器不会因为同一工作流拥有更多线程，就把它当作更多独立的公平实体。workflow_scheduler_select() 先收集可运行工作流，再依据滞后量、截止时间、权重和可运行负载选择实体；调度和记账分别记录服务开始时刻与已消耗时间。因此，调度公平性按工作流记账，而不是按偶然增加的线程数记账。

代码 23: 工作流 EEVDF 的候选收集与选择

c

```

408 int workflow_scheduler_select(
409     const struct workflow_scheduler_candidate *candidates, uint count,
410     uint64 now_cycle, uint64 now_tick, int *selected_domain)
411 {
412     int slots[WORKFLOW_SCHEDULER_ENTITY_CAP];
413     uint planned_latency[WORKFLOW_SCHEDULER_ENTITY_CAP];
414     uint planned_request_ticks[WORKFLOW_SCHEDULER_ENTITY_CAP];
415     uint planned_decays[WORKFLOW_SCHEDULER_ENTITY_CAP];
416     uint64 planned_vruntime[WORKFLOW_SCHEDULER_ENTITY_CAP];
417     uint64 planned_remaining[WORKFLOW_SCHEDULER_ENTITY_CAP];
418     uint64 planned_deadline[WORKFLOW_SCHEDULER_ENTITY_CAP];
419     uint reserved = 0;
420     uint new_slots = 0;
421     uint wake_transitions = 0;
422     uint64 rebase_offset;
423     uint64 planned_vtime;
424     int selected = -1;
425     int enabled;
426
427     (void)now_cycle;
428     if (selected_domain == 0 || candidates == 0 || count == 0 ||
429         count > WORKFLOW_SCHEDULER_ENTITY_CAP)
430         return -1;
431     *selected_domain = -1;
432     for (uint i = 0; i < WORKFLOW_SCHEDULER_ENTITY_CAP; i++)
433         slots[i] = WORKFLOW_SCHEDULER_SLOT_NONE;
434     enabled = intr_save();
435     /* Phase one is read-only: validate every identity and reserve local slots. */
436     for (uint i = 0; i < count; i++) {
437         int is_new = 0;
438
439         if (!workflow_scheduler_identity_valid(
440             candidates[i].lifecycle, candidates[i].account,
441             candidates[i].domain_id) ||
442             candidates[i].runnable_threads == 0 ||
443             candidates[i].runnable_threads > 0xffffU ||
444             candidates[i].latency_class >
445                 AGENT_WORKFLOW_LATENCY_BATCH)
446             goto fail;
447         for (uint j = 0; j < i; j++)
448             if (candidates[j].domain_id ==
449                 candidates[i].domain_id ||
450                 workflow_lifecycle_key_equal(
451                     candidates[j].lifecycle,
452                     candidates[i].lifecycle) ||
453                 resource_account_handle_equal(
454                     candidates[j].account,
455                     candidates[i].account))
456                 goto fail;
457         planned_request_ticks[i] = workflow_scheduler_request_ticks(
458             candidates[i].latency_class,
459             candidates[i].deadline_delta_ticks);
460         if (planned_request_ticks[i] == 0)
461             goto fail;

```

```

462     slots[i] = workflow_scheduler_plan_slot_locked(
463         &candidates[i], reserved, &is_new);
464     if (slots[i] < 0)
465         goto fail;
466     reserved |= 1U << slots[i];
467     if (is_new)
468         new_slots |= 1U << i;
469     else if (workflow_scheduler_entities[slots[i]].flags &
470         WORKFLOW_SCHEDULER_F_CACHE_INVALID)
471         goto fail;
472 }
473
474 /* Compute the entire post-selection state locally before any mutation. */
475 rebase_offset = workflow_scheduler_rebase_offset_locked();
476 planned_vtime = workflow_scheduler_vtime >= rebase_offset ?
477     workflow_scheduler_vtime - rebase_offset : 0;
478 for (uint i = 0; i < count; i++) {
479     struct workflow_scheduler_entity *entity =
480         &workflow_scheduler_entities[slots[i]];
481     uint64 request_cycles = workflow_scheduler_request_cycles(
482         planned_request_ticks[i]);
483
484     planned_decays[i] = 0;
485     if (new_slots & (1U << i)) {
486         planned_vruntime[i] = planned_vtime;
487         planned_remaining[i] = 0;
488         planned_latency[i] = AGENT_WORKFLOW_LATENCY_NORMAL;
489         wake_transitions |= 1U << i;
490     } else {
491         planned_vruntime[i] = entity->vruntime - rebase_offset;
492         planned_remaining[i] = entity->remaining_cycles;
493         planned_latency[i] = entity->latency_class;
494         planned_request_ticks[i] = entity->request_ticks;
495         if (!(entity->flags &
496             AGENT_WORKFLOW_SCHED_F_RUNNABLE)) {
497             wake_transitions |= 1U << i;
498             planned_remaining[i] = 0;
499             if (entity->flags &
500                 AGENT_WORKFLOW_SCHED_F_SLEEPING) {
501                 uint64 slept =
502                     now_tick >= entity->sleep_start_tick ?
503                     now_tick -
504                     entity->sleep_start_tick :
505                     0;
506
507                 planned_decays[i] = slept /
508                     WORKFLOW_SCHEDULER_SLEEP_DECAY_TICKS;
509                 if (planned_decays[i] >
510                     WORKFLOW_SCHEDULER_MAX_SLEEP_DECAYS)
511                     planned_decays[i] =
512                         WORKFLOW_SCHEDULER_MAX_SLEEP_DECAYS;
513                 planned_vruntime[i] =
514                     workflow_scheduler_decay_toward(
515                         planned_vruntime[i],
516                         planned_vtime,
517                         planned_decays[i]);
518             }

```

```

519     }
520 }
521 /* Urgency may shorten an open request, never mint more service. */
522 if (planned_remaining[i] == 0 ||
523     request_cycles < planned_remaining[i]) {
524     planned_remaining[i] = request_cycles;
525     planned_request_ticks[i] = workflow_scheduler_request_ticks(
526         candidates[i].latency_class,
527         candidates[i].deadline_delta_ticks);
528     planned_latency[i] = candidates[i].latency_class;
529 }
530 if (planned_vruntime[i] > ~0ULL - planned_remaining[i])
531     goto fail;
532 planned_deadline[i] =
533     planned_vruntime[i] + planned_remaining[i];
534 }
535 if (count == 1) {
536     if (planned_vruntime[0] > planned_vtime)
537         planned_vtime = planned_vruntime[0];
538 } else {
539     uint64 average = workflow_scheduler_average(
540         planned_vruntime, count);
541
542     if (average > planned_vtime)
543         planned_vtime = average;
544 }
545 for (uint i = 0; i < count; i++) {
546     if (planned_vruntime[i] <= planned_vtime) {
547         if (selected < 0 ||
548             workflow_scheduler_before(candidates,
549                                     planned_deadline,
550                                     planned_vruntime, i,
551                                     (uint)selected))
552             selected = i;
553     }
554 }
555 if (selected < 0)
556     goto fail;

```

代码 24: 工作流 EEVDF 的提交、调度与记账

c

```

558 /* Phase two commits a plan that can no longer fail. */
559 if (rebase_offset != 0)
560     workflow_scheduler_rebase_locked();
561 for (uint i = 0; i < count; i++) {
562     struct workflow_scheduler_entity *entity =
563         &workflow_scheduler_entities[slots[i]];
564
565     if (new_slots & (1U << i)) {
566         memset(entity, 0, sizeof(*entity));
567         entity->lifecycle = candidates[i].lifecycle;
568         entity->account = candidates[i].account;
569         entity->domain_id = candidates[i].domain_id;
570         entity->flags = AGENT_WORKFLOW_SCHED_F_ACTIVE;
571         entity->earliest_deadline_tick =
572             WORKFLOW_SCHEDULER_NO_DEADLINE;

```

```

573     entity->cached_latency_class =
574         (uchar)candidates[i].latency_class;
575     entity->schedulable_threads =
576         (ushort)candidates[i].runnable_threads;
577 }
578 workflow_scheduler_domain_slot[candidates[i].domain_id] =
579     (signed char)slots[i];
580 entity->vruntime = planned_vruntime[i];
581 entity->remaining_cycles = planned_remaining[i];
582 entity->virtual_deadline = planned_deadline[i];
583 entity->request_ticks = (uchar)planned_request_ticks[i];
584 entity->latency_class = (uchar)planned_latency[i];
585 entity->runnable_threads =
586     (ushort)candidates[i].runnable_threads;
587 entity->mode = AGENT_WORKFLOW_SCHED_MODE_EEVDF;
588 if (wake_transitions & (1U << i)) {
589     entity->sleep_decays = workflow_scheduler_counter32_add(
590         entity->sleep_decays, planned_decays[i]);
591     entity->wake_tick = now_tick;
592     entity->flags |= WORKFLOW_SCHEDULER_F_WAKE_PENDING;
593 }
594 entity->flags &= ~(AGENT_WORKFLOW_SCHED_F_SLEEPING |
595     AGENT_WORKFLOW_SCHED_F_ELIGIBLE |
596     AGENT_WORKFLOW_SCHED_F_FALLBACK);
597 entity->flags |= AGENT_WORKFLOW_SCHED_F_RUNNABLE;
598 if (planned_vruntime[i] <= planned_vtime)
599     entity->flags |= AGENT_WORKFLOW_SCHED_F_ELIGIBLE;
600 else
601     entity->eligibility_misses =
602         workflow_scheduler_counter32_add(
603             entity->eligibility_misses, 1);
604 }
605 workflow_scheduler_vtime = planned_vtime;
606 if (candidates[selected].deadline_delta_ticks == 0) {
607     struct workflow_scheduler_entity *entity =
608         &workflow_scheduler_entities[slots[selected]];
609
610     if (!(entity->flags & WORKFLOW_SCHEDULER_F_DEADLINE_MISSED)) {
611         entity->flags |= WORKFLOW_SCHEDULER_F_DEADLINE_MISSED;
612         entity->deadline_misses = workflow_scheduler_counter32_add(
613             entity->deadline_misses, 1);
614     }
615 }
616 *selected_domain = candidates[selected].domain_id;
617 intr_restore(enabled);
618 return 0;

```

算法 4：资源与 CPU 的工作流级执行过程

1. 执行约定把节点的执行额度、存储额度和记账类别写入当前声明。
2. 资源使用租约锁定、激活并结算本阶段的账户额度；超限或状态失效时返回受控结果。
3. 可运行线程按工作流汇集，EEVDF 根据实体的滞后量和截止时间选择下一次服务。
4. 执行约定路径在上下文和执行记录中写入终止状态（terminal state），使资源拒绝、取

消和超时能够回溯到具体节点和尝试号。

3.8.7. Guest runtime（客户机运行时）与结果文件发布

Nexus 用户态库会重新发现内核工具目录，逐项核对 ABI、编号、名称和 schema 是否一致；发起调用时还会检查当前智能体身份、产品角色和必要能力位。用户态校验不替代内核，而是让模型适配层尽早排除不可能成功的工具调用。

代码 25: Nexus 对内核工具目录的发现与一致性检查

c

```

681
682 unsigned long long agent_nexus_product_capabilities(
683     unsigned int product_role)
684 {
685     switch (product_role) {
686     case AGENT_NEXUS_ROLE_SYSTEM:
687         return AGENT_CAP_META_READ | AGENT_CAP_PROCESS_READ |
688             AGENT_CAP_MESSAGE_SEND | AGENT_CAP_WATCH |
689             AGENT_CAP_ARTIFACT_WRITE | AGENT_CAP_AUDIT_WRITE |
690             AGENT_CAP_TASK_ACCEPT;
691     case AGENT_NEXUS_ROLE_RESEARCH:
692         return AGENT_CAP_META_READ | AGENT_CAP_CONTENT_READ |
693             AGENT_CAP_MESSAGE_SEND | AGENT_CAP_WATCH |
694             AGENT_CAP_ARTIFACT_WRITE | AGENT_CAP_AUDIT_WRITE |
695             AGENT_CAP_TASK_ACCEPT;
696     case AGENT_NEXUS_ROLE_COORDINATOR:
697     case AGENT_NEXUS_ROLE_RELAY:
698         return AGENT_CAP_META_READ | AGENT_CAP_CONTENT_READ |
699             AGENT_CAP_PROCESS_READ | AGENT_CAP_MESSAGE_SEND |
700             AGENT_CAP_WATCH | AGENT_CAP_ACTION_WRITE |
701             AGENT_CAP_ARTIFACT_WRITE | AGENT_CAP_AUDIT_WRITE |
702             AGENT_CAP_META_WRITE | AGENT_CAP_ORCHESTRATE |
703             AGENT_CAP_LLM_RELAY | AGENT_CAP_WAIT_CANCEL |
704             AGENT_CAP_ROUTE_MANAGE | AGENT_CAP_TASK_ACCEPT;
705     default:
706         return 0;
707     }
708 }
709
710 unsigned long long agent_nexus_product_permission(
711     unsigned int product_role)
712 {
713     switch (product_role) {
714     case AGENT_NEXUS_ROLE_COORDINATOR:
715     case AGENT_NEXUS_ROLE_SYSTEM:
716     case AGENT_NEXUS_ROLE_RESEARCH:
717     case AGENT_NEXUS_ROLE_RELAY:
718         return 1ULL << (product_role - 1U);
719     default:
720         return 0;
721     }
722 }

```

代码 26: Nexus 带类型信息工具调用的用户态封装

c

```

894 {
895     nexus_identity_discard_inherited();
896     if (!nexus_identity_registered || control_id == 0 ||
897         (nexus_registered_identity.control_id != 0 &&
898          nexus_registered_identity.control_id != control_id))
899         return -1;
900     nexus_registered_identity.control_id = control_id;
901     return 0;
902 }
903
904 int agent_nexus_identity_current(struct agent_nexus_artifact_actor *actor)
905 {
906     struct agent_info info;
907
908     if (actor == 0)
909         return -1;
910     nexus_identity_discard_inherited();
911     memset(&info, 0, sizeof(info));
912     if (!nexus_identity_registered || agent_info(&info) != 0 ||
913         !info.is_agent || info.agent_id <= 0 ||
914         nexus_registered_identity.pid != (unsigned int)getpid() ||
915         nexus_registered_identity.agent_id != (unsigned int)info.agent_id ||
916         nexus_registered_identity.kernel_role !=
917             (unsigned int)info.agent_role)
918         return -1;
919     *actor = nexus_registered_identity;
920     return 0;
921 }
922
923 int agent_nexus_artifact_actor_current(
924     unsigned int product_role, unsigned long long control_id,
925     struct agent_nexus_artifact_actor *actor)
926 {
927     if (agent_nexus_identity_current(actor) < 0 ||
928         actor->product_role != product_role ||
929         (control_id != 0 && actor->control_id != control_id))
930         return -1;
931     return 0;
932 }
933
934 static int nexus_artifact_kind_valid(unsigned int kind)
935 {
936     return kind >= AGENT_NEXUS_ARTIFACT_TOOL_INPUT &&
937         kind <= AGENT_NEXUS_ARTIFACT_APPROVAL &&
938         kind != AGENT_NEXUS_ARTIFACT_RESERVED_7;
939 }
940
941 static unsigned long long nexus_artifact_required_provenance(
942     const struct agent_nexus_artifact_manifest *manifest)
943 {
944     unsigned long long required;
945
946     switch (manifest->source) {
947     case AGENT_NEXUS_SOURCE_SEED:

```



```

948     required = AGENT_PROVENANCE_UNTRUSTED_FILE_DATA;
949     break;
950     case AGENT_NEXUS_SOURCE_KERNEL_TOOL:
951         required = AGENT_PROVENANCE_KERNEL_FACT |
952                 AGENT_PROVENANCE_UNTRUSTED_TOOL_OUTPUT;
953     break;
954     case AGENT_NEXUS_SOURCE_WORKER_METRIC:
955         required = AGENT_PROVENANCE_AGENT_DERIVED |
956                 AGENT_PROVENANCE_UNTRUSTED_TOOL_OUTPUT |
957                 AGENT_PROVENANCE_CROSS_AGENT_DATA;
958     break;
959     case AGENT_NEXUS_SOURCE_MODEL:
960         required = AGENT_PROVENANCE_AGENT_DERIVED |
961                 AGENT_PROVENANCE_UNTRUSTED_TOOL_OUTPUT;

```

`agent_file_publish()` 先把 **header** 与 **payload** 一并拷入按 workflow 账户计费的内核页，再把这份完整 **snapshot** 交给 **VFS**。路径必须是 workflow 访问范围中的单个正式文件名，已有文件不会被覆盖。用户态缓冲区在 **syscall** 返回前即与后续 **I/O** 解耦。

VFS 为结果分配未命名 **inode**，完成标签、空间预留、数据写入和第一次 **checkpoint**；只有数据块和最终 **inode** 形态已经持久，才用一次 **dirlink** 接入正式目录并提交第二次 **checkpoint**。已知的接入失败会回收未命名 **inode**；**I/O** 状态不确定时则保留可能已经接入目录的 **inode**，并返回 **INDETERMINATE**。

Nexus 收到 **DUPLICATE** 或 **INDETERMINATE** 后不会直接重试写入。它打开正式路径，逐字节比较 **header**、**payload** 和 **EOF**；只有完整内容与本次请求一致才把发布收敛为成功。这样，其他 **Agent** 看见的是一份完整结果文件，而不是带无效 **header** 或缺少尾部 **payload** 的中间状态。

代码 27：未命名 **inode** 的完整写入与单次目录接入

c

```

7701 int fs_agent_file_publish_atomic(char *path, const struct vfs_cred *cred,
7702                                const void *snapshot, uint total)
7703 {
7704     struct fs_storage_charge charge;
7705     struct inode *dp = 0;
7706     struct inode *existing = 0;
7707     struct inode *ip = 0;
7708     uint intent_owner;
7709     char key[DIRSIZ + 1];
7710     int lookup_status = FS_LOOKUP_ERROR;
7711     int root_status = FS_LOOKUP_ERROR;
7712     int result;
7713     int status;
7714
7715     if (!fs_agent_publish_name_valid(path) || cred == 0 || snapshot == 0 ||
7716         total == 0 || total > AGENT_FILE_PUBLISH_MAX_BYTES)
7717         return AGENT_STATUS_BAD_PARAM;
7718     if (fs_dirent_canonicalize(path, key) < 0)
7719         return AGENT_STATUS_BAD_PARAM;
7720     dp = root_dir_status(&root_status);
7721     if (dp == 0 || root_status != FS_LOOKUP_FOUND) {
7722         if (dp != 0)
7723             iput(dp);

```

```

7724     return fs_agent_publish_io_status(root_status);
7725 }
7726 if (!vfs_inode_authorize(dp, cred, VFS_OP_CREATE) ||
7727     !vfs_create_request_authorize(cred, VFS_POLICY_WORKFLOW, 0, 1, 0) ||
7728     cred->scope_id < VFS_SCOPE_FIRST_DYNAMIC) {
7729     iput(dp);
7730     return AGENT_STATUS_DENIED;
7731 }
7732 existing = dirlookup(dp, key, 0, VFS_POLICY_WORKFLOW,
7733     cred->scope_id, &lookup_status);
7734 if (existing != 0) {
7735     iput(existing);
7736     iput(dp);
7737     return AGENT_STATUS_DUPLICATE;
7738 }
7739 if (lookup_status != FS_LOOKUP_ABSENT) {
7740     iput(dp);
7741     return fs_agent_publish_io_status(lookup_status);
7742 }
7743 if (fs_storage_charge_from_vfs(cred, &charge) < 0) {
7744     iput(dp);
7745     return AGENT_STATUS_DENIED;
7746 }
7747 ip = ialloc(dp->dev, T_FILE, &charge, &lookup_status);
7748 if (ip == 0) {
7749     iput(dp);
7750     if (lookup_status == FS_LOOKUP_BUSY)
7751         return AGENT_STATUS_RETRY;
7752     return fs_io_health == FS_IO_HEALTHY ? AGENT_STATUS_NO_SPACE :
7753         fs_agent_publish_io_status(lookup_status);
7754 }
7755 result = invalid(ip);
7756 if (result < 0)
7757     goto fail_inode;
7758 if (ip->type != T_FILE ||
7759     fs_qmap_transition_owner(ip->fs_owner_domain,
7760         FS_QMAP_ALLOCATING_FLAG,
7761         &intent_owner) < 0 ||
7762     intent_owner != charge.owner) {
7763     result = -1;
7764     goto fail_inode;
7765 }
7766 ip->fs_owner_domain = charge.owner;
7767 result = vfs_inode_init_label(ip, cred, VFS_POLICY_WORKFLOW);
7768 if (result < 0)
7769     goto fail_inode;
7770 result = iupdate(ip);
7771 if (result < 0)
7772     goto fail_inode;
7773 result = fs_preallocate_inode(ip, cred, total);
7774 if (result < 0) {
7775     if (result == FS_LOOKUP_BUSY ||
7776         result == VIRTIO_DISK_ERR_BUSY)
7777         status = AGENT_STATUS_RETRY;
7778     else
7779         status = fs_io_health == FS_IO_HEALTHY ?
7780             AGENT_STATUS_NO_SPACE :

```

```

7781         fs_agent_publish_io_status(result);
7782         goto fail_known;
7783     }
7784     result = writei(ip, cred, 0, (uint64)snapshot, 0, total);
7785     if (result != (int)total) {
7786         status = result > 0 && fs_io_health == FS_IO_HEALTHY ?
7787             AGENT_STATUS_RETRY : fs_agent_publish_io_status(result);
7788         goto fail_known;
7789     }
7790     /* The inode image and every data block must be durable before its only
7791     * namespace attachment can be staged. */
7792     result = fs_publish_checkpoint();
7793     if (result < 0) {
7794         status = AGENT_STATUS_INDETERMINATE;
7795         goto fail_known;
7796     }
7797     result = dirlink_publish(dp, key, ip->inum, cred);
7798     if (result == 0) {
7799         iput(ip);
7800         iput(dp);
7801         return AGENT_STATUS_OK;
7802     }
7803     if (result == FS_LOOKUP_INDETERMINATE ||
7804         fs_io_health == FS_IO_INDETERMINATE) {
7805         /* The official dirent may already exist; do not reclaim its inode. */
7806         iput(ip);
7807         iput(dp);
7808         return AGENT_STATUS_INDETERMINATE;
7809     }
7810     if (result == FS_DIRLINK_EXISTS)
7811         status = AGENT_STATUS_DUPLICATE;
7812     else if (result == FS_DIRLINK_NO_SPACE)
7813         status = AGENT_STATUS_NO_SPACE;
7814     else
7815         status = fs_agent_publish_io_status(result);
7816     goto fail_known;
7817
7818 fail_inode:
7819     status = fs_agent_publish_io_status(result);
7820 fail_known:
7821     iput(dp);
7822     return fs_agent_publish_discard(ip, status);
7823 }

```

代码 28: Nexus 对重复或不确定发布的精确核对

c

4. 项目测试

测试先核对接口结构和构建结果，再进入 RISC-V Guest 检查各项内核机制，最后运行控制台、Nexus 和相应的性能负载。功能结果来自真实系统调用输出和协议摘要；性能数据保留逐样本来源，配对比较、分布图和统计表都能追溯到对应的启动记录。各组测量随所检查的机制展开，不另设脱离功能路径的展示性实验。

4.1. 功能与性能测试

4.1.1. 测试组织

AgentOS-uCore 的测试从接口结构、内核镜像、Guest 行为和综合场景四个方向交叉检查。结构约定检查核对 UAPI 的尺寸、偏移和模块依赖；当前冻结布局清单覆盖 640 项合约，其中包含 26 项 Tool Registry 和系统调用 567/568 的 delegated Task 结构。交叉编译把真实内核和用户程序链接为 RISC-V 镜像；Guest 场景通过系统调用观察 Agent 身份、Context、VFS、IPC、资源和调度状态；Host 测试则覆盖帧编码、Provider、工作区 broker、Task ledger、artifact 与验证脚本。性能测量直接读取同一 Guest 工作负载产生的时钟和计数，不另写一套只用于出图的模拟程序。

对应机制	主要程序	观察重点
Agent 进程与 Context 区	agenteval_ucore, agenttrust_ucore, agentscope_ucore, agentfinal_ucore	创建、派生、装入和退出；固定映射、直接读取、回滚、清空与 FIFO 淘汰
结构化接口与工具协议	agenteval_ucore, agenttoolabi_ucore, agentcontract_ucore, agenttask_ucore	26 项目录与 schema；V2/V3；UTF-8/TASK 资源；TASK route、claim/complete、终态确认、唯一 CQE、背压与重新同步
文件查询与实时事件	agenteval_ucore, agentfs_ucore, agentbench_ucore	元数据与摘要；遍历和索引结果一致性；ENTER、UPDATE、LEAVE 与缺口恢复
Agent Loop 与 EEVDF	agenteval_ucore, agentloop_ucore, agentsched_ucore, agent_eevdf_ucore	原子等待、消息与 TIMER 唤醒、超时、线程放大、工作流服务份额
综合场景	labdemo_ucore, agentpublish_ucore, agentnexus_ucore	恢复流程；Nexus 三角色、Context active path、Guest Catalog、版本化工作区字节、原生 Task 委派与会话关闭

表 4.1: 系统机制、测试程序与观察重点

自动化检查依次覆盖智能体模块与 ABI 结构约定、Nexus 协议约定和 Host 测试程序。严格的 Nexus ReplayProvider 在全新 QEMU 启动中逐轮核对 Guest 规范请求摘要，并走过模型自主工具选择、Catalog 候选、Task Channel 委派、真实 TOOL artifact、Context 跨轮重建、回滚/清空和完整关闭路径。

```

1 make agent-module-check
2 make agentos-build TOOLPREFIX=riscv64-linux-gnu-
3 make agentos-nexus-check TOOLPREFIX=riscv64-linux-gnu-
4 make local-host-selftests

```

4.1.2. 任务一至任务五的 Guest 综合验收

agenteval_ucore 把赛题任务一至任务五放在同一个 RV64 Guest 中检查。运行命令如下：

```
1 AGENT_TEST_CASE=agenteval_ucore \
2 make agentos-test TOOLPREFIX=riscv64-linux-gnu-
```

这项测试关注系统调用之后真正可见的状态，而不是在 Host 侧搜索“成功”字符串。五段场景的观察内容见表4.2。

场景	Guest 中可观察的行为
Agent 进程创建与地址空间	Agent 与普通进程可以并存；身份和配额可查询；固定高地址的七页 Context 区中，六页内核镜像保持只读，用户缓存页可写。
内核结构化交互接口	Guest 能列举工具并发起多种带类型信息的调用；未知工具、错误参数类型和小返回缓冲区得到不同的结构化状态。
上下文路径管理	六轮以上连续调用的直接映射读取与系统调用查询一致；回滚和清空改变后续可见路径；连续写入 133 条记录后只保留最新 128 条，系统继续运行且未耗尽内存。
面向 Agent 的文件查询	文件按属性和摘要被找到，不要求调用者预先知道完整路径；遍历与索引返回同一结果；结构化结果可以写入第七页用户缓存。
Agent Loop 内核运行机制	等待线程在无事件时睡眠，消息或 TIMER 到达后继续运行；多个 Agent 同时活动时，工作流调度统计持续推进。

表 4.2: agenteval_ucore 的五段可观察行为

这组综合验收把创建、工具调用、Context、文件查询和 Loop 串在同一次启动中，能够发现单项测试不易暴露的状态衔接问题。例如，Context 映射若只在创建时存在、exec 后丢失，后续的查询缓存场景便无法继续；文件事件若没有正确唤醒等待者，Loop 场景也不会仅凭查询结果“看起来正常”。

4.1.3. 结构化工具与任务通道测量

任务通道测量让紧凑 Batch、单项 V3 和 SQ/CQ 分别执行 16 次等价 ECHO，并轮换三条路径的先后顺序。每条操作序列记录完整墙钟时间，同时核对工具、状态、Context 序号、执行记录票号和结果指纹，避免把少做检查或少执行操作误判为性能提升。4 次启动共保留 96 条操作序列。

图4.1中，紧凑 Batch、单项 V3 和 SQ/CQ 完成 16 项等价操作的中位数分别为 561 微秒、2051 微秒和 1620.5 微秒。三条路径分别使用 1、16、2 次系统调用：Batch 在一次进入内顺序执行短操作；V3 为每一项绑定执行约定、节点和尝试号；SQ/CQ 通过共享队列提交并回收结果。每次启动的首轮中位数依次为 5.747、7.098、6.1435 ms，后续轮次降至 0.548、1.9965、1.5615 ms，说明刚进入该路径时的建立和缓存开销会明显抬高延迟，不能只报后续轮次的中位数。

这一结果表明，短小、同质且需要同步返回的操作适合 Batch，系统没有必要为了接口形式统一而强制走更重的路径。V3 多付出的时间对应 Execution Contract、节点、尝试号和执行记录检查；SQ/CQ 的价值则在于有界提交、背压、取消和唯一 terminal CQE，而不是压低 16 个短调用的最低延迟。这组冻结数据只测量本地等价 ECHO，没有重新计入当前 Nexus 的跨 Agent claim/complete 或长 Provider 延迟，因此不能据此推断异步 backlog 下的表现。

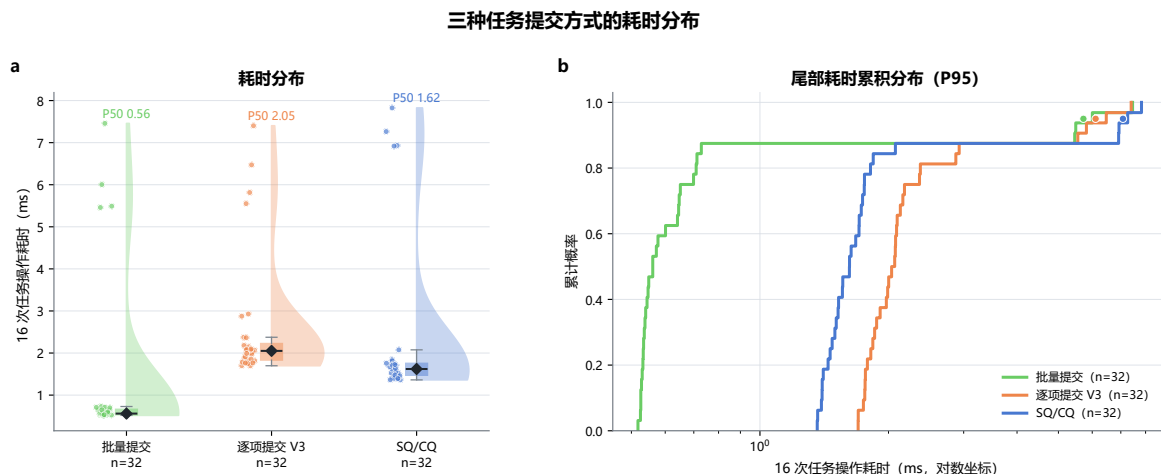


图 4.1: 三种任务提交方式的耗时分布

4.1.4. 结果文件发布与冲突处理

结果文件发布测试直接在 Guest 中调用 `agent_file_publish()`，同时检查正常写入、同名竞争、错误参数和 Nexus 收敛逻辑。运行命令如下：

```
1 AGENT_TEST_CASE=agentpublish_ucore \
2 make agentos-test TOOLPREFIX=riscv64-linux-gnu-
```

正常路径读取到完整 header、payload 和 EOF。同一访问范围内的两个调用并发发布同名文件时，结果恰好为一个 OK 和一个 DUPLICATE，已经发布的内容没有被覆盖。坏指针、非法路径、错误结构大小、错误版本和非零保留字段均未留下正式目录项；失败与重复请求也没有增加 workflow 资源计数，删除测试生成的两份文件后，块和 inode 计数回到基线。

Nexus 随后按正式路径逐字节回读。已有内容与本次 header、payload 和 EOF 完全一致时，发布结果可以收敛为成功；内容不同则明确拒绝。这组结果表明，未命名 inode 与单次正式目录接入避免了“文件名已经可见、内容尚未写完”的中间状态，同名竞争也没有演变为静默覆盖。失败和重复发布同样沿用 workflow 资源账户的记账与回收路径。

通用 FS epoch 动态回归在 dirty、inflight 和 durable 三种状态下均通过，覆盖有序批提交、fsync、重启重放和既有 power-cut 窗口。作为共用的分配、回收和恢复基础，分配器故障与重启矩阵的 36 个用例也全部得到预期状态；去掉关键 FLUSH 的变异会被测试捕获。结果文件发布沿用这条 checkpoint 与恢复路径。

4.1.5. 文件查询路径的 I/O 观测

查询测量同时记录文件读写、系统调用、扫描记录数、任务完成次数和 workflow 服务次数。图 4.2 上半部分把核心路径的 bytes_read 和完整流程的全局 I/O 计数分别按核心时间窗内的系统调用次数归一化；下半部分按已完成操作数展示三种任务提交方式的传输成本。

图 4.2 的颜色只能在同一列、同一指标和同一统计窗口内比较。原始中位数显示，indexed 路径把 physical reads 从 143.5 降到 58、VirtIO notify 从 247.5 降到 165、requests 从 402.5 降到 332。这与选择性索引先缩小候选集的设计相符，说明查询优化已经减少了读路径上的实际工作量。与此同时，writes 从 213 增到 225.5、flush 从 47 增到 48，目录探测和检查记录也没有下降；系统当前只是把核心查询做得更省，并没有使完整流程中的写入、刷新和收尾同步变轻，这与图 3.5 中的端到端结果一致。

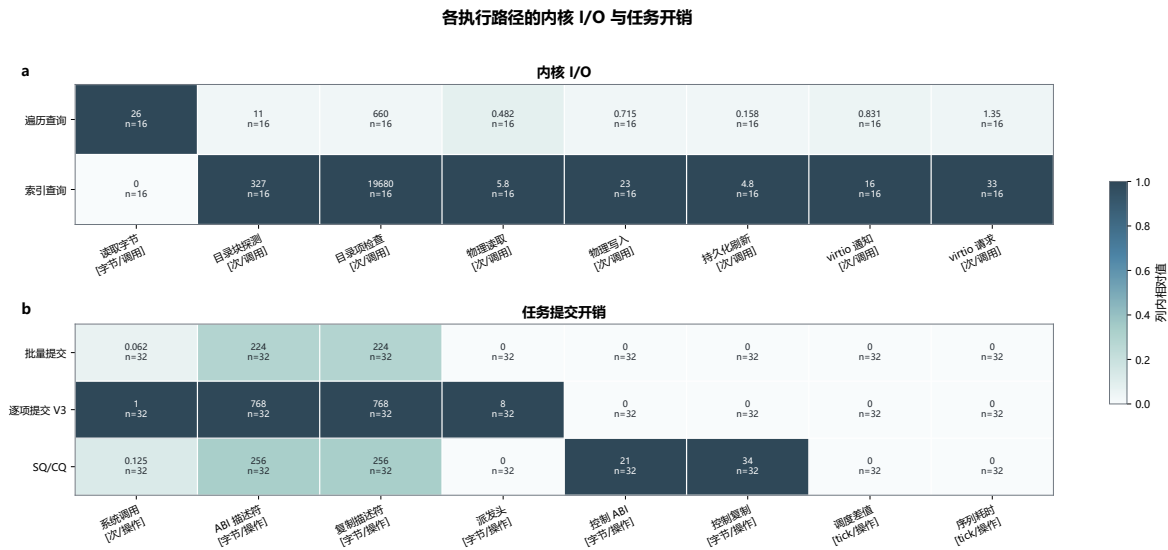


图 4.2: 各查询与任务路径的内核 I/O 观测

任务面板的 tick 中位数为 0 也不代表没有耗时：Batch、单项 V3、SQ/CQ 分别有 7/32、9/32、6/32 轮跨过 1 个 tick。10 ms 的 tick 适合观察是否跨越调度节拍，却不足以分辨这些微秒级路径的细小差异，因此延迟结论仍以墙钟测量为准。

4.1.6. 测量数据与可重复性

性能测量使用冻结在源码提交 2b14fb1f74b9bd093e6de939a16554620835699e 的 30 次相互独立全新 QEMU 启动，保存 33 份原始串口输出、19 个 CSV 数据表和 7,498 条结构化记录。每条记录都带有来源文件、来源 SHA-256 和原文序号。同一次启动中的内部配对用于观察相同工作量下的波动；跨启动汇总时，以源启动记录作为独立单位。

对应机制	测量设计	独立单位	样本量
综合恢复与文件查询	16 个 AB/BA 均衡配对启动	启动记录	16 组核心路径和完整流程配对
元数据目录与索引	3 × 4 参数网格，每格 15 个内部配对	源启动记录	180 组查询配对
工具协议与任务通道	4 次启动，8 组轮换顺序	启动记录/区段	96 条 16 次操作序列
Agent Loop 与 EEVDF	6 次启动，并发度为 1–4	启动记录/时间窗和探针	504 条精确唤醒探针

表 4.3: 各内核机制的测量数据构成

数据清单记录源码提交、工具版本、工作量计划、Guest 与构建哈希，并汇总原始输出、CSV 数据表和图像文件哈希；清单中的 75 项均通过哈希校验。保存这些材料的目的是，是让图表中的每个点都能回到对应的 Guest 输出。当前功能改动的正确性由日常回归另行验证；性能数据不替代功能验收，也不表示这些改动之后已重新测量。

4.2. 测试路径与结果

测试不只比对用户态输出文本，还通过真实 RV64 Guest 调用观察内核对象的状态、返回码、上下文序号、代次和运行统计。功能场景、机制探针和性能测量共用同一构建产物，因此表4.1中的项目都能追溯到实际执行路径。

4.2.1. Guest 原生工作负载

labdemo_ucore 在 Guest 中创建并使用 AgentOS 对象，收集查询、文件写入、消息等待和调度路径上的计数与延迟。遍历路径和索引路径共用业务工作量、结果指纹和报告结构，因此比较不会把 Host 预处理、日志格式或测试框架差异误认为内核性能差异。

代码 29: Guest 原生 AgentOS 工作负载的执行入口

c

```

745 static void run_native_workload(struct labdemo_workload_metrics *metrics)
746 {
747     static struct agent_file_query query;
748     static struct agent_file_query_result result;
749     static struct agent_file_meta target;
750     struct labdemo_lane_measurement_state *state = &native_measurement;
751     static char name[6];
752
753     memset(metrics, 0, sizeof(*metrics));
754     demo_quiescence_fence("native", "E2E_START", 1, &state->e2e_start);
755     metrics->end_to_end_started_us = demo_now_us();
756     seed_native_workload(&target);
757     memset(&query, 0, sizeof(query));
758     query.flags = AGENT_FILE_QUERY_USE_INDEX;
759     query.max_hits = AGENT_FILE_QUERY_MAX_HITS;
760     strcpy(query.project, DEMO_PROJECT);
761     strcpy(query.run_id, DEMO_BENCH_RUN);
762     strcpy(query.status, "failed");
763     demo_quiescence_fence("native", "CORE_START", 2, &state->core_start);
764     metrics->started_us = demo_now_us();
765     DEMO_DIAG_BEGIN();
766     check(agent_file_query(&query, &result) == 1,
767           "native failed-stage query");
768     DEMO_DIAG_END("native", "failed_index_query");
769     metrics->records_examined += result.scanned_records;
770     check(result.returned == 1 && result.used_index == 1 &&
771           strcmp(result.hits[0].stage, "align") == 0 &&
772           strcmp(result.hits[0].status, "failed") == 0,
773           "native failed-stage result");
774     metrics->discovered_us = demo_now_us();
775     demo_corpus_name(name, 'n', DEMO_BENCH_TARGET);
776     DEMO_DIAG_BEGIN();
777     {
778         int fd = open(name, O_WRONLY | O_TRUNC);
779
780         check(fd >= 0, "native recovery open");
781         check(write(fd, DEMO_BENCH_RECOVERED_BODY,
782                   strlen(DEMO_BENCH_RECOVERED_BODY)) ==
783               (ssize_t)strlen(DEMO_BENCH_RECOVERED_BODY),
784               "native recovery write");

```

```

785     target.update_mask = AGENT_FILE_META_UPDATE_STATUS |
786         AGENT_FILE_META_UPDATE_SUMMARY;
787     strcpy(target.status, "recovered");
788     strcpy(target.summary, "memory_limit recovered");
789     check(agent_file_meta_set(&target) == 0,
790         "native recovery metadata stage");
791     check(demo_quiescence_fsync(fd) == 0,
792         "native recovery primary ack");
793     check(close(fd) == 0, "native recovery close");
794 }
795 DEMO_DIAG_END("native", "grouped_recovery_primary_ack");
796 metrics->committed_us = demo_now_us();
797 strcpy(query.status, "recovered");
798 memset(&result, 0, sizeof(result));
799 DEMO_DIAG_BEGIN();
800 check(agent_file_query(&query, &result) == 1,
801     "native recovered-stage query");
802 DEMO_DIAG_END("native", "recovered_index_query");
803 metrics->records_examined += result.scanned_records;
804 check(result.returned == 1 && result.used_index == 1 &&
805     strcmp(result.hits[0].stage, "align") == 0 &&
806     strcmp(result.hits[0].status, "recovered") == 0,
807     "native recovered-stage result");
808 metrics->result_items = 1;
809 metrics->outcome_hash = demo_outcome_hash(result.hits[0].stage,
810     result.hits[0].status);
811 metrics->finished_us = demo_now_us();
812 take_performance_snapshot(&state->core_ack);
813 metrics->workload_syscalls = performance_delta(
814     state->core_start.performance.snapshot.observer_workload_syscalls,
815     state->core_ack.snapshot.observer_workload_syscalls);
816 check(metrics->workload_syscalls != 0, "native workload syscall receipt");
817 demo_quiescence_fence("native", "ACK_SETTLED", 3,
818     &state->ack_settled);
819 DEMO_DIAG_BEGIN();
820 for (int i = 0; i < LABDEMO_CORPUS_SIZE; i++) {
821     demo_corpus_name(name, 'n', i);
822     check(unlink(name) == 0, "native corpus cleanup");
823 }
824 DEMO_DIAG_END("native", "cleanup_unlink");
825 demo_quiescence_fence("native", "E2E_END", 4, &state->e2e_end);
826 metrics->end_to_end_finished_us = demo_now_us();
827 print_workload_lane("native", metrics);
828 print_mechanism_delta("native", "core", &state->core_start.performance,
829     &state->core_ack);
830 print_mechanism_delta("native", "end_to_end",
831     &state->e2e_start.performance,
832     &state->e2e_end.performance);
833 }
834
835 static void make_op(struct agent_op *op, int tool, uint64 id, uint64 arg0,
836     const char *payload)
837 {
838     memset(op, 0, sizeof(*op));
839     op->version = AGENT_CALL_VERSION;
840     op->tool_id = tool;
841     op->request_id = id;

```

```

842     op->arg0 = arg0;
843     if (payload)
844         strcpy(op->payload, payload);
845 }

```

查询图表由当前测量目录中的原始数据生成。核心路径与完整流程窗口的对比见第3.6章图3.5；该图同时保留机制路径上的时间和资源观测值。所有数值对应当前 QEMU/u-Core 配置、给定工作负载和该轮运行，不将其外推为其他硬件平台的固定吞吐量。

4.2.2. 工具、执行约定与任务通道测试

Nexus Runtime 发起跨 Agent 工具调用时，会先发布 Guest capsule artifact，再把 56 字节不可变描述符导入 Task Channel。提交路径把 delegate_task SQE 绑定到 Execution Contract；目标 Agent 经 claim 取得描述符并再次核对 owner、目标身份、任务关联和 capsule artifact。该路径能区分用户态编码失败、内核 schema/route 拒绝、任务终态和工具副作用失败。

代码 30: Nexus delegated Task 的资源导入与通道提交

c

```

6146     (unsigned long)result.sq_base;
6147     nexus_task_channel.cq = (volatile const struct nexus_task_cq_page *)
6148     (unsigned long)result.cq_base;
6149     nexus_volatile_read(&sq_header, &nexus_task_channel.sq->header,
6150         sizeof(sq_header));
6151     nexus_volatile_read(&cq_header, &nexus_task_channel.cq->header,
6152         sizeof(cq_header));
6153     if (sq_header.magic != AGENT_TASK_CHANNEL_SQ_MAGIC ||
6154         cq_header.magic != AGENT_TASK_CHANNEL_CQ_MAGIC ||
6155         sq_header.generation != result.generation ||
6156         cq_header.generation != result.generation ||
6157         (sq_header.flags & AGENT_TASK_CHANNEL_RING_F_ACTIVE) == 0)
6158         return -1;
6159     nexus_task_channel.request_id = 500000ULL;
6160     return 0;
6161 }
6162
6163 static int nexus_task_contract_retire(
6164     const struct agent_execution_contract_key *key);
6165
6166 static int nexus_task_observer_pause(
6167     struct nexus_delegated_submission *submission)
6168 {
6169     if (submission == 0 || submission->observer_paused ||
6170         live_observer_mutex < 0 || mutex_lock(live_observer_mutex) != 0)
6171         return -1;
6172     if (nexus_task_contract_active)
6173         live_fail("serialize delegated Task contract window");
6174     nexus_task_contract_active = 1;
6175     submission->observer_paused = 1;
6176     return 0;
6177 }
6178
6179 static int nexus_task_observer_resume(
6180     struct nexus_delegated_submission *submission)
6181 {

```

```

6182     if (submission == 0 || !submission->observer_paused ||
6183         !nexus_task_contract_active)
6184         return -1;
6185     nexus_task_contract_active = 0;
6186     submission->observer_paused = 0;
6187     return mutex_unlock(live_observer_mutex) == 0 ? 0 : -1;
6188 }
6189
6190 static int nexus_task_contract_create(
6191     struct agent_execution_contract_key *key)
6192 {
6193     static struct agent_execution_contract_control control;
6194     static struct agent_execution_contract_result result;
6195
6196     memset(&nexus_task_contract_node, 0,
6197         sizeof(nexus_task_contract_node));
6198     nexus_task_contract_node.version =
6199         AGENT_EXECUTION_CONTRACT_NODE_VERSION;
6200     nexus_task_contract_node.size = sizeof(nexus_task_contract_node);
6201     nexus_task_contract_node.node_id = 0;
6202     nexus_task_contract_node.tool_id = AGENT_TOOL_DELEGATE_TASK;
6203     nexus_task_contract_node.required_capabilities = AGENT_CAP_ORCHESTRATE;
6204     nexus_task_contract_node.accepted_input_labels =
6205         AGENT_PROVENANCE_KERNEL_FACT |
6206         AGENT_PROVENANCE_TRUSTED_USER_CONTROL |
6207         AGENT_PROVENANCE_AGENT_DERIVED |
6208         AGENT_PROVENANCE_UNTRUSTED_FILE_DATA |
6209         AGENT_PROVENANCE_UNTRUSTED_TOOL_OUTPUT |
6210         AGENT_PROVENANCE_CROSS_AGENT_DATA;
6211     nexus_task_contract_node.output_add_labels =

```

代码 31: 目标 Agent 对 claim 与 capsule artifact 的一致性核验

c

```

7970
7971     if (nexus_workspace_object_logical(entry->object_id, logical) < 0)
7972         return -1;
7973     nexus_workspace_run_id(nexus_workspace_generation, run_id);
7974     memset(&nexus_workspace_query, 0, sizeof(nexus_workspace_query));
7975     nexus_workspace_query.flags = AGENT_FILE_QUERY_USE_INDEX;
7976     nexus_workspace_query.max_hits = AGENT_FILE_QUERY_MAX_HITS;
7977     strcpy(nexus_workspace_query.logical_path, logical);
7978     strcpy(nexus_workspace_query.project, NEXUS_WORKSPACE_PROJECT);
7979     strcpy(nexus_workspace_query.workflow, NEXUS_WORKSPACE_WORKFLOW);
7980     strcpy(nexus_workspace_query.run_id, run_id);
7981     strcpy(nexus_workspace_query.kind, NEXUS_WORKSPACE_FILE_KIND);
7982     strcpy(nexus_workspace_query.status, NEXUS_WORKSPACE_READY);
7983     memset(&nexus_workspace_query_result, 0,
7984         sizeof(nexus_workspace_query_result));
7985     if (agent_file_query(&nexus_workspace_query,
7986         &nexus_workspace_query_result) != 1 ||
7987         nexus_workspace_query_result.returned != 1 ||
7988         nexus_workspace_query_result.truncated != 0 ||
7989         strcmp(nexus_workspace_query_result.hits[0].logical_path, logical) ||
7990         nexus_workspace_query_result.hits[0].dev == 0 ||
7991         nexus_workspace_query_result.hits[0].inum == 0 ||
7992         nexus_workspace_query_result.hits[0].incarnation == 0)

```

```

7993     return -1;
7994     return 0;
7995 }
7996
7997 static int nexus_workspace_read_projection_valid(
7998     const char *content, const char *path)
7999 {
8000     static char value[NEXUS_FILE_PATH_MAX_CODEPOINTS * 4U + 1U];
8001     const char *cursor = content;
8002
8003     if (!strcmp(content, "workspace_error=binary_file") ||
8004         !strcmp(content, "workspace_error=file_too_large") ||
8005         !strcmp(content, "workspace_error=start_line_out_of_range") ||
8006         !strcmp(content, "workspace_error=line_too_large"))
8007         return 1;
8008     return nexus_workspace_line(&cursor, "", value, sizeof(value), 0) == 0 &&
8009         !strcmp(value, "workspace_read") &&
8010         nexus_workspace_line(&cursor, "content_untrusted=", value,
8011             sizeof(value), 0) == 0 &&
8012         !strcmp(value, "1") &&
8013         nexus_workspace_line(&cursor, "path=", value,
8014             sizeof(value), 0) == 0 &&
8015         !strcmp(value, path);
8016 }
8017
8018 static int nexus_workspace_read(
8019     const char *path, uint start_line, uint max_lines, uint64 turn_id,
8020     uint64 request_id, uint64 corr_id, uint64 task_id,
8021     char *output, uint capacity, uint *output_size,
8022     char output_sha256[LIVE_SHA_HEX_SIZE + 1],
8023     char source_sha256[LIVE_SHA_HEX_SIZE + 1])
8024 {
8025     uint attempt = 0;
8026     uint cursor = 0;
8027     uint restarts = 0;
8028
8029     if (nexus_workspace_forget_generation() < 0)
8030         return AGENT_STATUS_IO_ERROR;
8031     nexus_workspace_source_reset();
8032     for (;;) {
8033         const struct nexus_workspace_entry *entry = 0;
8034         struct live_builder builder;

```

执行约定测试覆盖过期生命周期、过期执行约定、未知节点、工具不匹配、截止时间和依赖终止状态（terminal state）等分支。副作用前检查产生的完成记录关联执行记录票号和输出来源；对于直接调用或 V3 调用的拒绝，如果无法建立关键记录，调用会返回 NO_SPACE/RETRY，测试同样覆盖这条记录失败语义。因此，拒绝或取消不是孤立的错误码，而是能够与上下文、执行记录或明确的记录失败状态关联的结果。

代码 32：执行约定完成时的执行记录与 terminal state 不变量

c

```

1834     return AGENT_EXECUTION_EFFECT_ALLOWED;
1835     if (claim->slot < 0 || claim->slot >= WORKFLOW_LIFECYCLE_CAP ||
1836         claim->node_id >= AGENT_EXECUTION_CONTRACT_MAX_NODES)
1837         return AGENT_EXECUTION_EFFECT_STALE;

```

```

1838     enabled = intr_save();
1839     record = &agent_execution_contracts[claim->slot];
1840     if (!agent_execution_record_matches(record, claim->lifecycle) ||
1841         record->generation != claim->contract_generation ||
1842         claim->node_id >= record->node_count) {
1843         intr_restore(enabled);
1844         return AGENT_EXECUTION_EFFECT_STALE;
1845     }
1846     runtime = &record->runtime[claim->node_id];
1847     if (runtime->state != AGENT_EXECUTION_NODE_RUNNING ||
1848         runtime->request_id != claim->request_id ||
1849         runtime->attempt_id != claim->attempt_id ||
1850         !agent_execution_digest_equal(runtime->request_digest,
1851             claim->request_digest)) {
1852         intr_restore(enabled);
1853         return AGENT_EXECUTION_EFFECT_STALE;
1854     }
1855     if ((runtime->flags &
1856         AGENT_EXECUTION_RUNTIME_F_TIMEOUT_REQUESTED) != 0) {
1857         claim->decision_reason =
1858             AGENT_EXECUTION_REASON_DEADLINE_EXPIRED;
1859         intr_restore(enabled);
1860         return AGENT_EXECUTION_EFFECT_STALE;
1861     }
1862     if ((runtime->flags &
1863         AGENT_EXECUTION_RUNTIME_F_CANCEL_REQUESTED) != 0) {
1864         claim->decision_reason =
1865             AGENT_EXECUTION_REASON_CANCEL_REQUESTED;
1866         if ((runtime->flags &
1867             AGENT_EXECUTION_RUNTIME_F_FORCE_CANCEL) != 0)
1868             claim->retry_forbidden = 1;
1869         intr_restore(enabled);
1870         return AGENT_EXECUTION_EFFECT_CANCELLED;
1871     }
1872     runtime->flags |= AGENT_EXECUTION_RUNTIME_F_EFFECT_STARTED;
1873     intr_restore(enabled);
1874     return AGENT_EXECUTION_EFFECT_ALLOWED;
1875 }
1876
1877 enum agent_execution_delegate_cancel_admission
1878 agent_execution_contract_delegate_cancel_preflight_locked(
1879     struct agent_execution_claim *claim, uint64 now)
1880 {
1881     struct agent_execution_contract_record *record;
1882     struct agent_execution_node_internal *node;
1883     struct agent_execution_node_runtime *runtime;
1884
1885     if (intr_get())
1886         panic("execution delegated cancel unlocked");
1887     if (claim == 0 || !claim->active || claim->legacy ||
1888         claim->slot < 0 || claim->slot >= WORKFLOW_LIFECYCLE_CAP ||
1889         claim->node_id >= AGENT_EXECUTION_CONTRACT_MAX_NODES)
1890         return AGENT_EXECUTION_DELEGATE_CANCEL_STALE;
1891     record = &agent_execution_contracts[claim->slot];
1892     if (!agent_execution_record_matches(record, claim->lifecycle) ||
1893         record->generation != claim->contract_generation ||
1894         claim->node_id >= record->node_count)

```

```

1895     return AGENT_EXECUTION_DELEGATE_CANCEL_STALE;
1896     node = &record->nodes[claim->node_id];
1897     runtime = &record->runtime[claim->node_id];
1898     if (runtime->state != AGENT_EXECUTION_NODE_RUNNING ||
1899         runtime->request_id != claim->request_id ||
1900         runtime->attempt_id != claim->attempt_id ||
1901         !agent_execution_digest_equal(runtime->request_digest,
1902                                         claim->request_digest))
1903     return AGENT_EXECUTION_DELEGATE_CANCEL_STALE;
1904     if ((runtime->flags & AGENT_EXECUTION_RUNTIME_F_TIMEOUT_REQUESTED) != 0 ||
1905         (claim->deadline_tick != 0 && now >= claim->deadline_tick)) {
1906         claim->decision_reason = AGENT_EXECUTION_REASON_DEADLINE_EXPIRED;
1907         claim->retry_forbidden = 1;
1908         return AGENT_EXECUTION_DELEGATE_CANCEL_TIMEOUT;
1909     }
1910     if (node->cancel_policy != AGENT_EXECUTION_CANCEL_ALLOW)

```

任务通道在提交后维护已接纳和运行状态、生命周期引用以及回调引用；硬截止时间触发过期，完成或拒绝进入统一的完成队列。该状态机的测试同时验证“不丢失生命周期引用”和“不在过期后伪装完成”两项性质。

代码 33：任务提交后的回调、截止时间与完成分支

c

```

1900     return 0;
1901 }
1902 }
1903 if (agent_task_request_find_locked(private, sqe.request_id) != 0) {
1904     status = agent_task_channel_protocol_fault_locked(state);
1905     intr_restore(enabled);
1906     return status;
1907 }
1908 if (agent_task_resource_view_locked(
1909     state->resource_private, sqe.input, &input_view) < 0) {
1910     status = agent_task_channel_protocol_fault_locked(state);
1911     intr_restore(enabled);
1912     return status;
1913 }
1914 request = agent_task_request_alloc_locked(private);
1915 if (request == 0) {
1916     private->header.backpressure++;
1917     agent_task_channel_publish_locked(state);
1918     intr_restore(enabled);
1919     return 0;
1920 }
1921 lifecycle = agent_task_lifecycle_key(p);
1922 if (workflow_lifecycle_operation_enter(lifecycle) < 0) {
1923     intr_restore(enabled);
1924     return AGENT_TASK_CHANNEL_RETRY;
1925 }
1926 agent_task_callback_get_locked(state);
1927 intr_restore(enabled);
1928 if ((sqe.flags & AGENT_TASK_SQE_F_HARD_DEADLINE) != 0 &&
1929     (ops == 0 || ops->expire == 0)) {
1930     agent_task_validation_abort(p, ops);
1931     return AGENT_TASK_CHANNEL_EVIDENCE;

```



```

1932     }
1933     hook_status = ops->validate(
1934         p, &sqe, &input_view, &validation, &completion);
1935     if (hook_status != AGENT_TASK_HOOK_PENDING) {
1936         agent_task_validation_abort(p, ops);
1937         return AGENT_TASK_CHANNEL_RETRY;
1938     }
1939     if (!agent_task_validation_valid(&validation)) {
1940         agent_task_validation_abort(p, ops);
1941         return AGENT_TASK_CHANNEL_EVIDENCE;
1942     }
1943     enabled = intr_save();
1944     state = agent_task_channel_find_locked(p);
1945     if (!agent_task_channel_issuer_valid_locked(state, p, issuer) ||
1946         state->ops != ops ||
1947         state->private->header.generation != generation ||
1948         state->private->header.sq_head != position) {
1949         if (!agent_task_channel_owner_valid_locked(state, p) ||
1950             state->ops != ops ||
1951             state->private->header.callback_refs == 0)
1952             panic("Task Channel validate cleanup");
1953         agent_task_callback_put_locked(state);
1954         workflow_lifecycle_operation_leave(state->lifecycle);
1955         intr_restore(enabled);
1956         agent_task_reclaim_deferred(p, ops);
1957         return AGENT_TASK_CHANNEL_STALE;
1958     }
1959     private = state->private;
1960     request = agent_task_request_alloc_locked(private);
1961     if (request == 0) {
1962         private->header.backpressure++;
1963         if (private->header.callback_refs == 0)
1964             panic("Task Channel validate pin");
1965         agent_task_callback_put_locked(state);
1966         workflow_lifecycle_operation_leave(lifecycle);
1967         intr_restore(enabled);
1968         return 0;
1969     }
1970     memset(request, 0, sizeof(*request));
1971     request->sqe = sqe;
1972     request->accepted_tick = agent_task_now();
1973     request->expected_output_type = validation.output_artifact_type;
1974     request->provenance_labels = validation.output_provenance_labels;
1975     request->state = AGENT_TASK_REQUEST_ACCEPTED;
1976     request->flags = AGENT_TASK_REQUEST_F_LIFECYCLE_HELD;
1977     if (private->header.callback_refs == 0)
1978         panic("Task Channel submit transfer");
1979     agent_task_callback_put_locked(state);
1980     if (agent_task_request_input_acquire_locked(
1981         private, state->resource_private, request) < 0) {
1982         request->flags &= ~AGENT_TASK_REQUEST_F_LIFECYCLE_HELD;
1983         workflow_lifecycle_operation_leave(lifecycle);
1984         memset(request, 0, sizeof(*request));
1985         status = agent_task_channel_protocol_fault_locked(state);
1986         intr_restore(enabled);
1987         return status;
1988     }

```

```

1989     request->state = AGENT_TASK_REQUEST_RUNNING;
1990     private->header.sq_head++;
1991     private->header.submitted++;
1992     private->header.last_accepted_request_id = sqe.request_id;
1993     *consumed = 1;
1994     private->header.live_requests++;
1995     agent_task_callback_get_locked(state);
1996     agent_task_channel_publish_locked(state);
1997     intr_restore(enabled);
1998     hook_status = ops->submit(
1999         p, &sqe, &input_view, &validation, &completion);
2000     enabled = intr_save();
2001     state = agent_task_channel_find_locked(p);
2002     if (!agent_task_channel_owner_valid_locked(state, p) ||
2003         state->ops != ops || state->private->header.callback_refs == 0)
2004         panic("Task Channel submit callback");
2005     agent_task_callback_put_locked(state);
2006     intr_restore(enabled);
2007     agent_task_reclaim_deferred(p, ops);
2008     if (hook_status == AGENT_TASK_HOOK_PENDING &&

```

4.2.3. Nexus 会话与 Host Relay

Nexus 使用带序号的帧，把 Guest 的会话、工具事件、审批和运行指标接入 Host Relay。用户态解码器会校验会话、序号、类别和长度，再由中继程序完成 Provider 适配和本地端点转发。网络和模型协议留在 Host 侧，智能体身份、工具权限和任务状态仍由 Guest 和内核验证。

代码 34: Nexus V2 帧的编码入口

c

```

903 static uint nexus_next_child_task = NEXUS_CHILD_TASK_BASE;
904 static uint nexus_next_artifact_slot = NEXUS_FIRST_DYNAMIC_ARTIFACT_SLOT;
905 static int nexus_artifact_cleanup_failed;
906 static uint nexus_tasks_total;
907 static uint nexus_tasks_failed;
908 static uint nexus_artifacts_total;
909 static struct nexus_task_channel nexus_task_channel;
910 static char nexus_workspace_generation[LIVE_SHA_HEX_SIZE + 1];
911 static int nexus_workspace_catalog_ready;
912 static int nexus_workspace_watch_ready;
913 static int nexus_workspace_control_fid;
914 static uint64 nexus_workspace_watch_cause_sequence;
915 static int nexus_command_read_fd = -1;
916
917 static void nexus_copy_text(char *out, uint capacity, const char *text);
918 static const char *nexus_find_text(const char *text, const char *needle);
919 static int nexus_sha256_text_valid(const char *text);
920 static const char *nexus_workspace_operation_name(uint operation);
921 static int live_digest_text(const char *text,
922     char output[LIVE_SHA_HEX_SIZE + 1]);
923 static int nexus_remove_ephemeral_artifact(uint handle);
924 static int nexus_workspace_window_invalidate(void);
925 static int nexus_read_artifact(
926     uint handle, uint expected_kind, const struct nexus_identity *reader,

```

```

927     struct agent_nexus_artifact_header *header, unsigned char *payload,
928     uint capacity, uint *payload_size);
929 static const char *nexus_role_name(int role);
930 static void nexus_observer_worker(void *arg);
931 static int nexus_capture_self_snapshot(const struct agent_info *before,
932     uint64 control_id,
933     struct nexus_kernel_telemetry *out);
934 static int nexus_emit_self_snapshot(const struct agent_info *before,
935     uint64 control_id);
936 static struct nexus_task_event_wire *nexus_add_task_event(
937     struct live_tool_result_wire *result,
938     const struct nexus_identity *identity,
939     uint64 turn_id, uint64 request_id, uint64 corr_id,
940     uint task_id, uint parent_task_id, const char *event,
941     const char *state, int status, uint deadline_tick);
942
943 static void live_fail(const char *message)

```

代码 35: Host Relay 的串口写入与本地消息分发

Python

```

2154     proof: dict[str, object] = {}
2155     if self.guest_profile == "nexus":
2156         proof = {
2157             "generation": completion.generation,
2158             "provider": self.provider_name,
2159             "model": completion.model,
2160             "transport": self.provider_transport,
2161             "adapter_success": True,
2162             "request_sha256": completion.request_sha256,
2163             "raw_guest_request_sha256": completion.raw_guest_request_sha256,
2164             "history_bindings": [
2165                 dict(binding) for binding in completion.history_bindings
2166             ],
2167             "request_contains_user": True,
2168             "user_message_index": completion.user_message_index,
2169             "response_sha256": response_sha256,
2170             "user_content_sha256": turn.user_content_sha256,
2171             "user_bytes": turn.user_bytes,
2172         }
2173     if self.endpoint_origin:
2174         proof["endpoint_origin"] = self.endpoint_origin
2175     receipt = completion.reply.receipt
2176     if receipt is not None:
2177         proof.update(_provider_receipt_fields(receipt))
2178     if completion.reply.type == "final":
2179         proof["final_request_sha256"] = self._last_final_request_sha256
2180         self._last_final_corr_id = completion.corr_id
2181         self._last_final_response_sha256 = response_sha256
2182         self._last_final_provider_proof_sha256 = hashlib.sha256(
2183             relay.canonical_json_bytes(proof)
2184         ).hexdigest()
2185         proof["final_response_sha256"] = response_sha256
2186         proof["provider_proof_sha256"] = (
2187             self._last_final_provider_proof_sha256
2188         )
2189     public: dict[str, object] = {

```

```

2190         "type": "model_response",
2191         **envelope,
2192         **proof,
2193     }
2194     if completion.reply.type == "tool_use":
2195         public.update(
2196             {
2197                 "response_type": "tool_use",
2198                 "tool": completion.reply.tool,
2199                 "arguments": response["arguments"],
2200             }
2201         )
2202     else:
2203         public.update(
2204             {"response_type": "final", "content": completion.reply.content}
2205         )
2206     self._controller(public)
2207     if proof:
2208         self._telemetry(
2209             {
2210                 "event": "provider_result",
2211                 **envelope,
2212                 "status": completion.reply.type,
2213                 **proof,
2214             }
2215         )
2216     self._telemetry(
2217         {
2218             "event": "model_response",
2219             **envelope,
2220             "status": completion.reply.type,

```

测试运行还检查关闭、取消和过期后的清理过程。Host Relay 可以请求停止，Guest 把等待和工具事件转换为可观察的完成记录；内核则利用生命周期代次，阻止回收后的迟到 Task completion、工作区结果或订阅引用落到新的工作流。

4.2.4. 失效状态与恢复分支

测试不仅执行正常流程，也主动构造过期代次、错误 schema、资源不足、SQ/CQ 协议失步、实时查询缺口和失效 artifact 句柄。预期结果不是笼统的“调用失败”，而是由对应接口返回 STALE、DENIED、NO_SPACE、RETRY 等状态，并保持尚未提交的副作用不可见。任务通道在协议失步后要求显式 RESYNC；取消测试由同一活动生命周期内的 controller 使用 AGENT_TASK_DELEGATE_COMPLETE_F_REQUEST_CANCEL 和准确的 owner/channel/request/slot/task/correlation tuple 发起，同时检查 ORCHESTRATE、WAIT_CANCEL 与 TASK route。返回 OK 只确认 control request 已接收；任务已经 CLAIMED 时，provider 仍须清理预绑定结果并确认最新终态 offer。工作区 generation 变化后，Guest 通过 Typed Watch 使旧 Catalog 窗口失效并从 manifest cursor 0 重建。这些分支用于检查错误能否被上层识别和处理，而不是只验证正常路径能够跑通。

5. 总结与展望

5.1. 阶段性结论

本项目在 RISC-V uCore 中实现了内核中的智能体运行机制。AgentOS-uCore 以现有的进程、页表、VFS、系统调用和调度器为基础，把身份、工具、上下文、文件查询、事件等待、资源和调度纳入同一工作流。六组机制让智能体在发生副作用时都具有明确的身份、能力位、访问范围、代次、数据来源和资源账户。

项目取得了三方面进展。第一，Agent 已从应用层标签变为内核能够识别的进程身份，普通进程和 Agent 进程可以在同一个 uCore 中共存，高频 Context 数据也能从固定用户地址直接读取。第二，工具调用协议、V2/V3、上下文路径、实时查询、原生 Task Channel、资源账户和 EEVDF 贯通了 Agent 从调用、等待到取得资源服务的主要过程。第三，控制台和 Nexus 把这些内核机制用于科研恢复场景和三角色常驻协作；Nexus 直接复用 Guest Context、Metadata Catalog、Live Query、Typed Watch 与 delegated Task Channel，使确定性测试与模型驱动交互走同一条 Guest 路径。

实验展示了各条路径的性能特征。索引核心路径在 16 个独立配对中均取得更低延迟，配对加速比中位数为 3.118 倍；12 组文件目录网格的中位加速比介于 1.164–2.808 倍；504 条 EEVDF 唤醒探针均在 0–1 个 tick 内得到服务。端到端窗口中，索引路径相对遍历路径的中位差值为 +13.452 ms，说明核心窗口外的总开销抵消了索引收益；现有数据尚未把外围步骤逐段计时，后续应先定位主导项再优化。

下一阶段将沿三条主线推进：为实时查询补充分页游标，降低宽条件下反复收紧查询的成本；在长 Provider 延迟和密集事件下扩大压力测试，观察 Task Channel 与事件队列的背压；将工作流 EEVDF 扩展到 SMP 和真实 RISC-V 硬件，分析多核实体迁移、唤醒和资源记账。与此同时，完整流程会按准备、查询、写入、刷新和清理分别计时，先定位索引收益被外围开销抵消的具体环节，再决定优化顺序。

5.2. 当前技术限制

当前测试环境是 RV64 QEMU 单 Hart，工作流 EEVDF 的实体迁移和多核竞争尚未测量。实时查询单次最多返回 8 个结果，ABI 还没有分页游标；事件队列、Context 路径、任务通道和调度实体表也采用固定容量。现有负载以短时、可重复的 Guest 场景为主，尚未覆盖长 Provider 延迟下的大规模异步 backlog。调度观测使用 10 ms 的 tick，适合判断唤醒是否跨越节拍，不足以分辨微秒级差异。

5.3. 后续工作

下一阶段将补充分页查询和长时间压力场景，重点观察长 Provider 延迟、任务积压、订阅事件密集到达时的背压行为；随后把工作流 EEVDF 扩展到 SMP 和真实 RISC-V 硬件，分析实体迁移、唤醒和资源记账。性能测量还会进一步拆分场景准备、查询、写入、刷新和清理阶段，并使用更细粒度的时钟定位完整流程中的额外开销。

6. 参考说明

6.1. uCore 与系统基础

AgentOS-uCore 建立在 LearningOS/uCore 教学内核之上，继承 RISC-V 启动、进程、虚拟内存、VFS、异常处理、系统调用和基础调度框架。项目在这些现有对象上增加智能体状态和用户内核接口（UAPI），同时保持原有组件的职责边界。相关基础资料见参考文献中的 uCore 课程材料。

6.2. 协议、工具与 Runtime

结构化工具和任务通道使用定长结构、版本和大小字段、共享头文件以及 Guest 与内核布局检查。Nexus 的 Host Relay 管理本地套接字、TLS、Provider 适配器和帧转发；Guest 保存业务状态、artifact 以及与内核的交互。系统中的外部依赖及许可信息以仓库中的许可证和源码声明为准。

6.3. 测量资料

性能图来自 docs/figures/performance/ 的当前导出文件。测量目录保留原始串口输出、CSV 和逐样本来源信息，用于复核配对延迟、扫描组、任务操作序列和 EEVDF 唤醒探针。图表叙述依据这些保存的测量结果，不以模型推断替代原始记录。

6.4. 文献使用

系统设计参考操作系统、能力安全、结构化接口、事件驱动 Runtime 和公平调度等公开资料。

7. 参考文献

- [1] LearningOS Community. ucore-tutorial-code-2025s. <https://github.com/LearningOS/uCore-Tutorial-Code-2025S>, 2025. AgentOS-uCore 的教学内核基础.
- [2] Linux Kernel Documentation. Cpu accounting controller. <https://docs.kernel.org/6.11/admin-guide/cgroup-v1/cpuacct.html>, 2024.
- [3] Linux Kernel Project. rstat.c. <https://github.com/torvalds/linux/blob/master/kernel/cgroup/rstat.c>, 2026.
- [4] Linux Kernel Documentation. Bpf ring buffer. <https://docs.kernel.org/6.6/bpf/ringbuf.html>, 2023.
- [5] Linux Kernel Documentation. Eevdf scheduler. <https://docs.kernel.org/scheduler/sched-eevdf.html>, 2026.
- [6] Jens Axboe. Efficient io with io_uring. https://kernel.dk/io_uring.pdf, 2019.
- [7] Haiku Project. Practical file system design with the be file system. <https://www.haiku-os.org/legacy-docs/practical-file-system-design.pdf>, 1999.
- [8] Anthropic. How tool use works. <https://platform.claude.com/docs/en/agents-and-tools/tool-use/how-tool-use-works>, 2026.
- [9] Anthropic. How claude code works: The agentic loop. <https://code.claude.com/docs/en/how-claude-code-works>, 2026.